SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 1 - Lecture Transcripts

Lectures 1-13 | 13 lectures | 151 pages (source transcript pages 9-158)

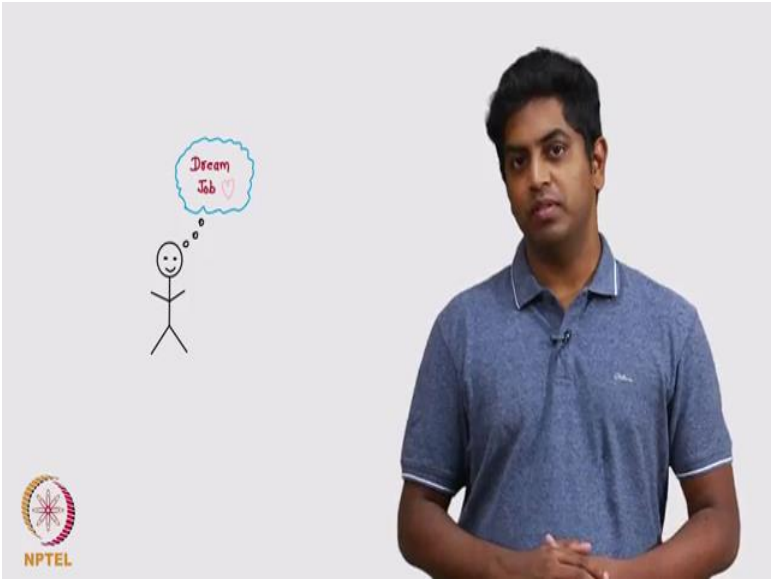
INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L01	Introduction	2
L02	Answer to the puzzle	13
L03	Introduction to Python-1	21
L04	Introduction to Python-2	45
L05	Introduction to Networkx-1	74
L06	Introduction to Networkx-2	85
L07	Social Networks: The Challenge	131
L08	Google PageRank	137
L09	Searching in a Network	140
L10	Link Prediction	143
L11	The Contagions	146
L12	Importance of Acquaintances	148
L13	Marketing on Social Networks	150

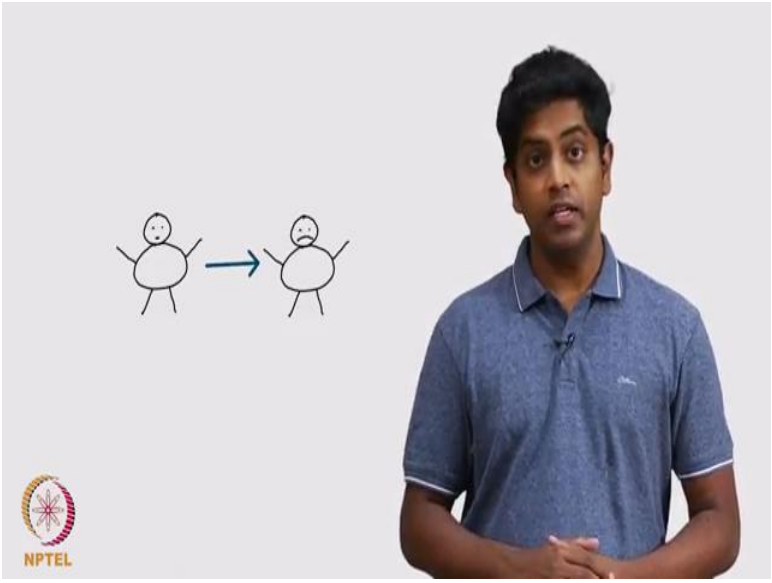
Social Networks
Prof S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 01
Introduction to Social Networks
Introduction

(Refer Slide Time: 00:16)

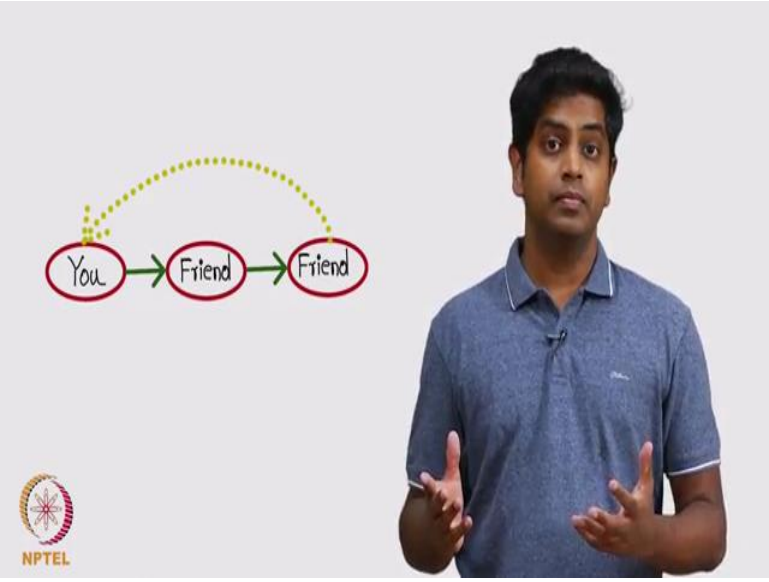


(Refer Slide Time: 00:20)

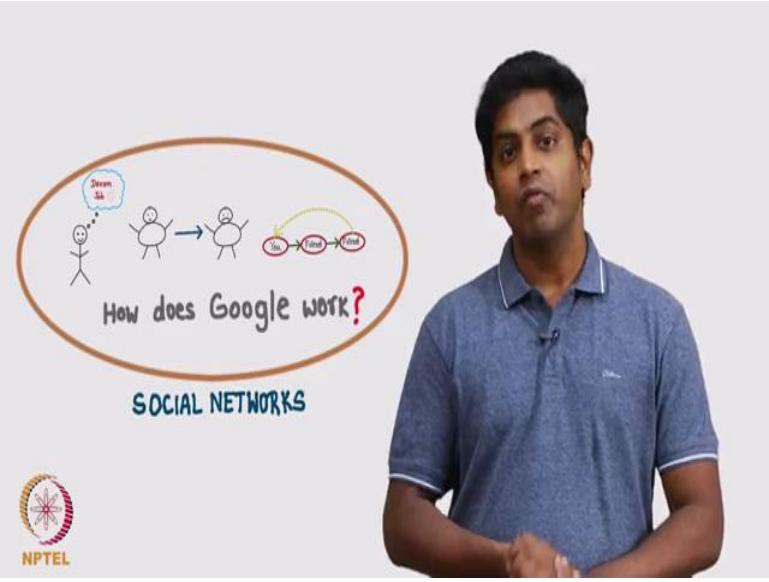


How do you find your dream job is obesity contagious? Do you think a friend's friend whom you do not know has any influence at all in your life?

(Refer Slide Time: 00:22)



(Refer Slide Time: 00:30)



And what has these questions to do with let say how google works? What are the commonalities across these questions? Are there any commonalities in the first place? Well yes, that is what makes the subject called social networks. So, we will be studying all these questions and more throughout the course without any further ado let us start off

with a nice question. So, we are going to watch a video clip right now, I am going to come back and then analyze what just happened in the video clip.

Hey.

Hey Ahmed, where were you? Class is about to start.

Leave it, do you even know Vardan and Simran are dating and for the Heaven's sake this is just the second day of our college.

How did you come to know about that?

You do not know? Everyone knows about it.

How is that even possible? Anamika told me about it yesterday and that too personally and I just told Harita about it.

Harita! Who is Harita? I do not know about her.

As far as I know she does not talk to many people here. It has been just one day since we have met and I am still wondering; how did you come to know about that?

Of course, not many of us know each other and we have met each other yesterday only and that too only few of us interacted.

I see some signs going around here.

Let me revise it. So, we are the bunch of people who have not met each other before and then met each other yesterday only and only few of us interacted and still.

And yet everyone knows about the news, Anamika told me personally.

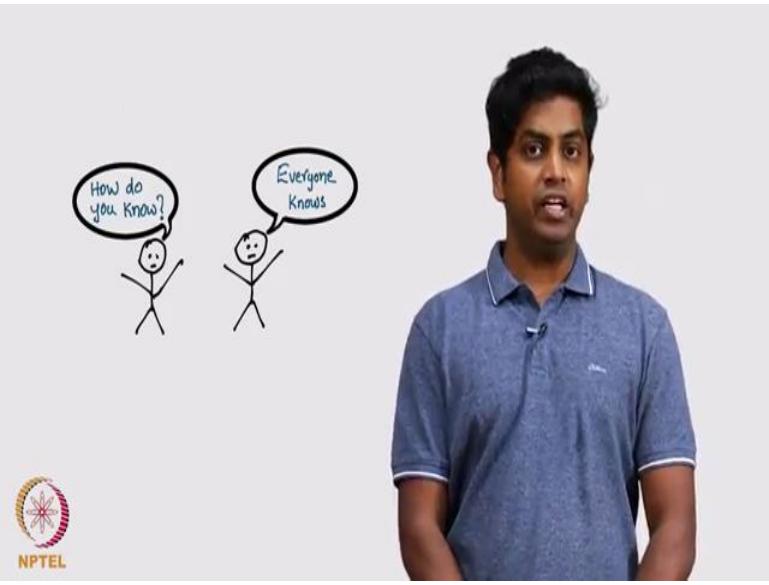
Indeed only few of us know each other still we are so connected.

(Refer Slide Time: 02:26)



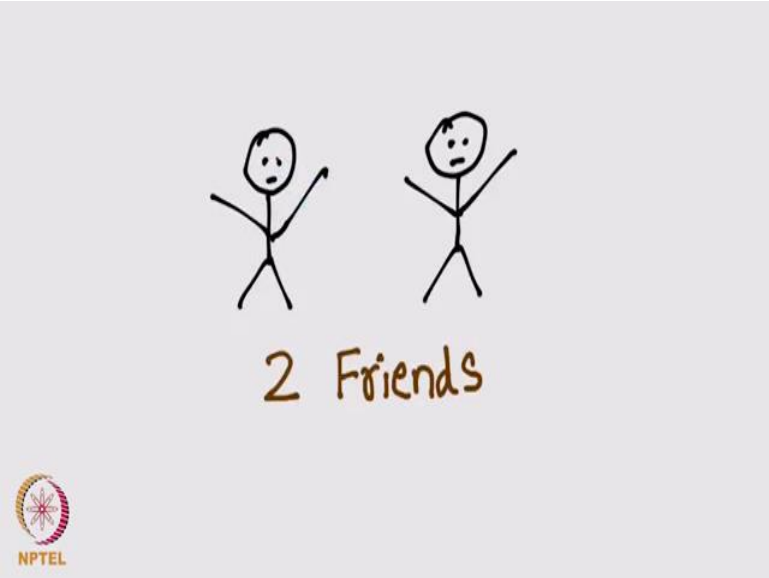
Did you see what just happened in the video clip, there were these 2 friends who newly joined this college and they are talking about a piece of gossip.

(Refer Slide Time: 02:37)



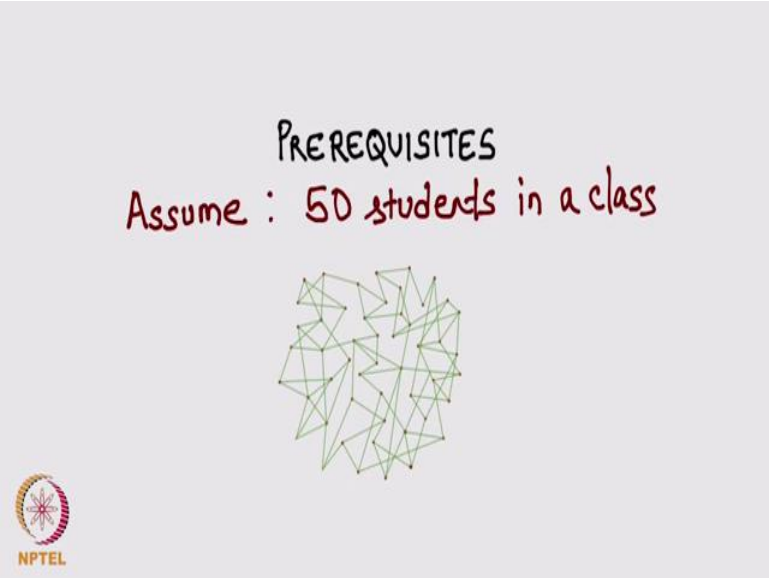
And a person is asking this question; how come you know of this while I thought nobody knows of this except me and in fact, that the person says the entire class knows of this what is so intriguing about this I actually find it intriguing for the following reason.

(Refer Slide Time: 03:11)



They are just less than one week into the class in the college and for everybody to know this piece of gossip they all must be friends with each other, is it not, without being friends with each other, who will come and tell each other about all these things? They may not even be Facebook friends this early, right. It has been less than a week since they have joined the college what is happening here, let us analyze.

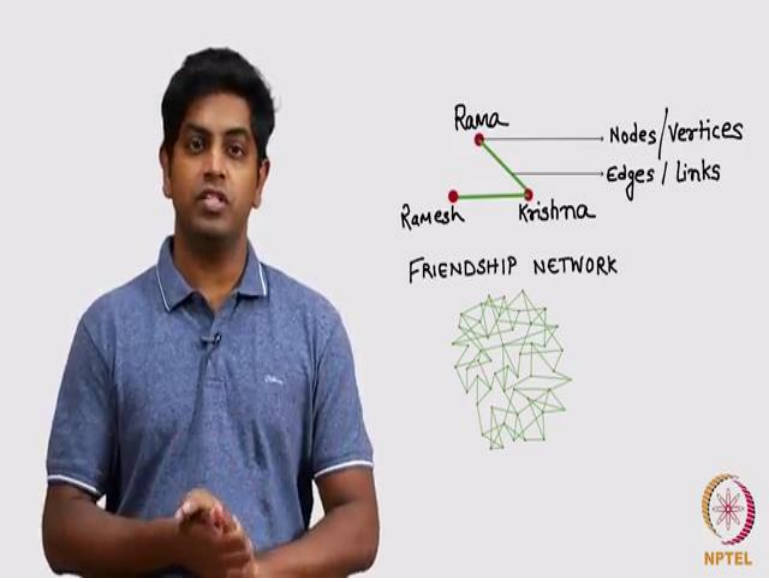
(Refer Slide Time: 03:30)



We need to develop a few pre requisites before we can answer this question. So, let me go slowly, the classroom let us say has some 50 people, 50 people may not be friends

with each other because as I told you it is just the first few days of the class, let me try modeling this these are the 50 friends let me use dots to denote these friends and the friendships between them let me draw a line to denote the friendship by this I mean.

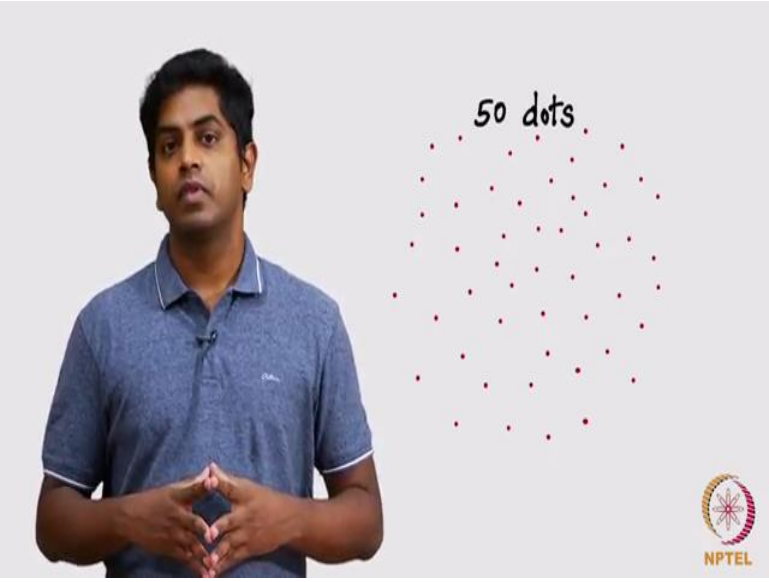
(Refer Slide Time: 04:09)



Assume Rama and Krishna are part of this classroom and they are friends with each other I put a line between them and Krishna and Ramesh are friends with each other I put this line between them and, but Rama and Ramesh are not friends. So, I do not put line between them and I develop the friendship network I call this friendship network.

It might look something like this some points denoting people and lines denoting friends. There are different ways in which people call this dots are called vertices or nodes in the subject and the lines are called edges or links. So, what we will be doing is we will we are going to use the jargon vertices or nodes for dots, edges or links for lines from now onwards throughout the course that is with the nomenclature.

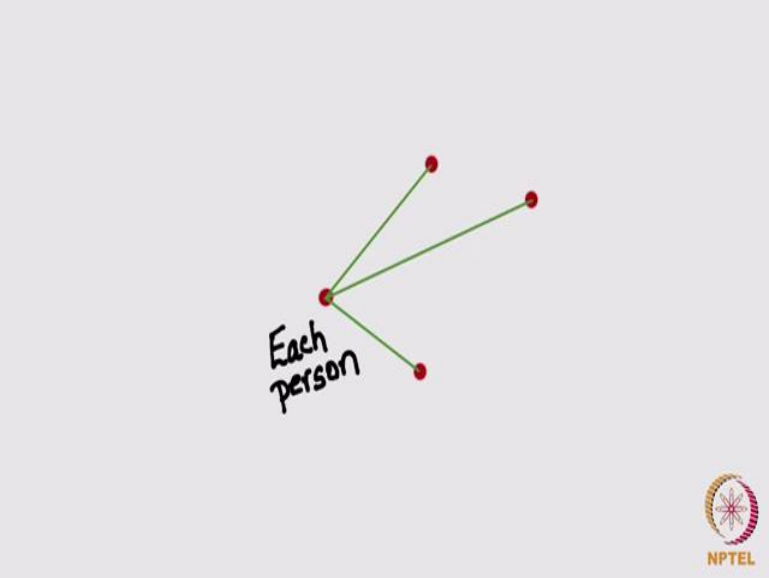
(Refer Slide Time: 05:11)



And now let me do a small experiment and observe what is happening here.

I will take 50 dots representing friends rather people in the class what I will do is let me take a guess first week of the program first week of the college and they are from different places from the country and it has only been first week.

(Refer Slide Time: 05:45)



So, let us say each person would have managed to have 3 friends on an average. So, what I will do I will take a person here and try to randomly pick 3 people and declare him as friends with these 3 people, I do it for every single person look at what is happening.

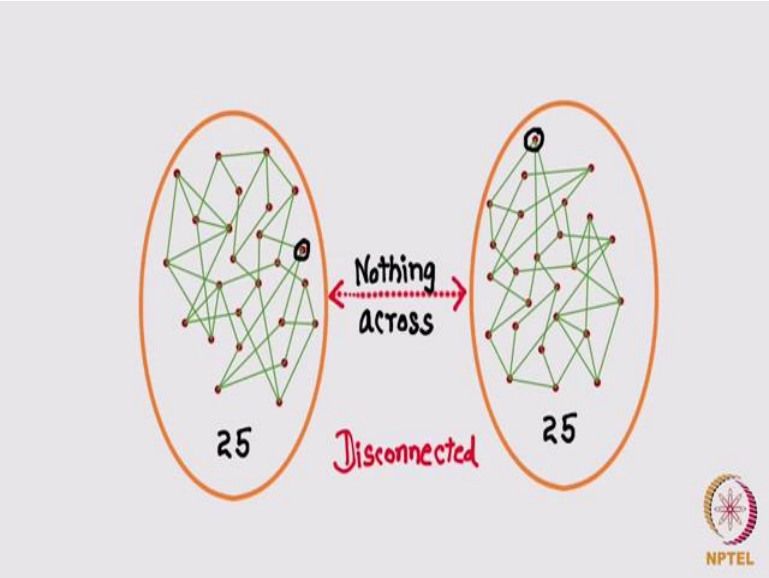
(Refer Slide Time: 05:58)



The final graph I am going to call it as structure a graph the final graph might look like this what is surprising about this graph? The surprising fact is that while a person can actually make all 50 people as his friends the entire class as his friends 49 to be precise excluding him. Let say he just makes 3 people as friends why because it is the beginning of the course and what do I observe here in this graph what is startling for me is that this graph this network is connected what do I mean by connected.

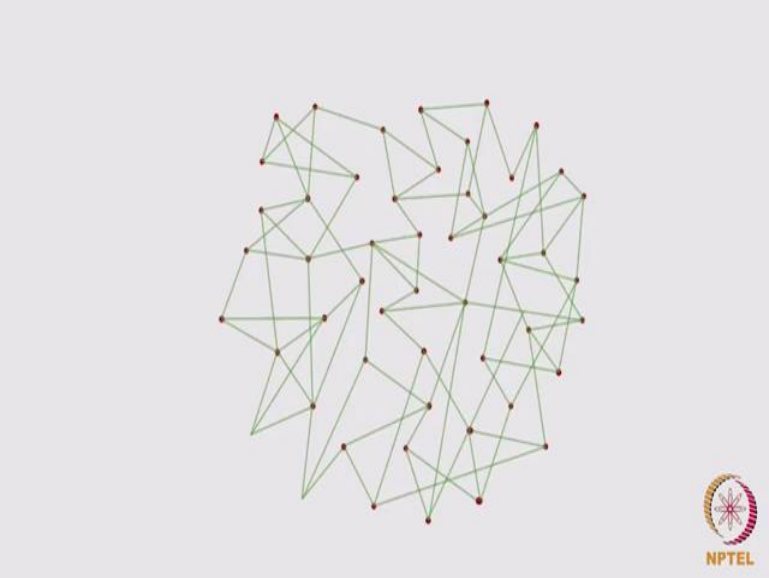
You see take any 2 people here in this graph there is a path that connects these 2 people this is strange is this always true not really.

(Refer Slide Time: 07:00)



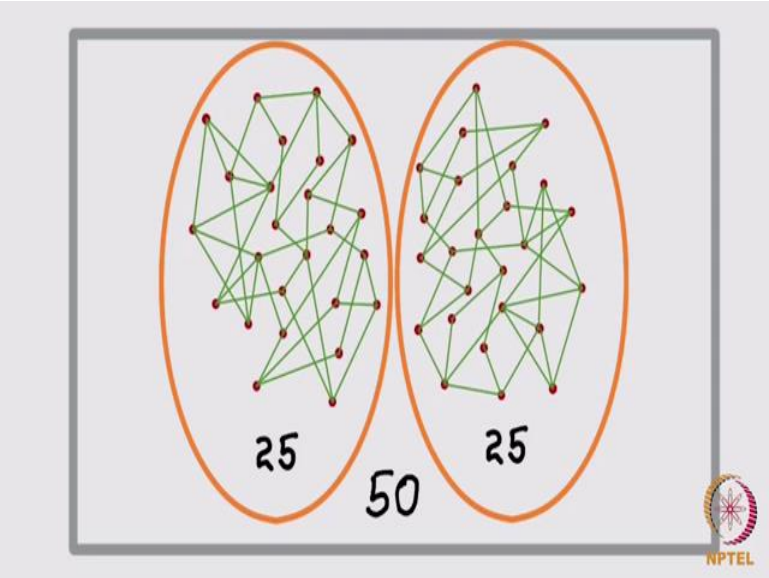
Observe this particular example where there are 50 people, the 20 people, this side 25 people that side they have some friendships within, but there is no friendships across this might also happen in such a case I do not call this graph connected.

(Refer Slide Time: 07:32)



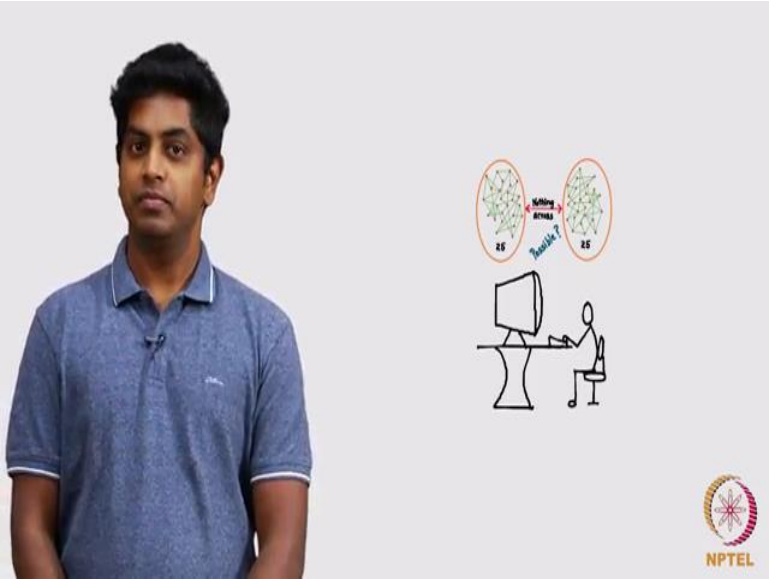
Why because I can take a person from this end and a person from that end there is not path connecting these 2 people, but then look at my previous graph on an average if I make 3 friends per person by picking these 3 people uniformly at random I observe that the graphs gets connected is this always true.

(Refer Slide Time: 07:47)



So, let us look at this network 25 people this side, 25 people that side as I told you and this side there is a network of connections and that side there is a network of connections remember here is a classroom of 50 people and there is a bifurcation of 25 this side, 25 that side.

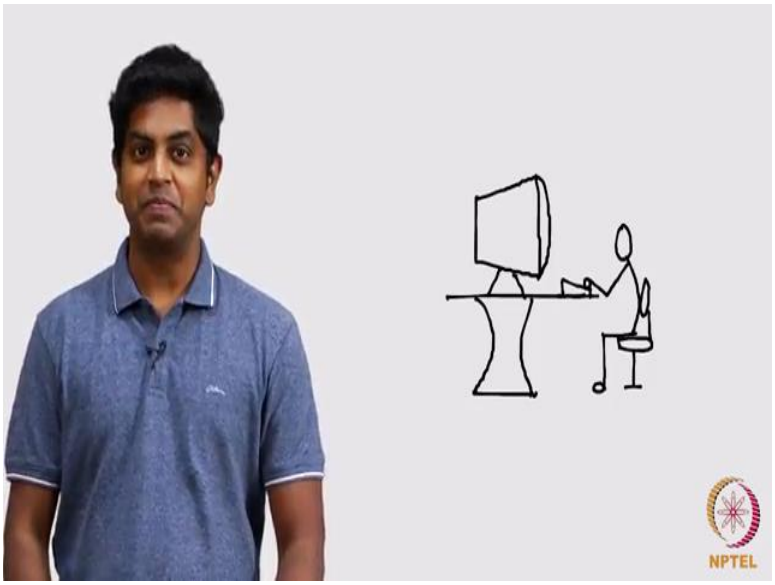
(Refer Slide Time: 08:11)



What do I mean by bifurcation? I mean if you look at the friendships between these people 25 people have friendships within them, 25 people have friendships within them

that side and a point to note is there is no friendship across my question was is this even possible.

(Refer Slide Time: 08:46)

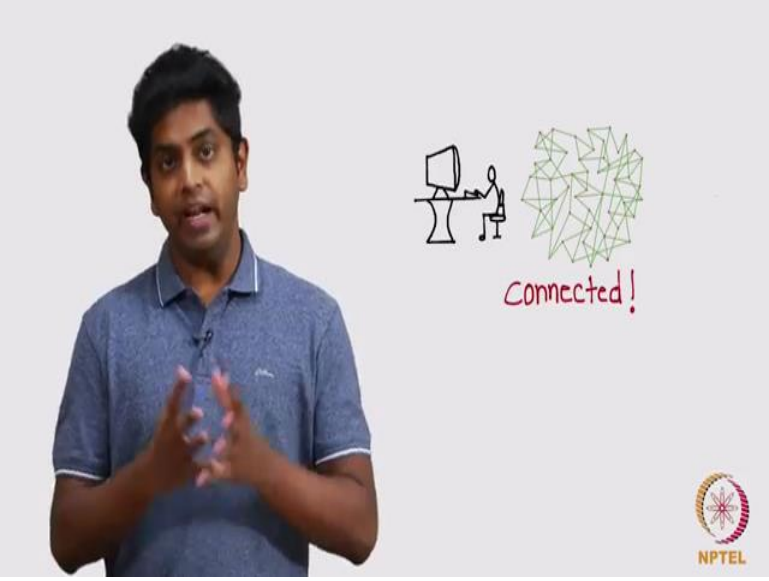


So, let me do one thing, I will now take a break, write a piece of program and get back to you people and tell you my observation of course, I will not show you the program, I will go do the program and came back and tell you the output of the program.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

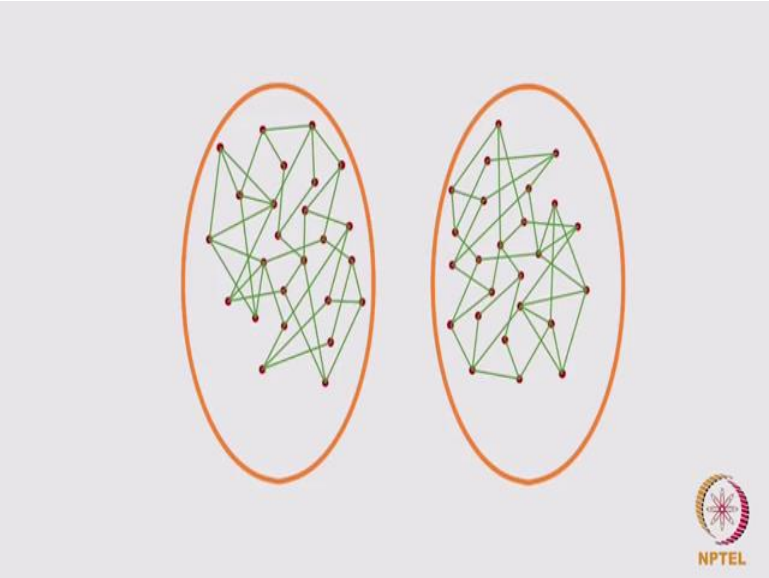
Lecture - 02
Introduction to Social Networks
Answer to the puzzle

(Refer Slide Time: 00:05)



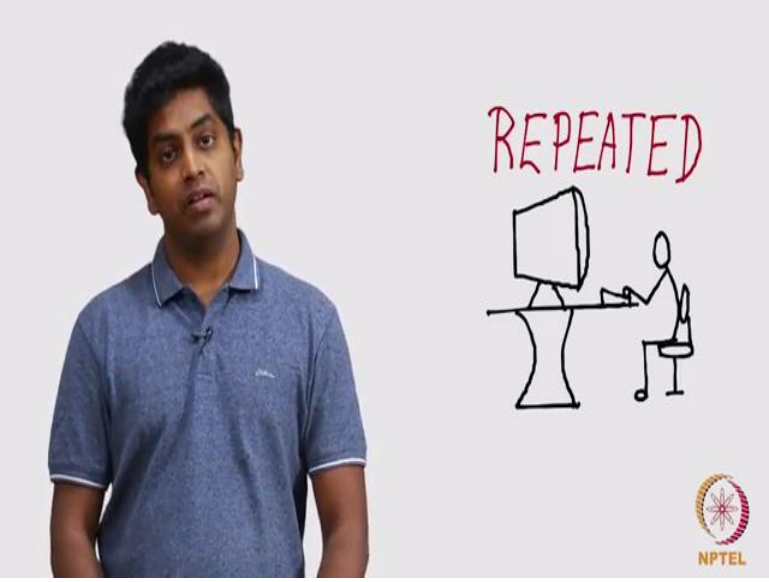
Fine, I am back. So, I did this piece of programming, what did I do? I took 50 dots, for each dot I randomly chose 3 people.

(Refer Slide Time: 00:21)



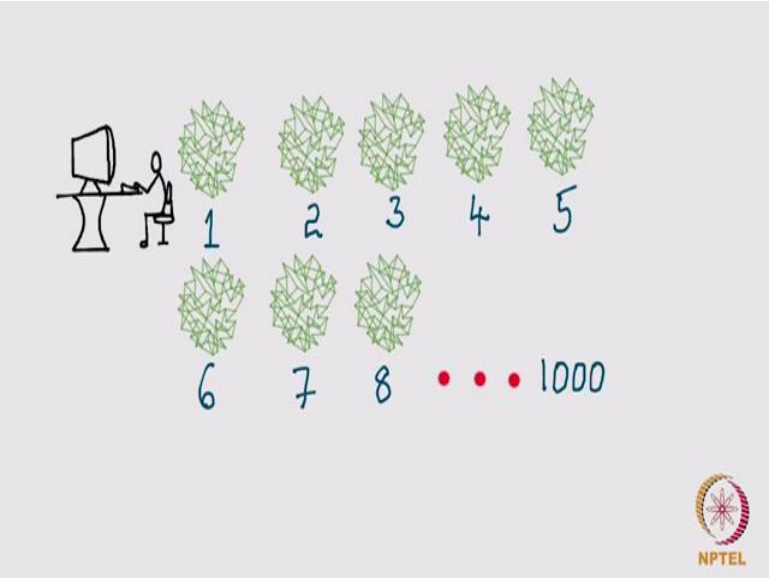
Whatever I told you just a while back I did I choose 3 people uniformly at random per person and made them friends. I did this for everyone and I saw the resultant network, it was connected by that I mean I did not see 2 components like this.

(Refer Slide Time: 00:45)



I did not see this at all, I only saw this. So, what did I do? I repeated my experiment, what do I mean by repeating the experiment?

(Refer Slide Time: 00:49)



Anyway I take a person and make 3 friends uniformly at random for every person, it turned out to be connected for the first time, I repeated this experiment for the second time, I saw it was connected, third time, connected, fourth connected, fifth, sixth, seventh, eighth, I repeated this a 1000 times my program always said it was connected.

(Refer Slide Time: 01:28)



Now, there are 2 possibilities possibility one is something is wrong with my programming I probably I am a bad programmer there is a error in the code bug in the code or there is something happening here. So, I am sure you people are confident with

the instructor that he knows how to write a piece of code. So, probably it is not the first one it is indeed the second one what is happening here let us recollect the question.

If you take a network of let say 50 people with each person having just a few friends the network turns out to be connected unbelievable whys is this true think for a minute.

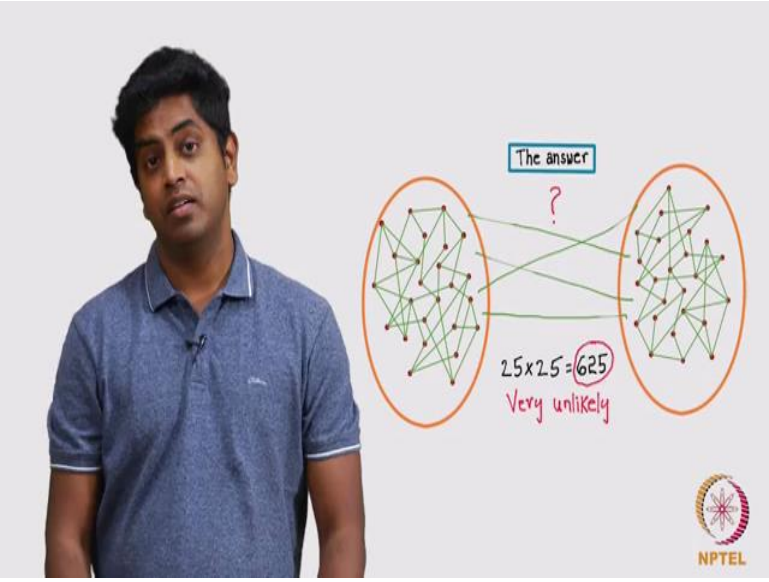
(Refer Slide Time: 02:07)



Well I can tell you the answer, the answer is pretty simple let us get back to this 25; 25 example.

The graph can have this kind of 2 partitions a class with 50 people can have such two partitions or maybe even three partitions or four partitions and so on.

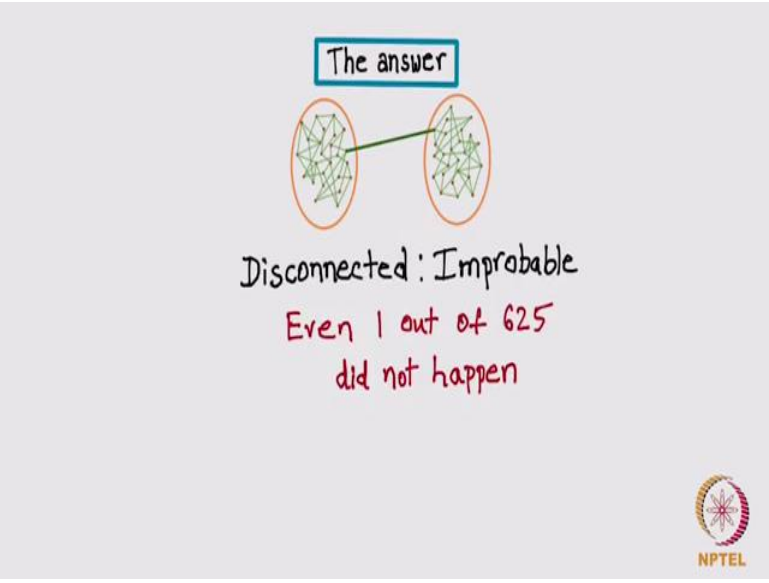
(Refer Slide Time: 02:34)



I think that is impossible why because 2 partitions if it exists 25, 25, 20, 30, whatever two partitions do you see what are the possible friendships between these 2 partitions 25, this side each one has the possibility of making 25 different friends and he chose not to fine maybe he does not like these 25 people.

Another person from this group had 25 possible friendships to make he chose not to so on, so on and so on. So, what do you observe there were roughly 25 into 25 possible friendships here that did not happen how likely is this.

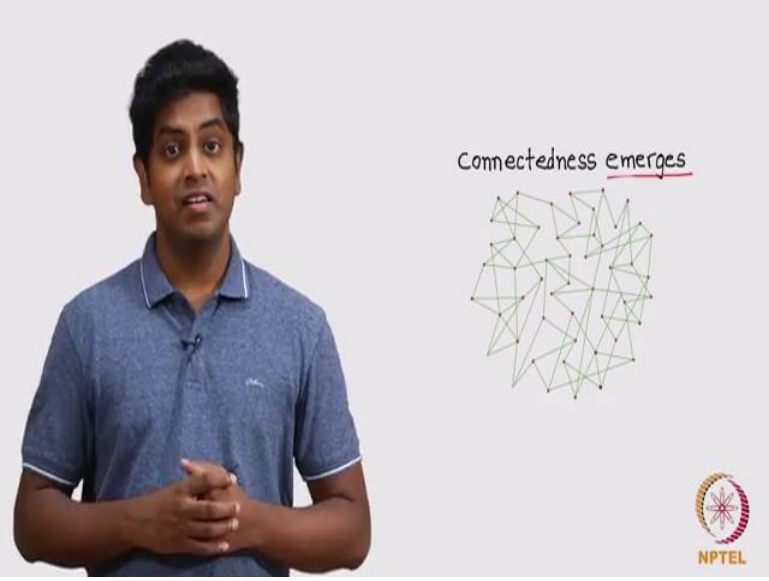
(Refer Slide Time: 03:23)



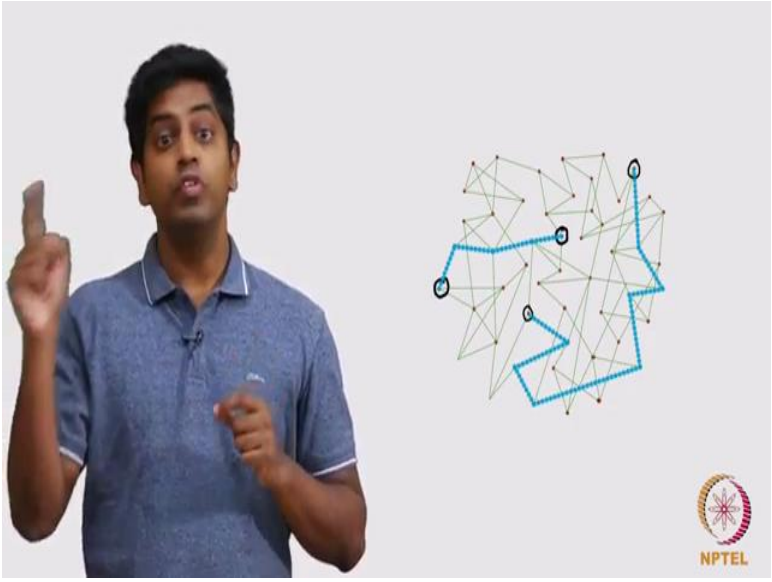
Intuitively very unlikely by that I mean even if one person on this side had friends, let us say a friend on that side over the graph would not look like this it would look like this with one connection, it would not have 2 clusters, 2 clusters would become one cluster out of 625 possible friendships even if one friendship got executed it will not have led to this bifurcation which means we all now we have developed this intuition that bifurcation is improbable.

Although possible with a very small probability I do not worry if you do not know about probability you just need to have this intuition that there are 625 possibilities out of which even one possibility did not get executed that is a very improbable event correct.

(Refer Slide Time: 04:29)

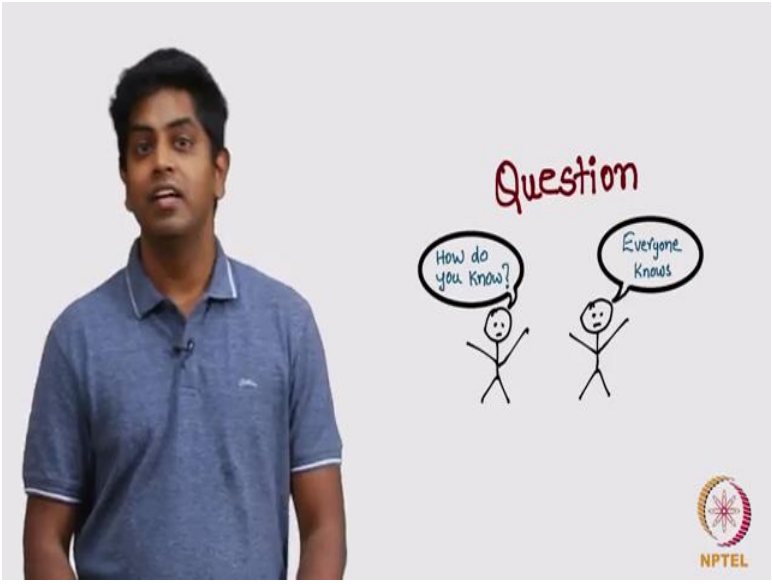


(Refer Slide Time: 04:38)



And that is the reason why in a classroom with 50 people there emerges what is called connectedness by that we mean any 2 people have a path between them which means let us get back to the question that we asked.

(Refer Slide Time: 04:43)



Remember the clip, 2 friends talking to each other and they were wondering how come everyone knows this a piece of juicy gossip that is because even with a fact that the class is only one week old, they are just into the college still with each person having 2 to 3 friends, the graph becomes connected and a piece of gossip. Such as this that that are

friends are discussing can spread like wild fire and if it starts from one node it can reach to all nodes simply because it is connected and not disconnected.

Social Networks
Prof S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

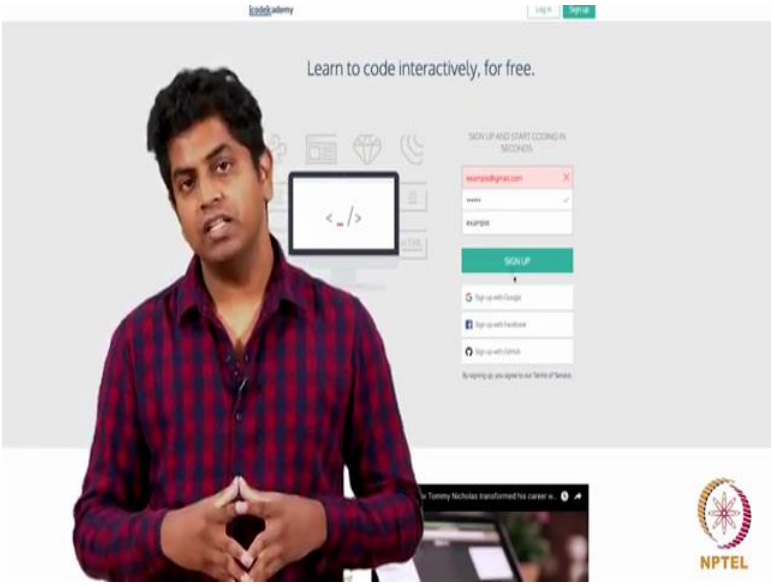
Lecture - 03
Introduction to Social Networks
Introduction to Phyton-1

Hi, in this module, we will be introducing you to python. Python is a very powerful programming language yet very simple to understand, very easy to use and it is an open source package. It is a scripting language which has a whole lot of APIs online and it is very easy for you to install these APIs and work with python this is one major reason why I thought you will use python for our network analysis related work social network analysis related work.

So, in this session I will be showing you some quick ways of learning python; quick things that you must know about python and you can get started. It is not required that you undergo a complete course in python and then start using python 3-4 things and you will and you are on you can start using python.

So, my personal recommendation, my personal favorite is this code academy dot com where you can go create a user name then start learning python, it has very nice exercise unless you solve the exercise, you cannot go further and these exercises are in increasing difficulty.

(Refer Slide Time: 01:00)



I strongly suggest that you finish a few lessons there and get yourself familiarized with python.

And or you can just look at what we do on the screen cast which I am going to show you right now and learn python parallely. Now a word on installing python, if you are using o one 2 the process is pretty straight forward on MAC again it is again very easy to install.

(Refer Slide Time: 01:41)

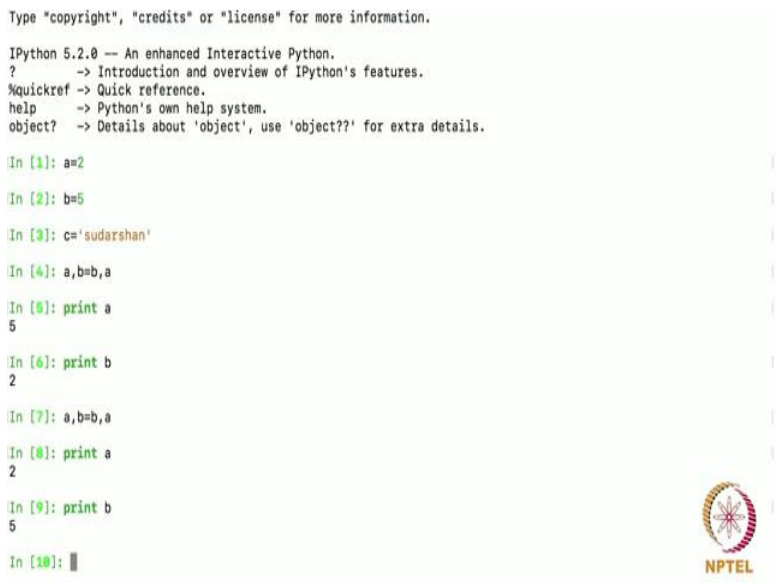


But if you are using windows, I would suggest that you download what is called anaconda which is a big package which includes all the scientific programming

requirements using python and once installed I think you are all done, but one o one 2 one MAC it is a very straight forward process.

For our demonstrations we are using a MAC machine and this is the terminal that I have opened on a MAC machine. So, what I do is I type ipython. So, this ipython is interactive python which opens a shell like this where I can easily type the code.

(Refer Slide Time: 02:15)



A note on IPython it is very easily installable on o 1 to 1 MAC, but on windows as I told you need anaconda in which ipython is a built-in package.

In ipython let me try showing you a few things a = 2, this is basically python with a interactive shell and then let say b = 5. So, unlike c you do not have any declarations that are required for variables you just say a equals to b equals 5 and it assigns a to 2 and b to 5 and if I say c equals let say Sudarshan. Now c will be string type variable automatically and look at the ease with which I can swap to variables a comma b is equal to b comma a and then if I print a and print b the values are swapped, let me swap them once again back to normal if I print a it will be 2 if I print b it will be 5.

We will now see how one can use for loop while loop and if loop in python. It is pretty straight forward I will still give you an illustration I would suggest that you try getting your hands on with these things so that you get familiarized with it, so a for loop works like this.

(Refer Slide Time: 03:41)



Say for i in range 10 I say print I, it is simply a prints i from 0 to 9, this is how a for loop works, a while loop also works in a similar way if I say k equals let us say 10 and I say while k is less than or equal to 20 do print k and then increment k just starts k with 10 and goes on till 20.

(Refer Slide Time: 04:13)

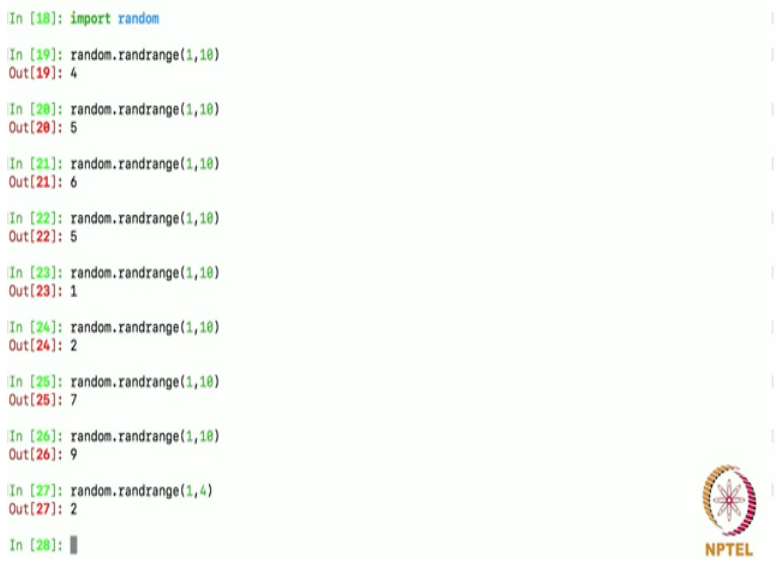


So, the value of k now is 21, what I will do is I will illustrate if loop now using this k value, if k is greater than 20 which by the way is the case k's value seems to be 21, I will say print the value of k is greater than 20.

So, this is true. So, it displays the value of k is greater than 20. So, if I say if k is greater than 25 then print something something I am illustrating the usage of else here I will say else, print the statement is incorrect is false let say is false let say the statement is false right this is how if else loop works. So, we now just now say how for loop works while loop works and if loop works; so just that you people play around with it so that you get familiarized with it.

A big advantage of using a scripting language like python is it comes with a whole lot of library functions I am now going to show you one such library function which will be using a whole lot throughout our case.

(Refer Slide Time: 05:41)



Let me use a library function called random this is how you input if you load the library function to your memory, you say import space random and then in this function you can call in this library you can call a particular function let say rand range one comma 10 what does this do this simply gives you a random number from 1 to 10, let me execute this command once again.

(Refer Slide Time: 06:39)



You press the up arrow in ipython shell and the entire things comes press enter, you will get another random number from the range 1 to 10 again you see I am getting random numbers from 1 to 10.

(Refer Slide Time: 06:44)



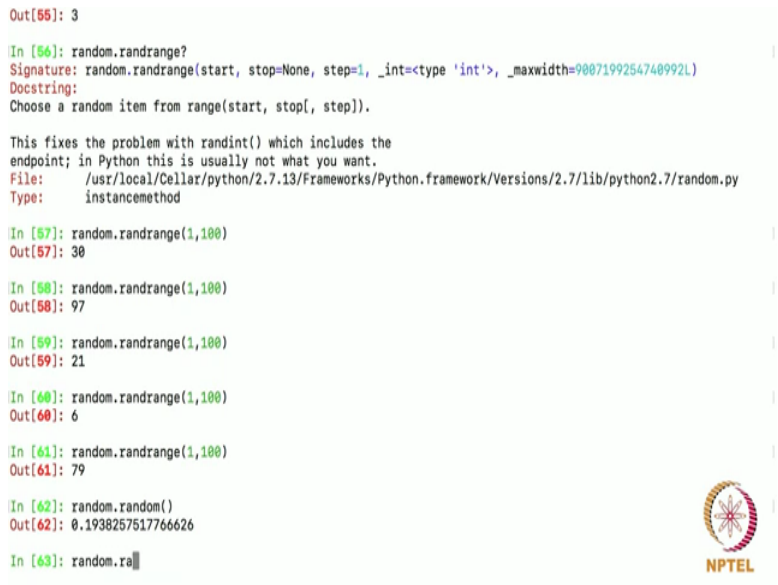
Let me now change this instead of 10, I will make it 4, this will give you random numbers from 1 to 4, 1, 2, 3 and so on.

(Refer Slide Time: 06:53)



Yeah, you will get 4 2 as you keep trying or maybe it is only till yeah it is only till. So, you will not get 5 you will only get the number just before. So, random dot rand; rand range I do not know what it does. So, what I do is random dot rand range question mark.

(Refer Slide Time: 07:11)



This will tell me everything of what it does as you can see choose a random item from the range start till stop the stop is mostly one before the numbers specified here.

So, if I say random rand range one comma hundred it will give me a random number from 1 to 100, in case I say random dot random once again this is another library

function which gives me a random number between 1 0 to 1 see it gave me a random number between 0 to one let me type that once again.

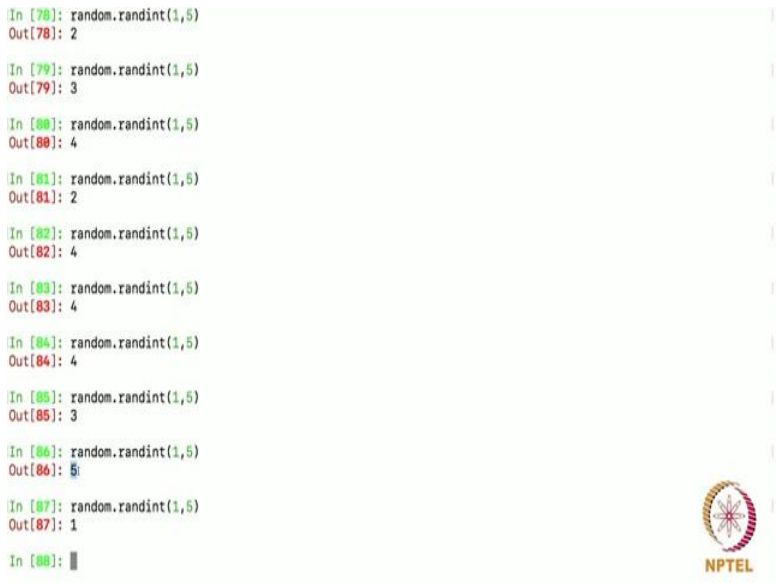
(Refer Slide Time: 07:49)



Another random number between 0 to 1, another random number between 0 to 1 so on and so forth. So, basically when I am doing this I am pressing my up arrow and pressing enter.

Up arrow just repeats the previous command all right if you want to see what all you can do with the random library function.

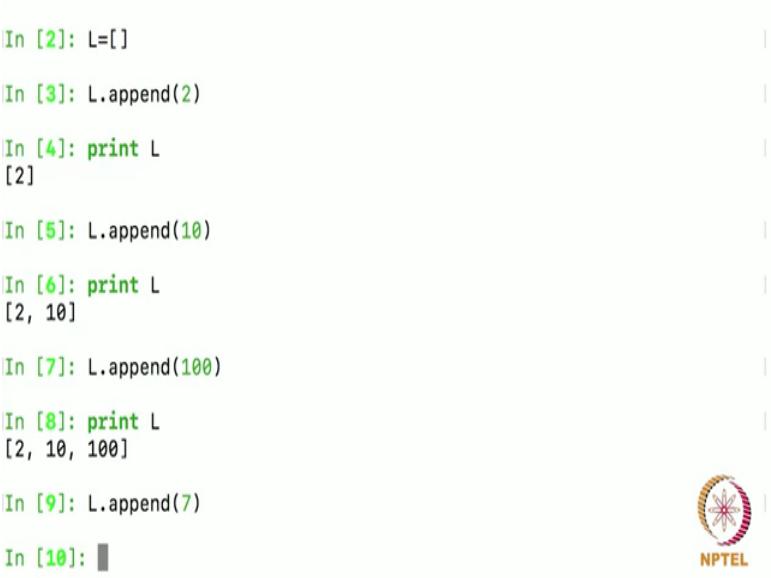
(Refer Slide Time: 08:02)



Just put a dot after random and press tab you will get all possibilities and you can use the arrow mark to go around and see what does what and there is a whole lot of functions written here.

Let us look at yet another function random rand int. So, what does it do put a question mark you will get to know that it returns a random integer in the range a comma b including both end points. So, let us try that random rand int 1 comma 5 gives you 2 so on and so forth. Somehow strangely 5 is not coming, yeah 5 came right now. So, it is between 1 to 5, but rand range is different as you saw before.

(Refer Slide Time: 08:58)



We will now look at what are lists in python, lists in python as I said are very similar to arrays and we will see how to use them. So, initially what I will do right now is I will declare lists called L, this is how you declare it L equals open bracket close bracket and then you say L append you put some number 2 print L, L will have that number 2, right, Now let me say L append 10, now to the existing list L, the number 10 gets appended if you say print L, we will get 2 comma 10 so on, I keep adding and then if I print you will see 2, 10, 100 and let us say I will append 7 print L, 7 gets appended so on and so forth.

(Refer Slide Time: 09:49)

```
In [10]: print L
[2, 10, 100, 7]

In [11]: L.append(77)

In [12]: L.append(14)

In [13]: print L
[2, 10, 100, 7, 77, 14]

In [14]: L.sort()

In [15]: print L
[2, 7, 10, 14, 77, 100]

In [16]: L.reverse()

In [17]: print L
[100, 77, 14, 10, 7, 2]

In [18]:
```



Best part is you can append whatever you want including character string; character string and whatever you want any data type now look at his let me add more numbers to this 77 L append 14 and then print L you have these numbers.

Sorting this is very easy simply say L dot sort and it will sort L as you see L is now sorted if you want to reverse L you simply say L dot reverse see what happens to this see it got reversed descending order.

(Refer Slide Time: 10:37)

```
In [17]: print L
[100, 77, 14, 10, 7, 2]

In [18]: L.remove?
Docstring:
L.remove(value) -- remove first occurrence of value.
Raises ValueError if the value is not present.
Type:      builtin_function_or_method

In [19]: L.remove(77)

In [20]: print L
[100, 14, 10, 7, 2]

In [21]: import random

In [22]: for i in range(100):
    ...:     L.append(random.random())
    ...:

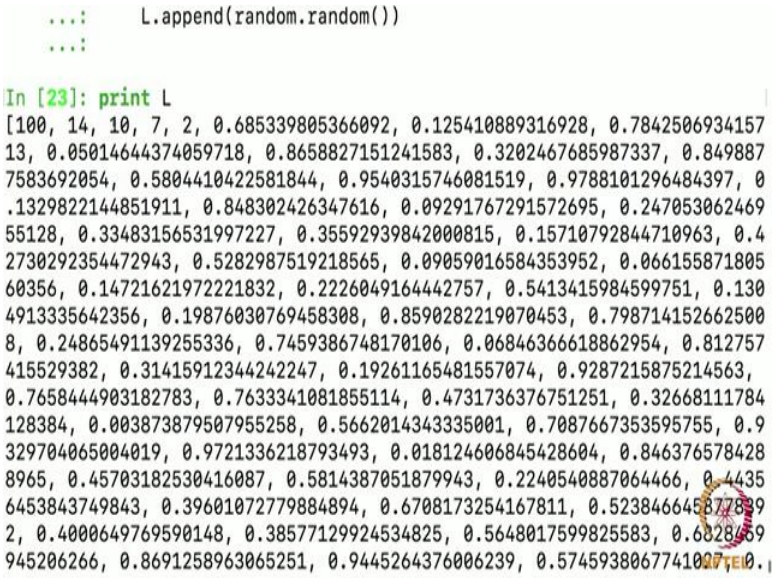
In [23]: print
```



What I can you do with this sort and reverse is what I told you what all can you do simply put a L dot and then press tab key and you will see all possibilities append count extend index insert pop remove. So, if you want to know what remove does type remove and put a question mark and it will tell you remove first occurrence of the value which means if I say L remove 77 it will remove 77 from the list as you can see 77 is here in this list let see if it removes or not I have removed 77 print L yes 77 is removed now.

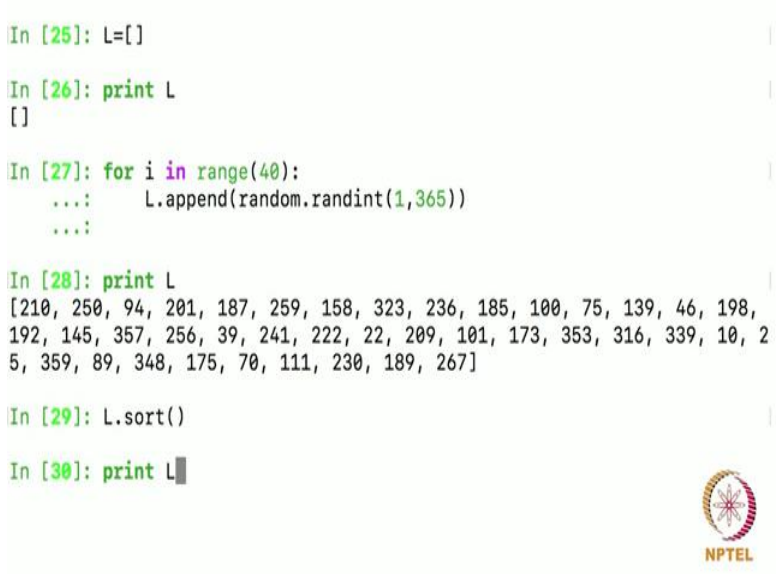
So, this is about list let me type something non trivial and show you what all you can do with the list we use random function rights. So, let us import random library function and then I say for I in range hundred L append random.

(Refer Slide Time: 11:39)



So, which means it appends 100 random numbers to L and L you see a lot of random numbers that apart from the initial 114 10 7 2 that we appended, we now have 100 random numbers appended.

(Refer Slide Time: 11:58)



So let us do a small exercise let me clear the screen this is the command list for clearing the screen what does this do else I will reinitialize l. So, that whatever was present when L is lost, now I will do for I in range 40 L append random rand int 1 comma 365, what am I doing here? I am picking data of birth of a person let say if he was born on February first then you assign the number 32 to him if he was born on January fifth, you assign the number 5 to him. So, based on which day he was born you assign a number from 1 to 365 to that person pick forty people and assign this date of birth randomly to each person.

The point I am trying to make is I am I have now created a list L which comprises of 40 different birthdays dd, mm, day, month, only not the year, 40 different birthdays of people, let me see how it looks like print L has 40 different numbers from 1 to 365 let me sort this let me print this.

(Refer Slide Time: 13:16)

```
In [30]: print L
[10, 22, 25, 39, 46, 70, 75, 89, 94, 100, 101, 111, 139, 145, 158, 173, 1
75, 185, 187, 189, 192, 198, 201, 209, 210, 222, 230, 236, 241, 250, 256,
 259, 267, 316, 323, 339, 348, 353, 357, 359]

In [31]: L=[]

In [32]: for i in range(60):
    ...:     L.append(random.randint(1,365))
    ...:

In [33]: print L
[249, 279, 351, 214, 354, 207, 3, 142, 278, 364, 37, 268, 336, 136, 72, 2
11, 274, 337, 337, 358, 61, 343, 15, 330, 188, 271, 297, 318, 317, 363, 3
31, 202, 12, 147, 152, 139, 225, 74, 132, 103, 76, 153, 140, 261, 191, 25
3, 143, 85, 143, 327, 179, 305, 33, 338, 119, 39, 274, 300, 226, 224]

In [34]: L.sort()

In [35]: pri
```



So, you see there are all these numbers and there are no repetitions I do not see any repetition here correct there are no repetitions perfect.

Now, what if I were to do the same thing like say I reinitialize for I in range 60 I say L append random rand int 1 comma 365 print l. So, I will again sort this.

(Refer Slide Time: 13:59)

```
In [35]: print L
[3, 12, 15, 33, 37, 39, 61, 72, 74, 76, 85, 103, 119, 132, 136, 139, 140,
 142, 143, 143, 147, 152, 153, 179, 188, 191, 202, 207, 211, 214, 224, 22
5, 226, 249, 253, 261, 268, 271, 274, 274, 278, 279, 297, 300, 305, 317,
318, 327, 330, 331, 336, 337, 337, 338, 343, 351, 354, 358, 363, 364]

In [36]: L=[]

In [37]: for i in range(60):
    ...:     L.append(random.randint(1,365))
    ...:

In [38]: print L
[287, 232, 271, 173, 74, 83, 239, 150, 147, 199, 351, 206, 281, 315, 301,
 154, 178, 246, 307, 103, 197, 152, 18, 75, 65, 53, 318, 285, 148, 160, 2
19, 91, 301, 218, 311, 140, 338, 168, 30, 89, 344, 298, 353, 178, 128, 36
3, 173, 206, 311, 241, 233, 164, 187, 287, 276, 249, 306, 307, 297, 303]

In [39]: L.sort()

In [40]: print L
```



So, that I can check for repetitions if an I will see if there is anything repeated 3, 12, 15, 33, let us see if there is any repetition I do not see a repetition yeah I see a repetition look at this 143 is picked twice, 153 and so on is there any other repetition I do not see any

other repetition 274 is another repetition correct and then any other repetition that you can spot let me check 3; 3, 7 is another repetition there are 3 repetitions. So, fine there were 3 repetitions, but is it coincidental. So, what do I do?

Let me repeat this experiment L equals initialize once again for say for I in range 60 L append random rand int 1 comma 365 then you print L it is; obviously, not sorted. So, you sort it and then print L you will see for 40.

(Refer Slide Time: 15:00)

```
19, 91, 301, 218, 311, 140, 338, 168, 30, 89, 344, 298, 353, 178, 128, 36
3, 173, 206, 311, 241, 233, 164, 187, 287, 276, 249, 306, 307, 297, 303]

In [39]: L.sort()

In [40]: print L
[18, 30, 53, 65, 74, 75, 83, 89, 91, 103, 128, 140, 147, 148, 150, 152, 1
54, 160, 164, 168, 173, 173, 178, 178, 187, 197, 199, 206, 206, 218, 219,
 232, 233, 239, 241, 246, 249, 271, 276, 281, 285, 287, 287, 297, 298, 30
1, 301, 303, 306, 307, 307, 311, 311, 315, 318, 338, 344, 351, 353, 363]

In [41]: print L
[18, 30, 53, 65, 74, 75, 83, 89, 91, 103, 128, 140, 147, 148, 150, 152, 1
54, 160, 164, 168, 173, 173, 178, 178, 187, 197, 199, 206, 206, 218, 219,
 232, 233, 239, 241, 246, 249, 271, 276, 281, 285, 287, 287, 297, 298, 30
1, 301, 303, 306, 307, 307, 311, 311, 315, 318, 338, 344, 351, 353, 363]

In [42]: len(L)
Out[42]: 60

In [43]:
```



Picking there was no repetition for 60 there was I am repeating the experiment to see if it is true that you will always get repetitions there is a repetition once again once again 173, 173, 178, 178, second repetition right and then 206, 206, another repetition any other repetition let see 287, 287, third repetition, 301, 301, 4 repetitions in this case 301 and 301, excellent, 307, 307, 5 repetitions 6 repetitions, 311 and so on, correct.

So, what did we just observe we observed that by just taking list L with how many entries? Let us say the; I just printed L before let me again print it. So, there were len of L tells you the length of L number of elements in L. So, there are 60 elements because of a for loop it is 60 elements and if you pick randomly some 60 people you will have couple of them sharing their birthdays this is always true almost always true with a high probability when you pick 60 people from randomly and ask them for their birthdays you will have 2 people with the same birthday, what does that mean?

You go to a classroom randomly with some 60 students and ask them for their birthdays you will see 2 people with the same birthday in that class walk in to any classroom, this is almost always true, this is called the birthday paradox this is not a command by the way in python I am just writing it to illustrate it. So, a birthday paradox you can just google for it for more information a very interesting fact that if you pick some 40 to 60 people 40, it is rare, 60 definitely if you pick definitely you will observe 2 people with the same birthday we just took a code and observed it right now like.

That is with less as you would have seen the course homepage the prerequisites for this course is simply a first course in programming I suppose you all have studied programming in one form or the other one of the most powerful facility that you get in programming is your ability to write functions and python has a very easy way in which one can write functions we are going to illustrate that right now.

(Refer Slide Time: 17:41)

```
In [109]: def sumup():
     ...:     a=random.random()
     ...:     b=random.random()
     ...:     return a+b
     ...:

In [110]: sumup()
Out[110]: 1.1412489051246113

In [111]: sumup()
Out[111]: 1.6141913467879667

In [112]: sumup()
Out[112]: 0.5679386404761081

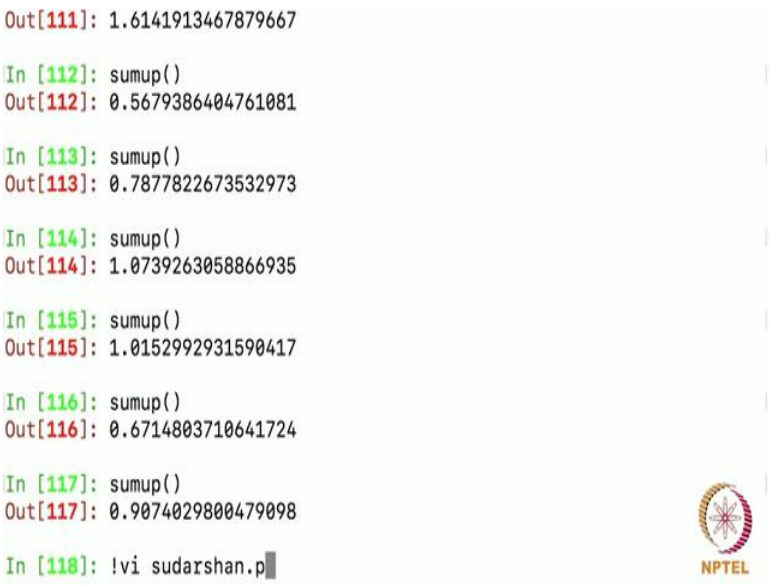
In [113]: sumup()
Out[113]: 0.7877822673532973

In [114]: sumup()
```

Now, demonstrate how we write functions in python it is a pretty straight forward process let me try writing a function on the shell itself first and then I will show how to write functions in a file. So, it is done the following way you say d e f define and then you will say sumup and then a colon open bracket close bracket and a colon and then you say let say a equals random which means assign a random number from 0 to one to a and then b equals same random assign a random number to b and then I return a plus b enter. So, what does it do when I invoke sumup it just shows me the answer please note that the

answer is more than one here that is because a and b are 2 random numbers between 0 to 1, there is some can beyond one let me invoke sumup once again what will happen picks 2 random numbers between 0 to 1 add them and then display it so on, perfect.

(Refer Slide Time: 18:58)



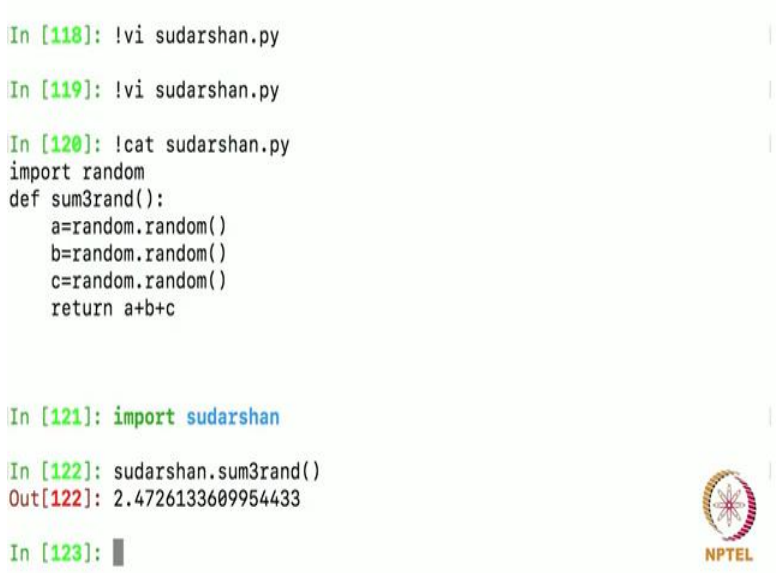
So, what you can do is whatever command whatever function definition you wrote right now you can actually open a file and then type it as well let me do that I am opening my favorite notepad I mean favorite editor which is vi editor. So, how do I invoke it I use exclamation vi space let say file name say sudarshan dot py.

(Refer Slide Time: 19:30)



This opens a new file you can use your own favorite text editor I use my I use vi editor note that exclamation stands for the here.

(Refer Slide Time: 19:41)



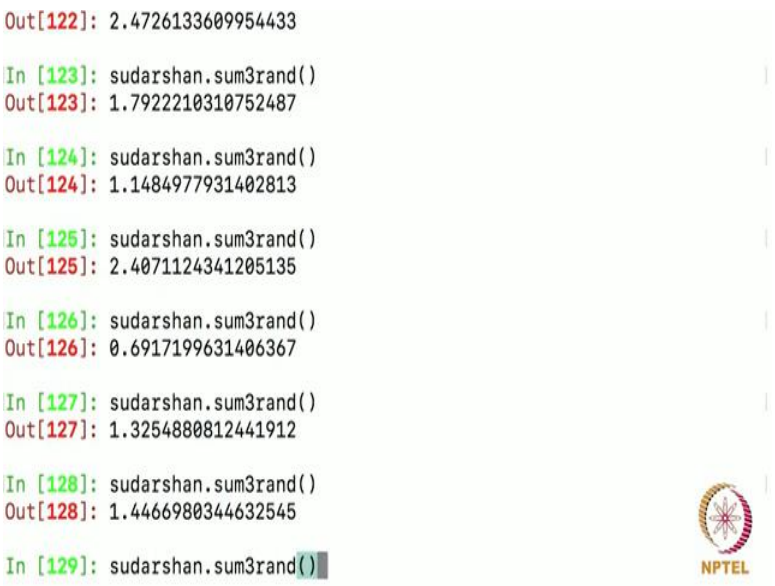
When you say exclamation it stands for outside command and then editor name and I type my file name you can even go outside this terminal and then open the text editor and then type it there also no problem.

So, let me define something here define sum 3 rand which is sum in 3 random numbers how do I do it I should first import random without that I cannot use the random function here in a file a equals random random b equals random random c equals random random and then I return a plus b plus c and then I come out its now saved I will display the contents of sudarshan dot py using the cat command of Linux it will just display the contents of the file you see that this much is there please note I have put import random here otherwise this will not work this was not the case with the defined thing that I wrote here I straight away used it that is because I had imported random already perfect.

Now, what do I do how do I invoke the sum 3 rand that is written inside a file by name sudarshan dot py that is pretty simple. So, what I do is I say I use the same import command when I say import sudarshan the sudarshan library whatever is contained in sudarshan gets loaded basically when I say import sudarshan import random happens and a function sum 3 rand gets defined with a following statements and now I can invoke it

see how I can invoke it I say simply say sudarshan dot sum 3 rand and that is it there you are random number between 0 to 1 added thrice.

(Refer Slide Time: 21:54)



Let me type it once again sudarshan dot sum 3 rand again gives you the same where sum 3 rand gives you some random number between 0 to 3 it is not really random because its picking a random number from 0 to 1 adding it thrice, this is actually not uniformly at random if you know probability you will know that, but that is not our concern here we are just observing that it is generating a random number thrice and adding it and then displaying it.

(Refer Slide Time: 22:18)

```
Out[129]: 1.768291238275506

In [130]: sudarshan.sum3rand()
Out[130]: 1.6856580370369079

In [131]: sudarshan.sum3rand()
Out[131]: 0.7293873918240366

In [132]: sudarshan.sum3rand()
Out[132]: 1.7983099756881937

In [133]: sudarshan.sum3rand()
Out[133]: 1.4387419167035427

In [134]: sudarshan.sum3rand()
Out[134]: 1.0149422968898556

In [135]: sudarshan.sum3rand()
Out[135]: 1.9272455647807818

In [136]: sudarshan.sum3rand(
```



The point is you see what is happening here sudarshan is the name of the file and then in that I am invoking sum 3 rand and I am displaying it.

(Refer Slide Time: 22:41)

```
 1 import random
 2 def sum3rand():
 3     a=random.random()
 4     b=random.random()
 5     c=random.random()
 6     return a+b+c
 7
 8 def sumkrand(k):
 9     ans=0
10     for i in range(k):
11         ans=ans+random.random()
12     return ans
13
14
15
~
~
~
~
~
:
```



Now, let me further edit sudarshan dot py, how do I do that vi sudarshan dot py and let me define another function here define sum k rand what does that mean add k random numbers now how do I do that I take k as a parameter here and then I say for i in range k you have had some programming experience that is the pre requisite for this course you know what I am doing? I will say answer is equals 0 and then I will add to answer plus

random, random that is it let me go inside I must return this, I am sorry, I came out without returning it. So, I say return ans.

So, what have I done here k is the parameter here as we can see and then I have initialized the answer to 0 and then I am running a for loop k number of times that is what this means for i in range k means assign value 0 one to k up to k minus one to I whatever is assigned to I, it does not matter to me here I; all I want is I want to execute this loop k times this particular thing k times. So, answer equals answer plus random dot random simply picks random number from 0 to one and adds into the existing answer and assigns the new value to answer and finally, I come out of the; for loop and the return answer. So, what does this do? So, let me see let me save this and then come out.

(Refer Slide Time: 24:09)

```
In [136]: sudarshan.sum3rand()
Out[136]: 1.1629215697772501

In [137]: !vi sudarshan.py

In [138]: !vi sudarshan.py

In [139]: import sudarshan

In [140]: !vi sudarshan.py

In [141]: reload(sudarshan)
Out[141]: <module 'sudarshan' from 'sudarshan.py'>

In [142]: sudarshan.sumkrand(10)
Out[142]: 4.575307276426816

In [143]: sudarshan.sumkrand(10)
Out[143]: 5.2256366698481544

In [144]: sudarshan.sumkrand(100
```



Now, please note import sudarshan will actually not work what you should do is whenever you have opened a file editor and you have added something new and then saved and come out you should not use import you should use reload sudarshan. So, sudarshan is reloaded in to the memory with the new updates a new function that I wrote just now and then you can open the new file what is that sum k rand and then I say ten. So, what should this output 10 random numbers sum if I say hundred instead of 10 you see hundred instead of 10 it will add hundred times random numbers from 0 to 1.

(Refer Slide Time: 24:57)

```
In [147]: sudarshan.sumkrand(100)
Out[147]: 52.737544589525804

In [148]: sudarshan.sumkrand(2)
Out[148]: 1.0681717586350272

In [149]: sudarshan.sumkrand(1)
Out[149]: 0.8563130656430317

In [150]: sudarshan.sumkrand(1)
Out[150]: 0.03881258471291482

In [151]: sudarshan.sumkrand(1)
Out[151]: 0.6739417080580347

In [152]: sudarshan.sumkrand(1)
Out[152]: 0.5814914963040022

In [153]: !vi sudarshan.py

In [154]:
```



And then output the answer yeah, perfect, if I just say 2 if I just say one it is as good as a random function correct. So, what did we just now learn you learnt the following?

(Refer Slide Time: 25:22)

```
Out[154]: 2.572630972476484

In [155]: sudarshan.sumkrand(5)
Out[155]: 2.563732366891008

In [156]: !vi sudarshan.py

In [157]: !vi sudarshan.py

In [158]: reload(sudarshan)
Out[158]: <module 'sudarshan' from 'sudarshan.py'>

In [159]: sudarshan.sum3rand?
Signature: sudarshan.sum3rand()
Docstring:
This function takes no input, but outputs the sum of 3
random numbers picked between 0 and 1
File:      ~/nptel/sudarshan.py
Type:      function

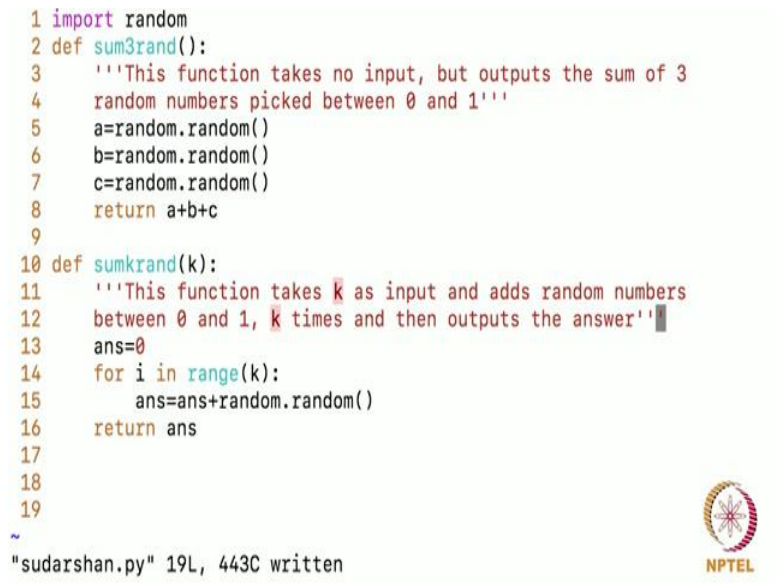
In [160]:
```



You can create a file and then write down your functions there and then invoke it from outside how do you invoke just say sudarshan dot followed by the name of the file sum 2 sum 3 rand is what we declared without only a parameter it generated a random number by picking a random function thrice and we also saw we also wrote a function by name

by sum k rand with an input parameter let say 5 which took 5 random numbers added and then displayed.

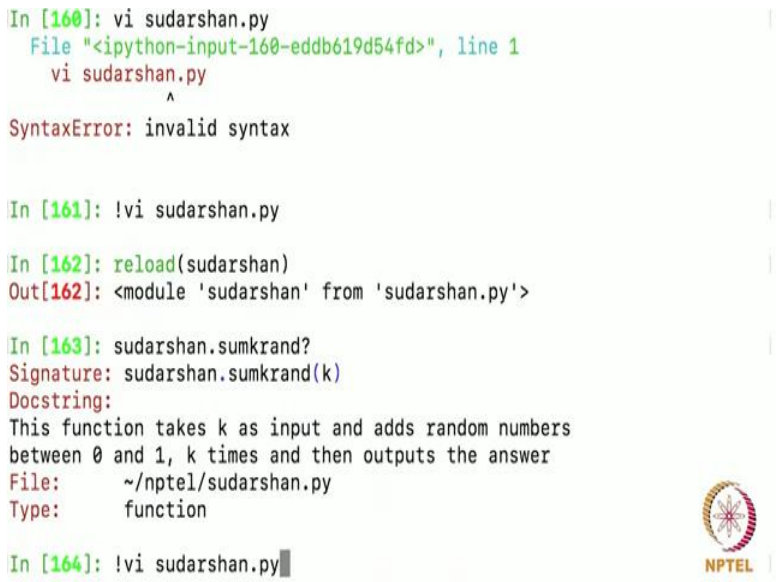
(Refer Slide Time: 25:56)



One important tip open the editor an important tip you can come here and then write your favorite help one liner you can say this function takes no input, but outputs the sum of 3 random numbers picked between 0 and one and then close it I will see what happens let me show you I am going out saved you just saw what I did I opened the editors sudarshan dot py and I have added this thing here what did I do 3 single quotes followed by some random text and then I close the 3 single quotes and I came out there is some change to the file. So, I should say reload sudarshan.

Now, when I when I want to execute some command in sudarshan I type sudarshan and then I put a dot and then tab is a very important key when you press a tab here it will show you all the functions that are available in sudarshan random is available because random is imported there right sum 3 rand sum k rand is available if you put sum 3 rand which by the way is a function that I just defined and then put a question mark guess what happens.

(Refer Slide Time: 27:20)



It shows you your help file that you just now wrote you see this; this function takes no input the output the sum of 3 random numbers picked between 0 and one who wrote this we wrote this just now correct.

Let me write as similar help file for the other command too. So, what do I do vi sudarshan dot py and then I am sorry I forgot to put an exclamation vi sudarshan dot py without exclamation the outside commands do not work vi is an outside command I come here what does this do let me just state that here this function takes k as input and adds random numbers between 0; between 0 random numbers between 0 and 1 k times and then outputs the answer towards the single code save you see what I did I just wrote a help function help one line for sum k rand function and then I go out and then I reload my sudarshan file now I say sudarshan sum k rand and then question mark it shows me the herewith whatever you have typed comes here.

Now, why is this even used this is used because when you write hundreds of functions. So, here we wrote only 2 functions what if I write hundred functions here in this file and I would not know what function is doing what.

(Refer Slide Time: 29:22)

```
In [163]: sudarshan.sumkrand?
Signature: sudarshan.sumkrand(k)
Docstring:
This function takes k as input and adds random numbers
between 0 and 1, k times and then outputs the answer
File:      ~/nptel/sudarshan.py
Type:      function

In [164]: !vi sudarshan.py

In [165]: !vi sudarshan.py

In [166]: import sudarshan

In [167]: reload(sudarshan)
Out[167]: <module 'sudarshan' from 'sudarshan.pyc'>

In [168]: sudarshan.sum3rand()
Out[168]: 2.333830823621529

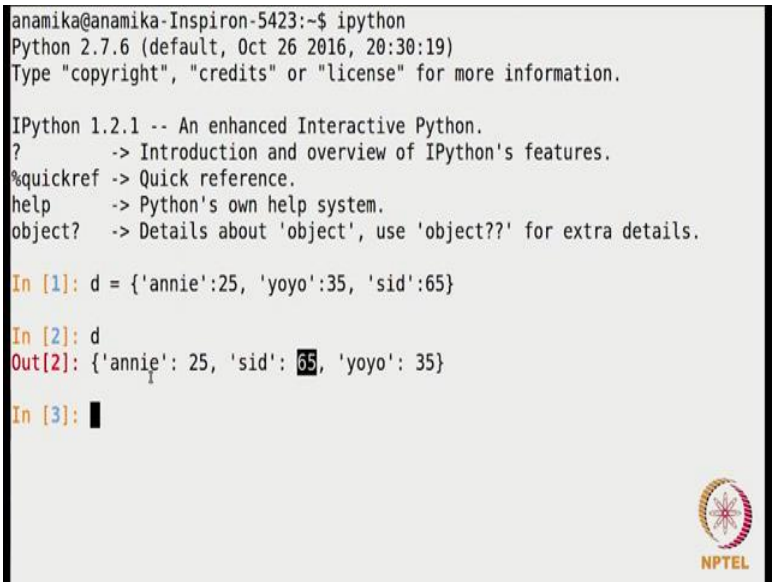
In [169]:
```

So, it is important for us to write down what a function is doing in the form of help file that with the introduction to functions and you know how to define a function on the ipython terminal you also know how to define how to how to create a file edit the file and then type the functions there and then come out of it and then invoke it by saying. Firstly, import sudarshan and then second time when you edit something and then you want to reload it you may say reload sudarshan note reload is used when you edit the file and reload means update the contents of sudarshan, I have just now changed it and then what you do is you say sudarshan dot sum whatever is the name of the file name of the function and it executes that function; however big; however, complicated it executes the function and shows you the answer that is with functions.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 04
Introduction to Social Networks
Introduction to Phyton-2

(Refer Slide Time: 00:08)



Hi everyone, in one of the previous videos you would have seen how we can make use of list data structure and we saw various functions that we can use with list. In this video we are going to look at yet another data structure that python provides and that is dictionary. So, we are going to see how we can create a dictionary and we are going to look at various functions that we can use with it. Before we get started; let me tell you in which situations, it is ideal to use a dictionary data structure.

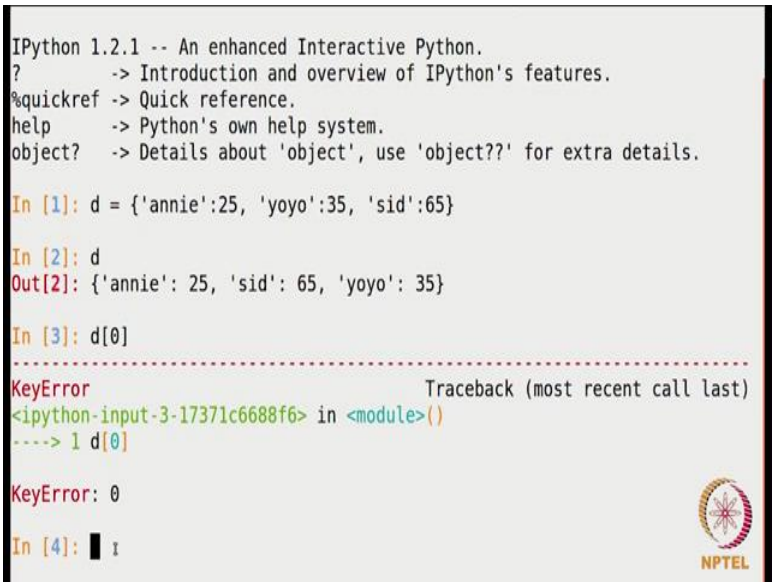
So, dictionary is used when we have to store elements that have some sort of attribute attached with them; that is we have some elements and we also have to store some sort of property along with every element. In other words, in dictionary every element has two things; that is key and value, so there will a key value pair for every element, where these values stores the attribute of the key. Another thing to note is that dictionary is stored in memory as hash tables, which makes it faster than as compared to other data structures.

So, if you have some million entries and on one side you store them in the form of list and on the other side you store them in the form of dictionary, accessing these elements through the dictionary will always be faster. This is because accessing elements in dictionary does not go in sequential manner. So, let us get started and I believe you people might be comfortable with ipython, so I will continue using that.

Now, let me create an example dictionary here; assume my aim is to store name the few people and along with that I also want to store the age of these people. So, here I have this age which is an attribute that is attached to every person. So, here I have two things the key is the name of the people and value is their age, so this is the sort of information that I want to store; let me create an example dictionary here. So, I am going to add a few people's names and their age, let me add few elements here; so I add another element and I add another element alright. So, this is how we create a dictionary, so you note here that every element has two things; so this is the name which is the key here and this is the age which is the value here. So, this is the key value pair and we separate them with comma and we add another element and so on and the elements are kept inside these basis; so you can print this dictionary.

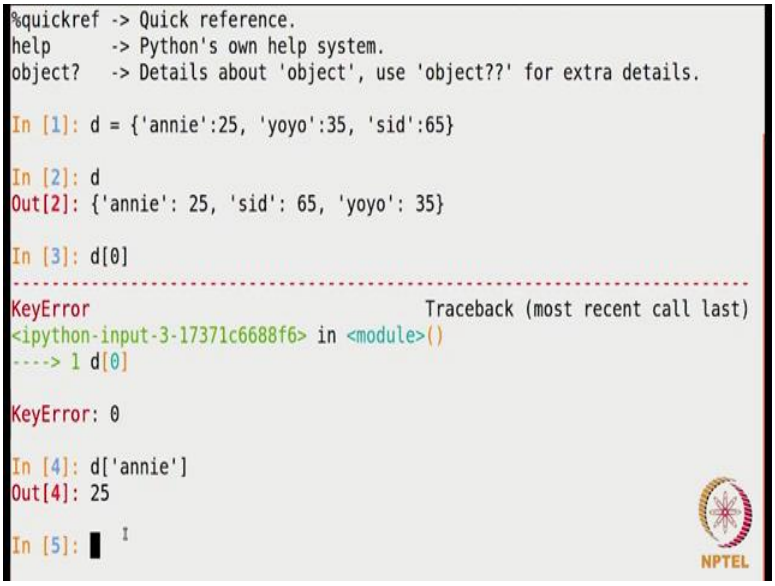
An interesting thing to note here is that these values might not necessarily be integers. So, rather they can be strings, they can be floats, they can be another list as well, they can be another dictionary as well. So, this is sort of flexibility that dictionary will provides us while its operation. Now, let me see and let me tell you how we can access the elements of a dictionary, so it is important that whenever we access we should have the key. Now, let me show you usage which does not work with dictionary; let me try accessing the first element by tracking this d0.

(Refer Slide Time: 03:32)



Now, this is not going to work; now this is because there is the elements in dictionary are not associated with index unlike the case of a list. For example, in list you can access the elements using their index; however, it does not happen in dictionary; this is because there is not association of any index with the elements. The only association that takes place in a dictionary is between the key and the value. So, just in case you want to access this element Annie, you would have to access it through the key.

(Refer Slide Time: 04:04)



So, I will write the key here and I will be getting the value that is 25, so this is how we access the elements of a dictionary. Now, why is this sort of thing happening, so d; 0 as I showed you is not working, this is because the order of elements in dictionary is not important. Basically, we cannot control the order in which the elements are displayed; let me print dictionary d.

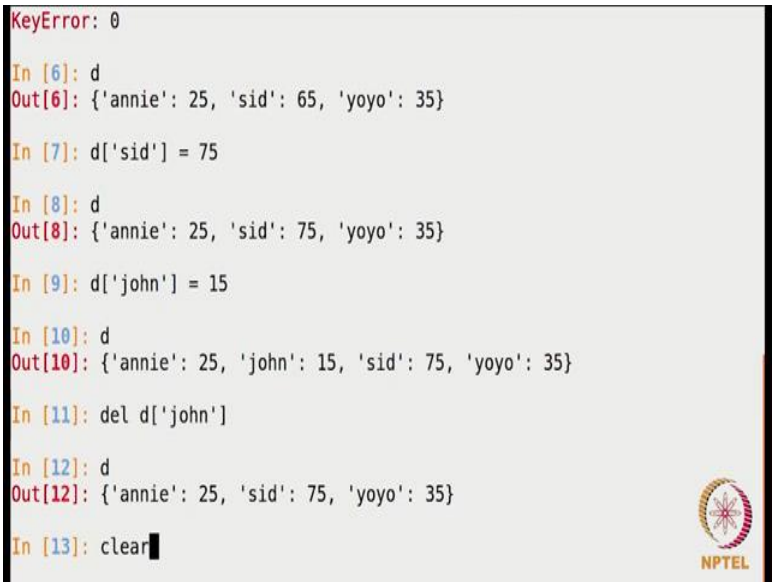
(Refer Slide Time: 04:30)



This is what I wanted to show you, we had added first Annie and then we had added yoyo and then we had added sid. However, when we are printing this dictionary; we are getting a different order. Now this is because it is up to the dictionary how it is storing the elements in the memory, so they are not like; they are not stored in the same sequence in which we add the elements.

Now, let me show you how we can update the elements in a dictionary; for example, I want to change the value of one of these elements.

(Refer Slide Time: 05:00)



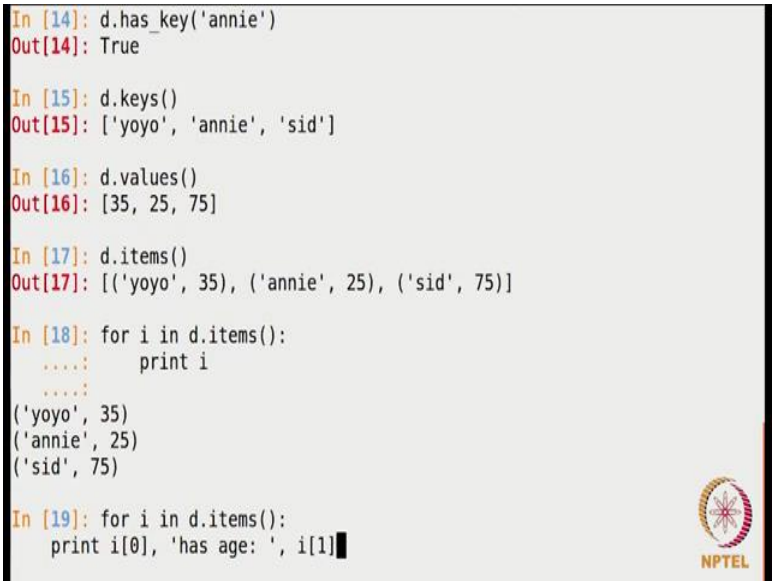
So, assume I want to change the value of this element to be this and then I print the dictionary, so you see the value has been updated and now let us see how we can add more elements into this dictionary. So, let me add one more person here say john and I want its age to be some this number and now let me print this, so this element has been added into the dictionary.

Now point to be noted here is that; the syntax for adding an element like this and the syntax for updating some value in the dictionary are the same. So, just in case the element is not presented to be added and in case the element is present, its value will be updated; so it is that simple. Now, let me show you how we can delete entries from a dictionary, so the keyword for that is del. Now, assume I want to remove this element john out of this dictionary, so I will access it using the key again as I told you. So, I will write this and this element should have been removed; let me check I will print this, the element has been removed.

Similarly, if you want to remove all the elements from the dictionary; you can also do that by using a simple function that is d.clear. So, what this function do, it will remove all the elements from the dictionary, but the dictionary will stay, so you can add more elements later. I will not use it because I do want this dictionary as if now and in case you want to remove the entire dictionary from the memory, you have another keyword;

you can just write del d, d is the name of the dictionary and the dictionary will be removed, so again I will not execute it.

(Refer Slide Time: 06:44)



Now, let me tell you a few functions that might be useful while we are doing the analysis of social networks. One of the functions that is quite used is; to check whether an element is present in the dictionary or not. So, assume I want to check whether the dictionary has an element Annie or not, so I will write d.has key; so this is an important function its quite frequently used. So, since I do not; I cannot see the entire dictionary, I can just execute this function and see whether Annie is present in it or not, so its gave me true which means the element is present.

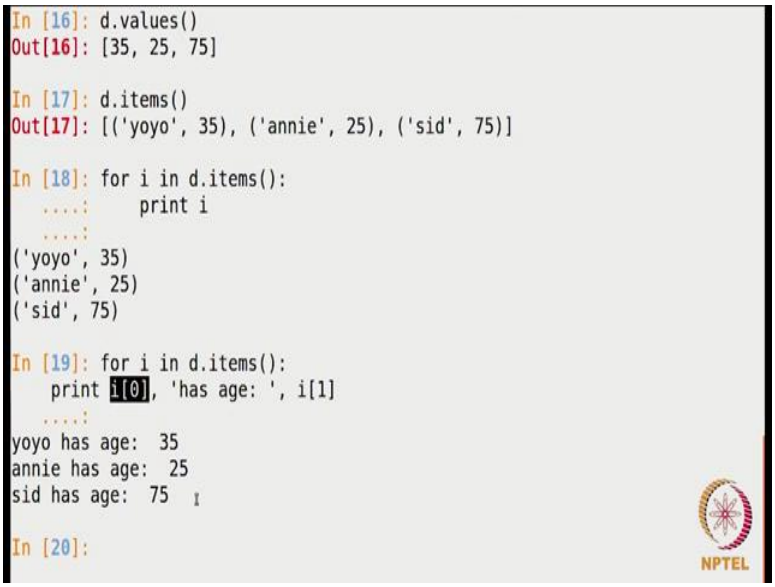
If I want to access all the keys from the dictionary, I can just write d.keys and it will give me a list. So, this is basically a list of all the key values from all the elements, similarly if we want to access all the values; we have this function; d.values, again it gave me a list, which is having all the ages of all the people.

Now, there is one more function which is again quite frequently used and that is items. So, in case we want to access key as well as value for every element; we can make use of this function d.items. So, you see here again we got a list, but the element of the list are basically tuples, where every tuple has two things; first is key and second is value. Also, if we want to access these elements one by one; we can use a for loop, let me show you

that. So, what I want is; I want to access the individual elements out of these tuples that we are getting from these items function.

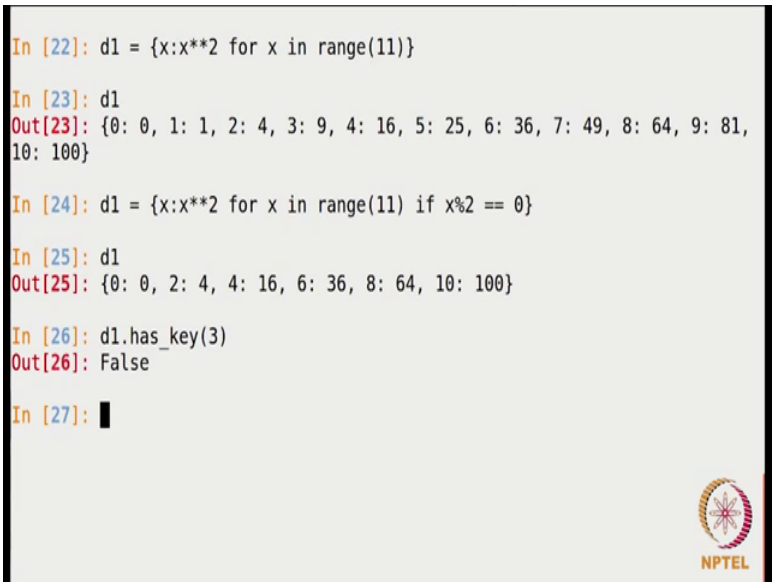
So, I will write for i in; d.items, so this d.items is going to return me a list of tuples and I am just going to print them; let us see. So what we did? We accessed these elements one by one that is all. Now, if you want to access key separately and value separately; you can use the index of these elements. For example, here I can write i; 0, so i; 0 will point to the key here and i; 1 will point to the value here, so I can write i; 0 has age for example, i; 1.

(Refer Slide Time: 09:05)



So, what we are doing here is; we are accessing the individual elements of the tuple by the index, so that is always possible.

(Refer Slide Time: 09:13)



Now, let me show you how we can quickly create a dictionary, where the elements follow some sort of pattern. So, I am going to make use of for loop for creation of a dictionary, so let me create this dictionary; where the elements should be of this sort and I will use for loop for x in range say 11; I am sorry in 11. So, this dictionary is created, so the elements in the dictionary are of this sort, where key is x and the value is x square. So, this operator you might be knowing in python, it is exponentiation operation and I am using for loop; where the x is going from, so this range returns a list going from 0 to 10, so 11 is excluded.

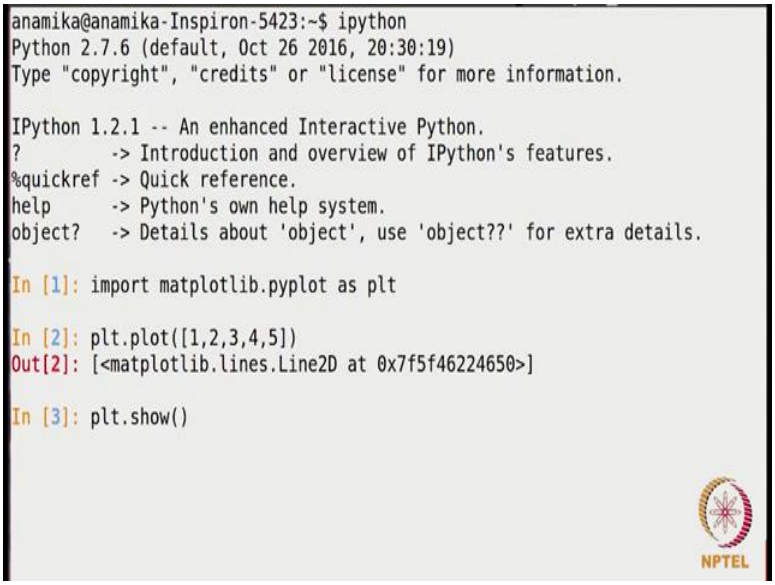
Let us print this dictionary and see what it contains, so we have 0, 1, 2, 3 up to 10 as keys and the squares as the values. So, this is how we can quickly create a dictionary; we can also play around with it. For example, I want only even numbers; so I can add another condition here, if x mod 2 is equal to 0 that is all. So, let me see what the dictionary contains, its now contains only even numbers are is the values and sorry even numbers as the keys and squares as the values.

We can also check the function that we just learnt; has key, so if I write even.has key; if I write 3 here, it should return false because there is not odd number as the key here. So, I think that was our pretty much basic introduction to dictionaries.

Plotting is an integral part of any sort of analysis, since in this course we are going to analyze social networks; we might need to plotters us to better visualize them. Therefore,

in this video we are going to see how we can make use of the package called matplotlib, which is provided by python for plotting purpose.

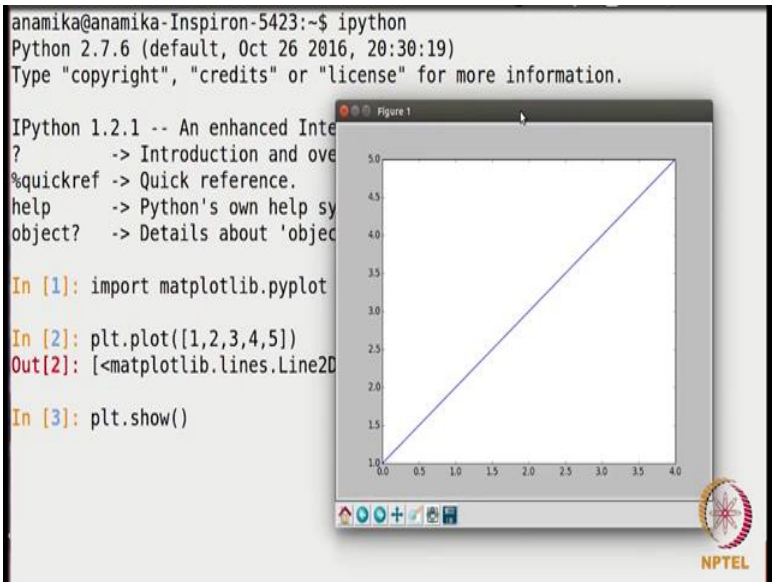
(Refer Slide Time: 11:15)



So, let us get started, so you might be to import this package first of all. So, the package name is matplotlib, inside matplotlib we have pyplot which contains all the function for plotting. I am going to give it a short name so that it is easy to use, now this means we are going to use plt instead of matplotlib.plyplot now on; so that will make things easier.

This package has a lot of functions for plotting and to customize our plots as well. The very basic function that this function has is called plot, so we are going to first use that function plot. So, I am going to write plt.plot, after that I will be giving various parameters to this plot function. Now again this plot function has various versions, to start with we are going to look at the basic version of this plot function which accepts a single list as a parameter. So, what I am going to do is; I am going to pass a list having some 5 elements here alright, so this is a very basic usage of this plot function. In order to see the plot we need to use a function that is called show, so I am going to write plt.show.

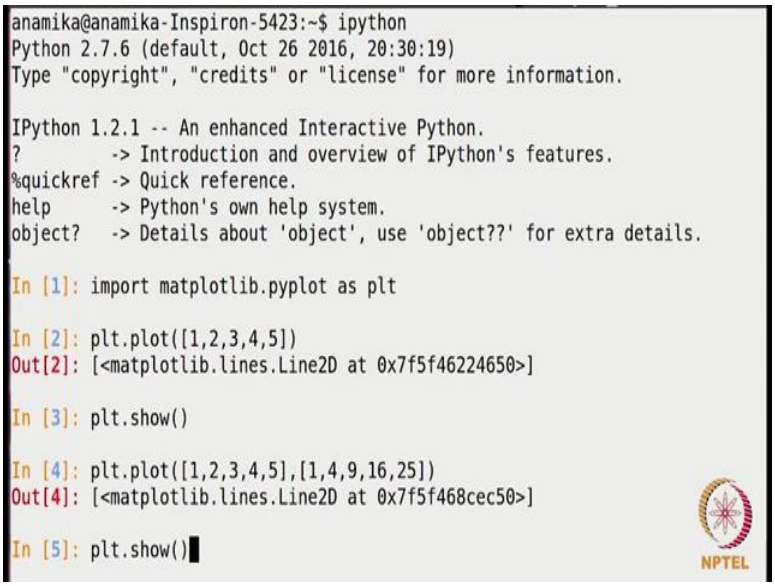
(Refer Slide Time: 12:39)



When I do that, I get this plot in the other window and as you can see this is very basic plot with no customization, there is no title of the plot, there is not title on the x and x and y axis and you see the x axis is starting from 0 and the y axis is starting from 1. So, let me tell you the reason for it; we called plot function with single parameter with which was only one list. Now when we do that, the values at the past in the single list they are taken as y values, which means this 1, 2, 3, 4, 5 are taken as the y values of the plot and since we have not given any values for the x axis, they are automatically taken up by matplotlib.

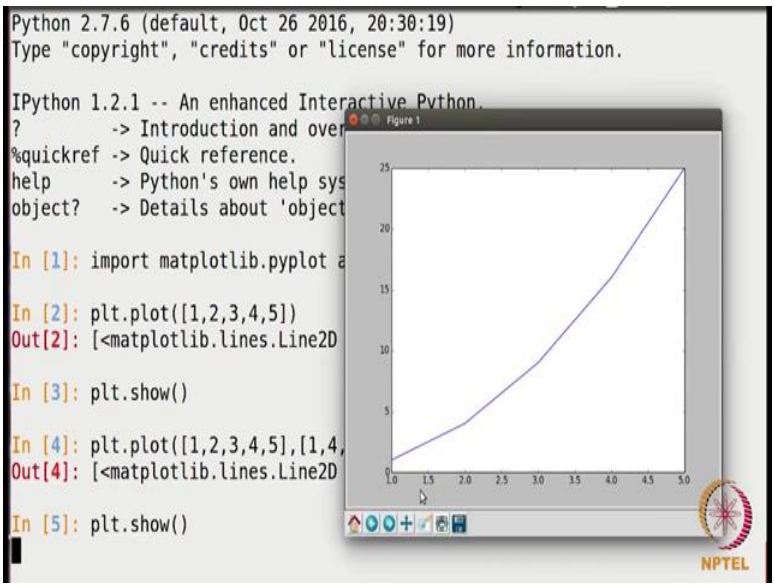
Now, since the index in python starts from 0; the x values also start from 0 automatically, which means x is equal to 0 is assigned to y value of 1, x is equal to 1 is assigned to y value of 2 and so on. So, you can see in this plot; x is equal to 0 has y (Refer Time: 13:44) 1, x is equal to 1 has y (Refer Time: 13:46) 2, x is equal to 2 has y value of 3 and so on. So, of course we can always change that, but when we provide only one parameter through this plot function this is how it happens. But we can always customize things; we can make this plot specific to our requirements, so we are going to see how we can do that using other functions. Now, I am going to close this window and I am back on the command window.

(Refer Slide Time: 14:17)



Now, let us look at some versions of this plot function; let us use your plot function where we are passing both x and y values. So, what we are going to do is; we will be passing two lists as parameter, the first list will be the values for the x axis and the second list will be the values for the y axis. So, I am going to pass say 1, 2, 3, 4 and 5 is x values and for y values, let me just pass the squares for example, so this is what I am passing here; so I am explicitly passing both the x and y values alright. Now to see the plot, I will write plt.show and I see this plot.

(Refer Slide Time: 14:56)



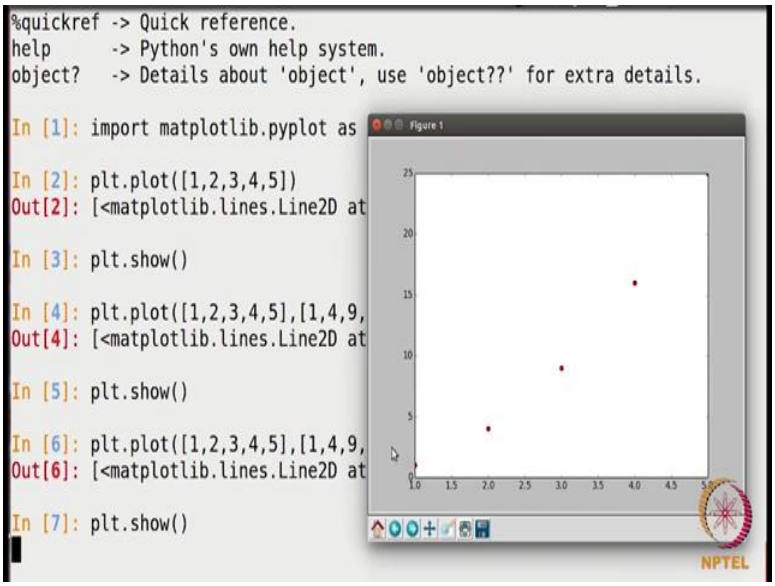
So, here the values; the corresponding to 1, we have 1 corresponding to 2, we have 4 and so on. Now, I am closing this window; I am going to show you some more versions of plot function.

(Refer Slide Time: 15:15)



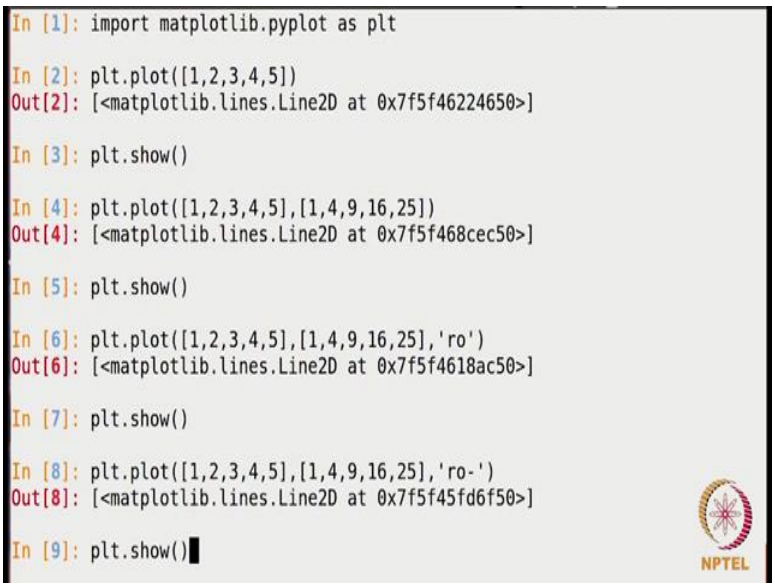
Let us get back here, so by default we were getting a blue line here as the plot; we can always change it, we can change the colour, we can change appearance of the plot. For example, by default we were getting a line; we can always change it to some points. Let me give you an example here, so I want the plot to be red in colour and say I want the dots; the points and not the line. So, I will write r; o here; let us see what we get, so every time you call this plot function, you have to write plt.show as well; in order to see the plot.

(Refer Slide Time: 15:50)



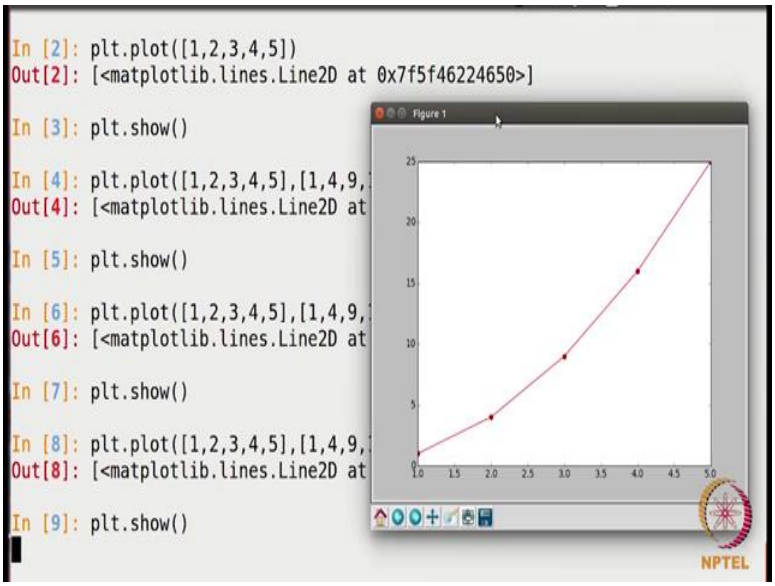
Now, here you see instead of the line I am now getting the points and that too red in colour.

(Refer Slide Time: 15:57)



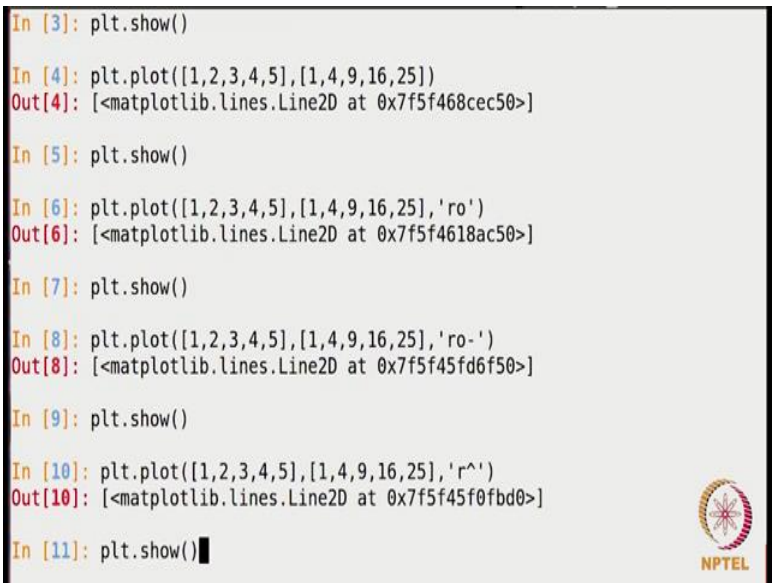
And in case you want the line as well, so you can put a dash here enter and then plt.show, now you get the points as well as line.

(Refer Slide Time: 16:05)



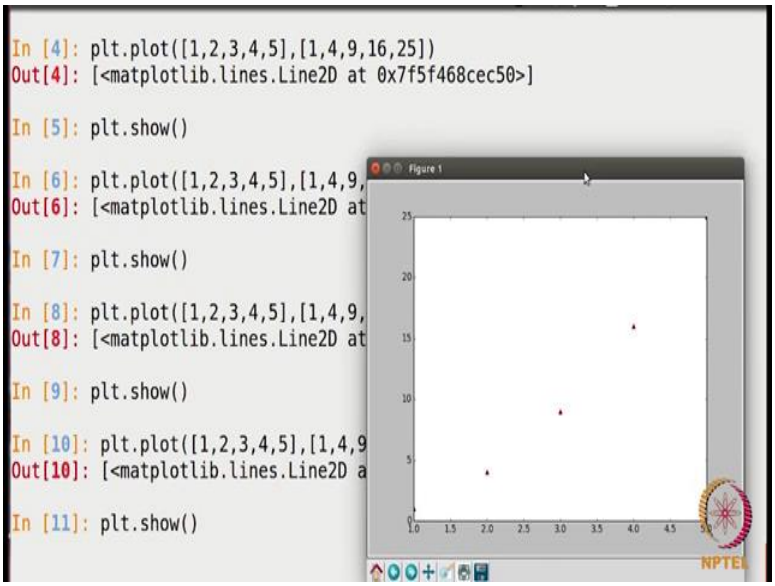
So, these are some of the versions; you can even change the other things, let me show you some more markers here so that you can play around with them.

(Refer Slide Time: 16:23)



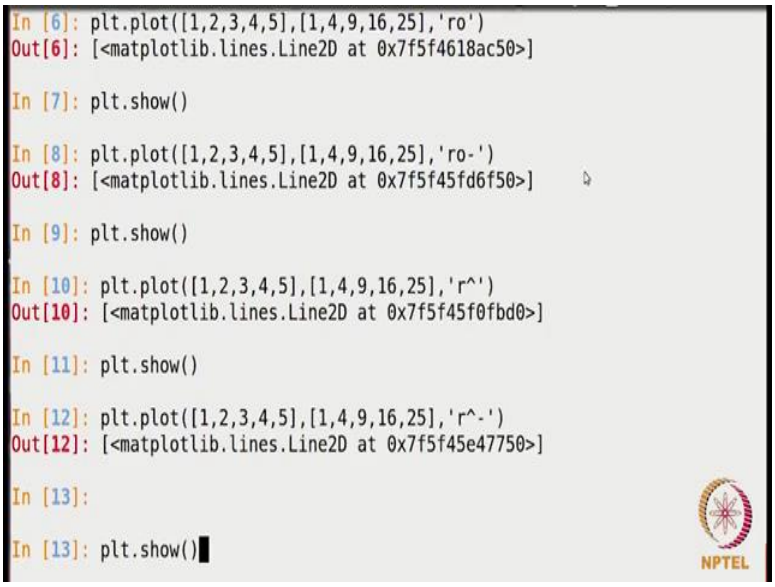
For example, if you want blue triangles; I will just write this symbol here and then I will write plt.show.

(Refer Slide Time: 16:29)

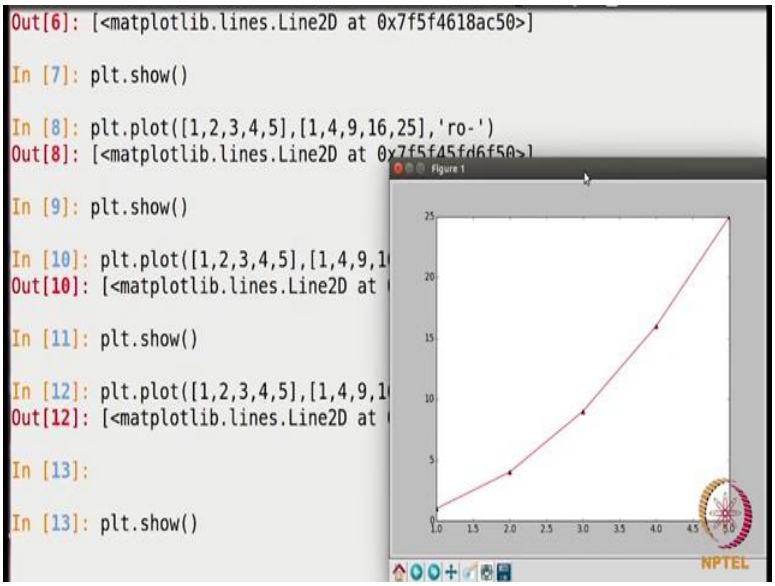


Now, you see you are getting the triangles here.

(Refer Slide Time: 16:33)

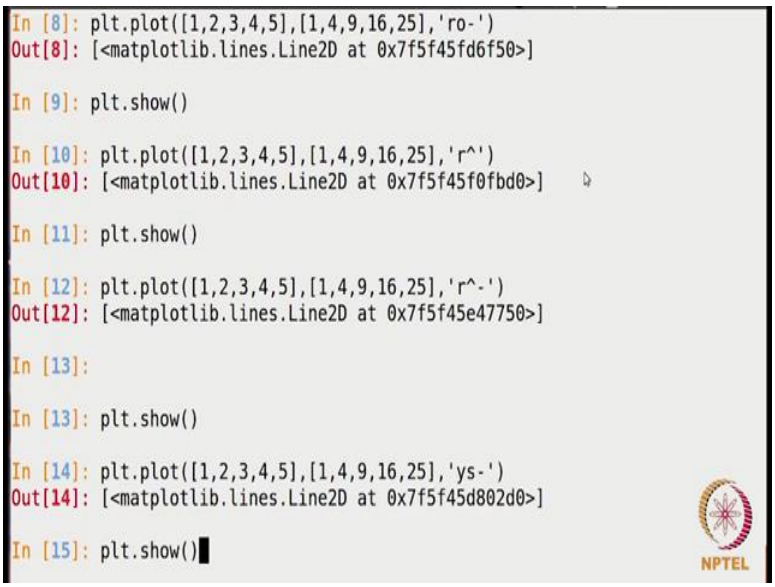


(Refer Slide Time: 16:39)



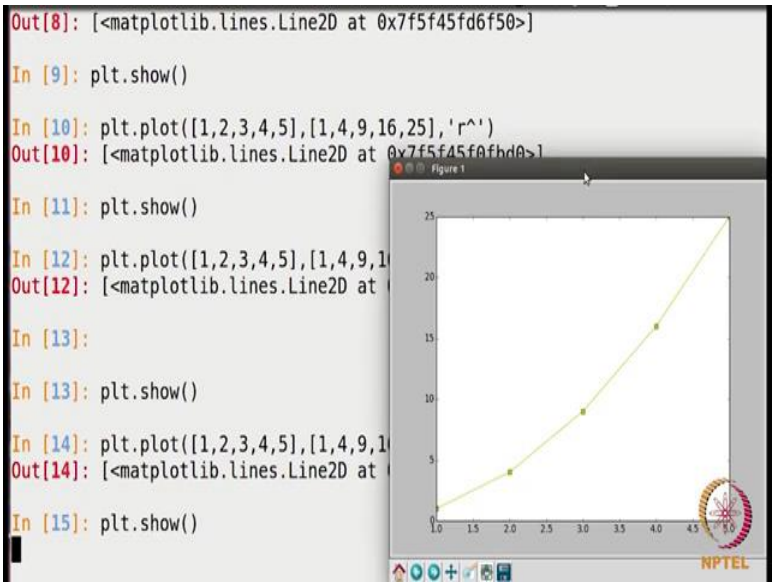
And if you want triangles as well as lines you can again; you can do that, so you are getting both the things.

(Refer Slide Time: 16:42)



And if you want the squares, so here you can write s and say I want yellow squares and then I will write plt.show.

(Refer Slide Time: 16:51)



So, we getting squares here and the line is also there.

(Refer Slide Time: 16:58)

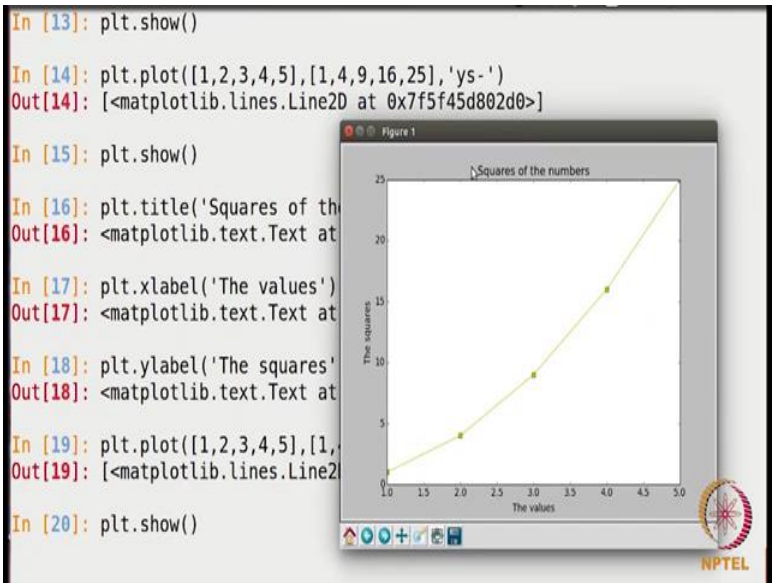


So, there you can basically look at the documentation and you will see there are various and you can just play around with them and use them as per your requirement. As you saw that, there was no title for the plot, there was no title on the x and y axis. So, let us try doing that; the function that we will be using for doing that, there is title function. So, we will write plt.title and we can keep the title of the plot here. Let me give something like squares of the numbers and I also want to give title for the x axis. The function that

we will use for that is x label, so I will write plt.x label; say the values. I also want to give a label for the y axis, so I will write plt.l label and may be the squares.

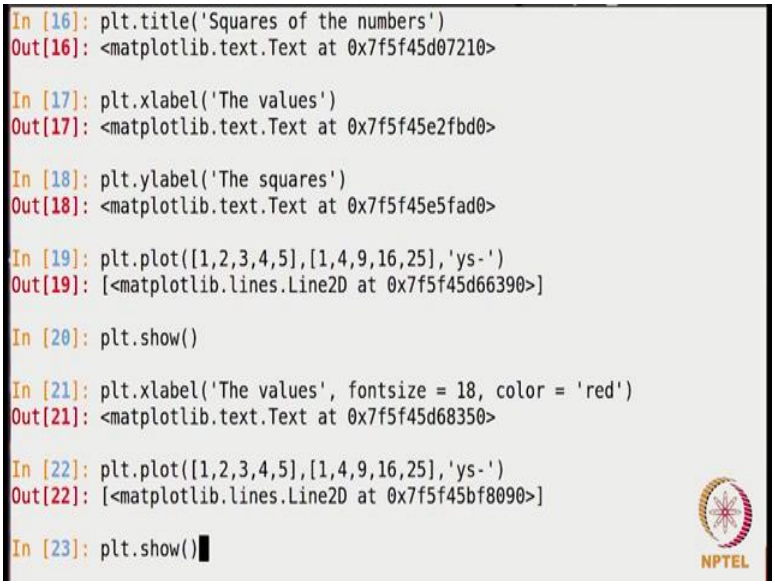
Now, again I will call this plt.plot and then I will call plt.show.

(Refer Slide Time: 18:06)



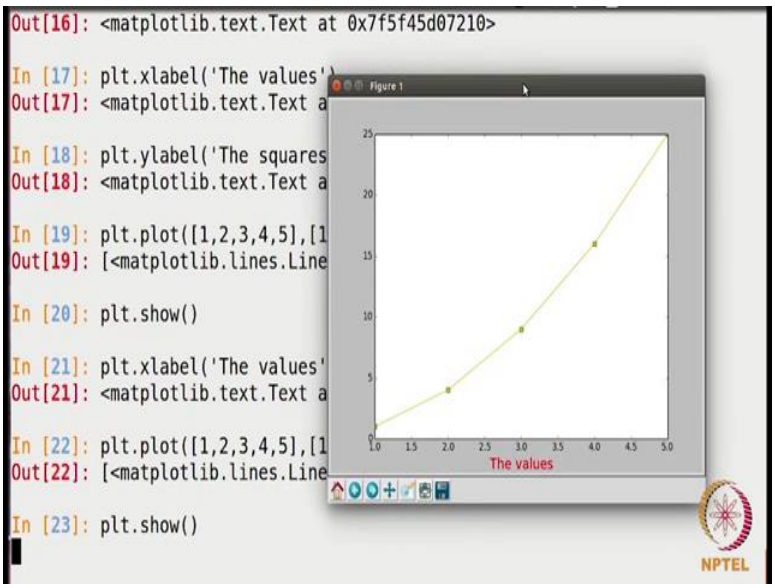
Now, you see we have got this; the title of the plot, we have the title of the x and y axis as well.

(Refer Slide Time: 18:18)



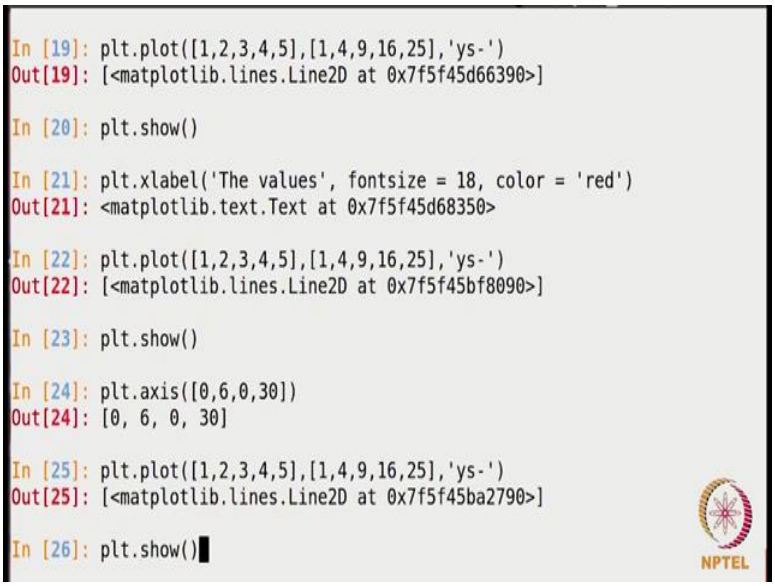
Let me also show you that you can also change the colour and the font size of these labels as well, as per your requirement. So, let me give you one example here; I want to change the x label to be say red in colour. So, we already made use this function rate x label plt.x label sorry, so we had only passed the label here. So, I am going to pass few more parameters here to customize it, so I am going to change the font size say 18 and I also want to change the colour; so I will write colour is equal to say red. Now label is changed, so I will call the plot command again and then I will show it.

(Refer Slide Time: 19:11)



So, you see the x label has changed, the colour has changed, the font has changed; so you can always; you know keep it the way you want it. Another thing that I want you to notice here is that; the values on x axis is starting from 1 to 5 and the values on the y axis is starting from 0 to 25 and if you see the values that we had passed are; the x axis values are from 1 to 5 and y values are from 1 to 25. So, it has automatically chosen the values where to start from and where to end; where we can always change it, we can always customize it. So, let me show you how we can do that.

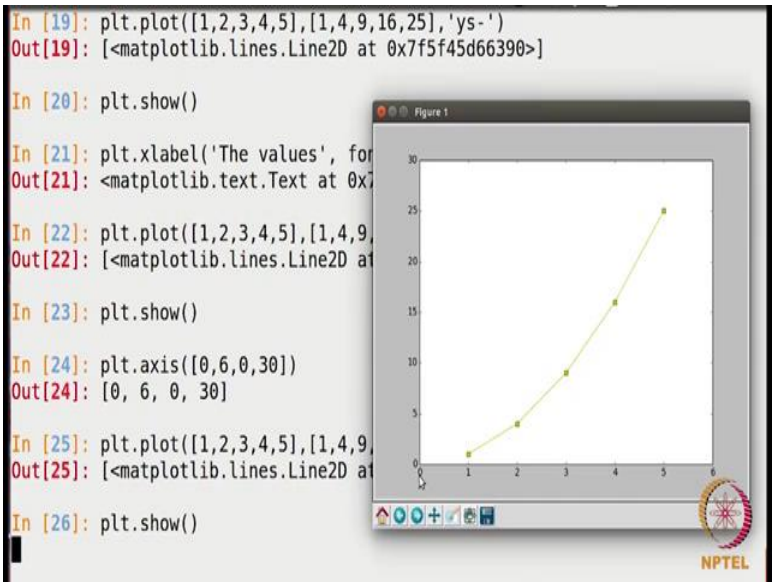
(Refer Slide Time: 19:52)



The function that we use to change the starting and ending of the axis is called plt.axis itself. So, I will write plt.axis; now this function accepts one parameter which is a list having four values, so the first value is the starting of x axis, the second value is the ending of x axis, the third value is the starting of y axis and the fourth value is the ending of y axis.

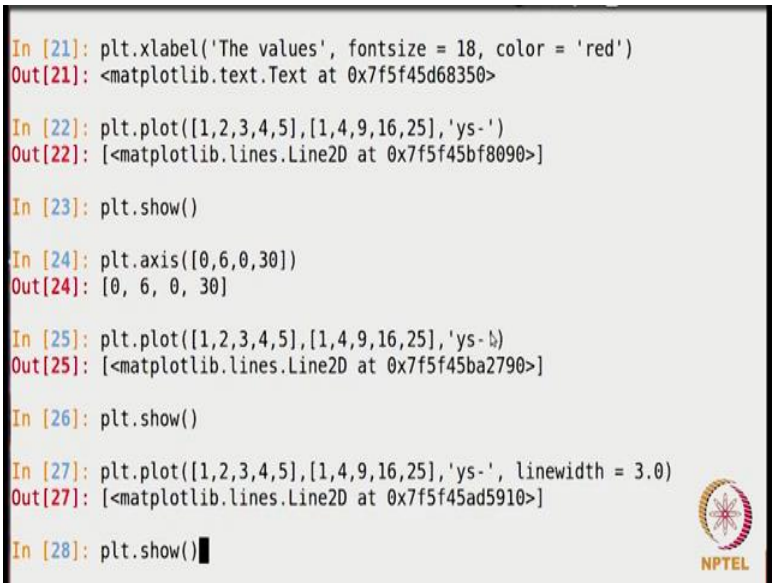
So, here you see our values; x values are starting from 1 to 5, so let me in order to show you its usage; I am going to give it 0, the starting of x axis and the ending of x axis; I am just like that going to give 6; just to show you how it works. And now I have to give the starting y axis, I am going to give 0 here and the ending of y axis; you see it is up to 25. So, let me just give it 30 so that we see how it works alright, so again I will call plt.plot and then I will call plt.show.

(Refer Slide Time: 20:56)



Now, you see here; x values are now starting 0 to 6 and y values are now starting from 0 to; are now going from 0 to 30 alright. So, you can this is always in control you can change it the way you want it.

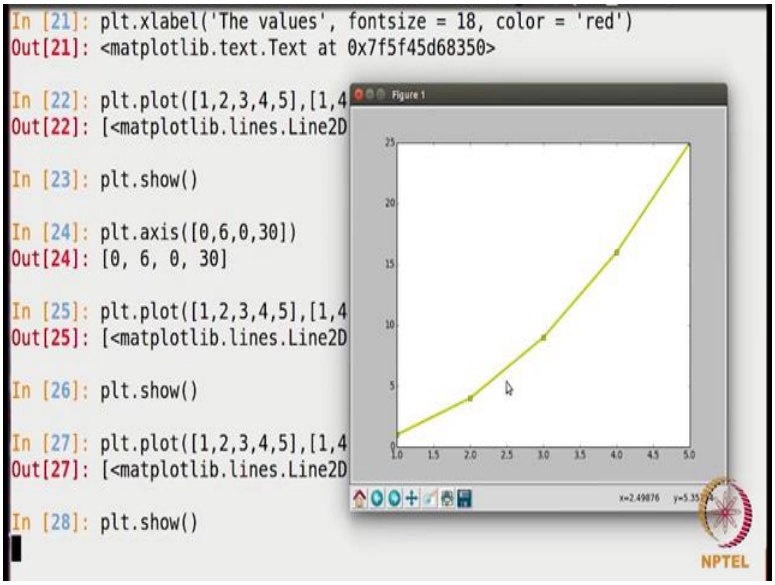
(Refer Slide Time: 21:15)



Let me now show you couple of more features of this plot function; for example, you got a very thin line as a plot. If you want to increase the thickness of this line you can do that, let me show you how to do that. So, plt.plot; let me reuse the same function, so here we were passing some three parameters, I am going to add one more parameter here for

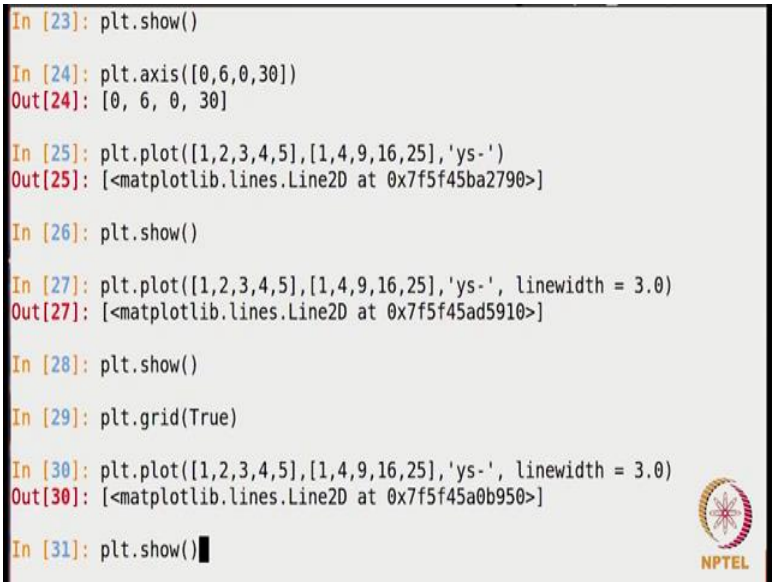
the thickness of the line. So, the keyword is line width here, so line width is equal to say 3.0; by default it is 1.0, so I give this line width and then I will call plt.show.

(Refer Slide Time: 21:48)



Now, you see the thickness of this line has increased, so you can always change that it is all about decorating your plot, so that it is better visualizing.

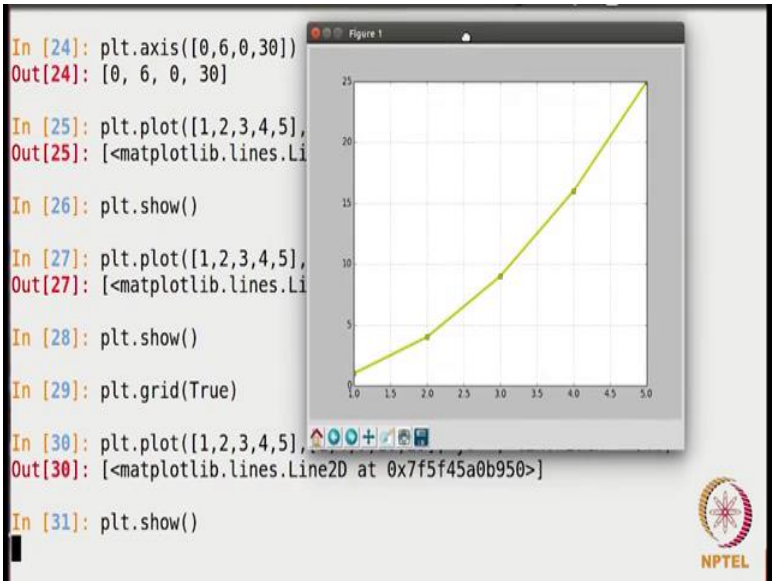
(Refer Slide Time: 22:02)



Another thing that is sometimes needed is a grid that you want to display in your plot sometimes. So, by default it is turned off, but you can always turn it on by passing a true parameter there, so the function that we use there is grid. So, I will write plt.grid and I

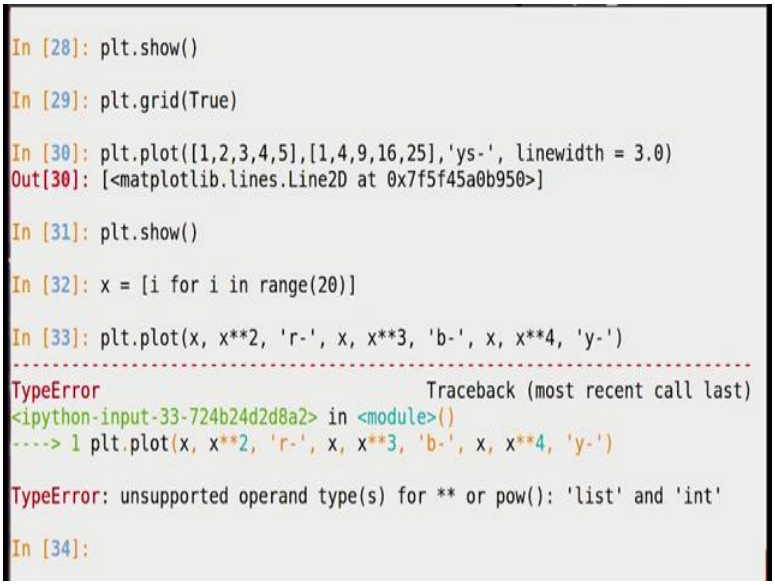
will pass true here, which is by default false. So, once I do that and I call this plot function and then I will call the show function.

(Refer Slide Time: 22:30)



You see there is a grid here, so that is there in the background you can always turn on and off as per your requirement. You can also save this plot if you want; there are various functions, there are various features here that you can make use of ok.

(Refer Slide Time: 22:51)



Let us get back here, as of now we were showing only one plot inside the plotting region. But that is not necessary; we can always display multiple plots inside the same plotting

region. Let me show you an example for that, let me create the x values by using a loop for example, that will also show you how you can use a loop to create a list, if you do not know already.

So, I right i for i in range; 21, if I write this; I will get the values from 0 to 20; excluding 21. So, basically I will be getting 21 values and let me keep 20 here, so I got the x values and then I want to plot this x along with its squares. So, I will write x and this is exponential operator and I want it red in colours, maybe in line. Now in case I want to also display the x cube as well in the same plotting reason, I can do that. So, let me show you how to do that, so I will write x cube here and say I want that blue in colour, blue lines and assume I want to display x to the 4 as well in the same plotting region. So, I will write x to the 4 here and say I want that yellow in colour; I will do this.

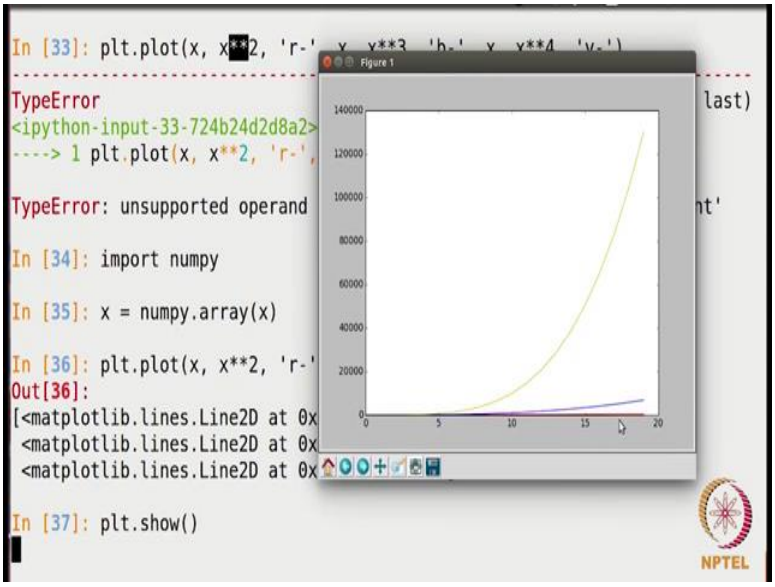
But there is an important point to be noted here, which I want you to know. When I run this command, it should show me an error; let me see here, it is showing an error. Now what is the reason for that? The reason is that x is a list, this is a list and then we are applying this exponentiation operator on a list, which is not allowed in python. So, what we can do here is that; we can make use of a package which is called numpy, which allows such sort of operation. So, if we if we convert this list x into a numpy array; then in that case such operations are allowed. So, the crux is that this kind of operation is not allowed on lists, but it is allowed on numpy arrays. So, let us do one thing we will make use of this numpy package.

(Refer Slide Time: 25:10)



Let me put that; now I will basically convert this x into a numpy array. So, I will write numpy.array, this is a function, so I am passing x as a list and I am getting it as a numpy array. Now after that, if I called this plot; it is now giving me an error and it is working fine, now we can do p l t dot; sorry.show.

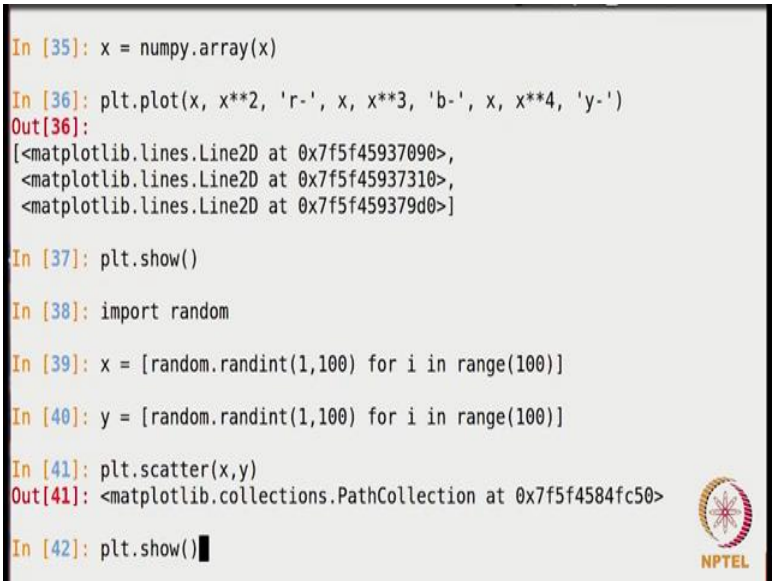
(Refer Slide Time: 25:41)



So, here you see I am getting all the 3 plots here; yellow, blue and red; I am getting all the plots here. Now since this due to these dimensions of this y axis, you are not really

able see this red line, but you can always do that as I told you, you can customize, you can change the starting and ending of y axis; so you can play around with that alright.

(Refer Slide Time: 26:05)

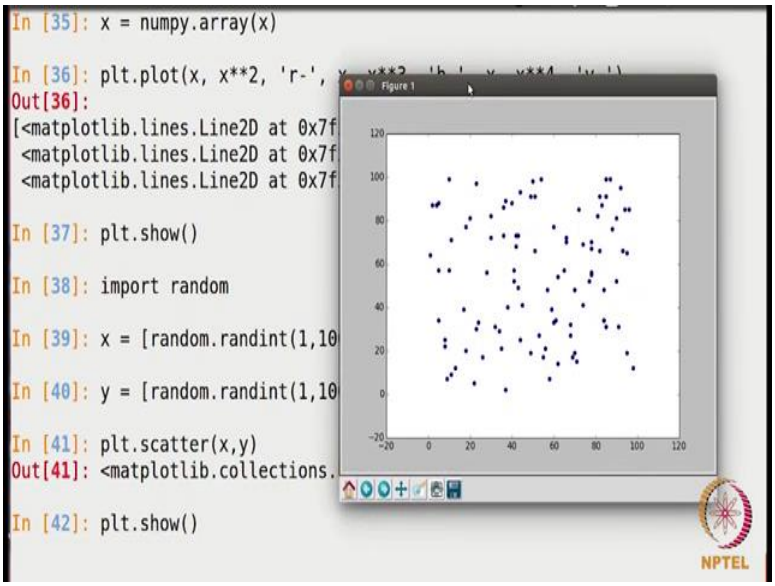


As of now we have seen only a simple plot function, we have seen a simple plot in form of lines or points, but let me tell you that we can also have a different kinds of plots using this matplotlib package; for example, we can have scatter plot, we can have bared plot, we can have heat max as well. So, there are various kinds of plots that we can draw; now I will be showing you some two, three examples just to give an idea.

I will show you how to draw scatter plot, so in order to draw the plot let me create a list randomly also that will show you how you can make use of a random function. Before that I need to import the package, so I will import random package and then I am going to create a list which will be having a 100 random values. So, how do we do that? I will make use of the function randint from this package random and I want the values to be between 1 and 100 and I want 100 such value, so I will write for i in range 100.

So, I will getting 0 to 99, I will be getting 100 values here alright, so let me also have y values randomly. So, basically I am having a 100 random values as x values and 100 random values as y values. Now, let me draw the scattered plot; the function is scatter itself. So, I will plt.scattered x, y and in order to see the plot, we have to write p l t.show.

(Refer Slide Time: 27:49)



So, here we get the plot; now again the kinds of operation that we applied in the simple plot function, like we changed the axis, we changed the colour, we changed the titles and everything that also we can do here. For example, here you see by default; the x axis starting from minus 20 and same goes with the y axis which is not looking so good.

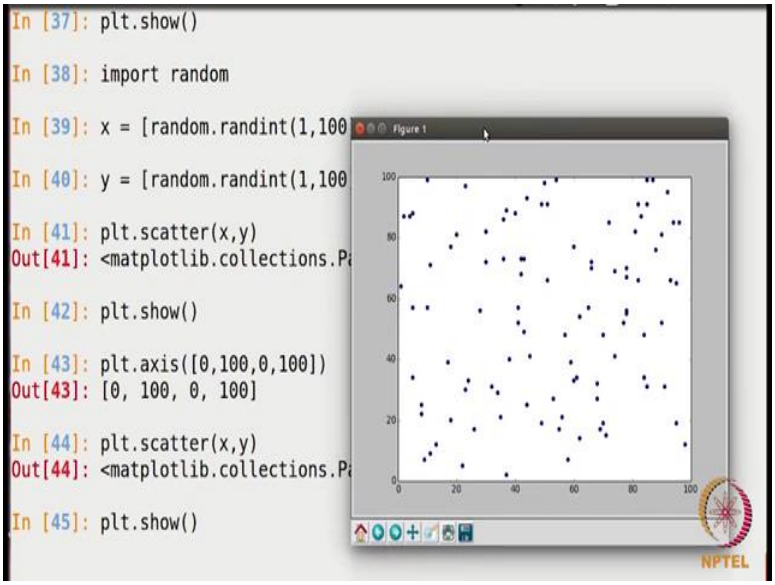
(Refer Slide Time: 28:16)



Let me change that, so what I will do is plt.axis, I am going to pass a list with four values; starting of x axis to be 0 and ending to be 100, starting of y to be 0 and ending to

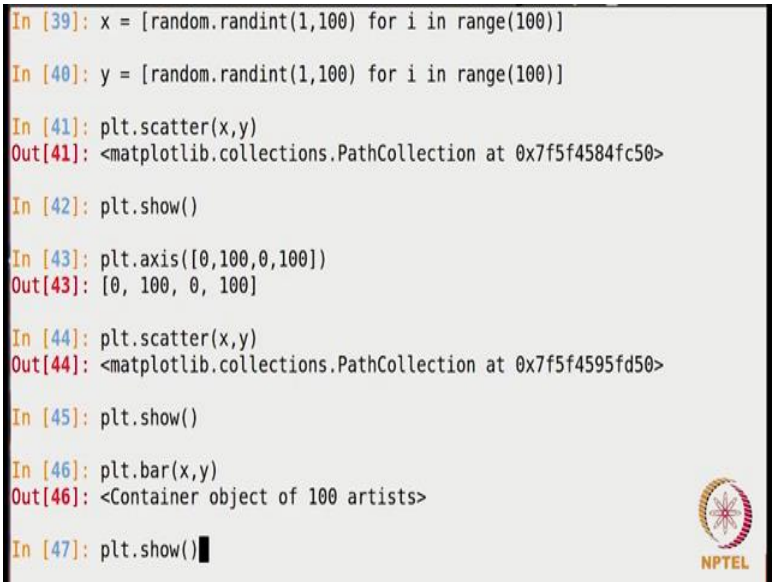
be 100. Now, I will call this plot function again, sorry scattered plt.scatter and then plt.show.

(Refer Slide Time: 28:41)



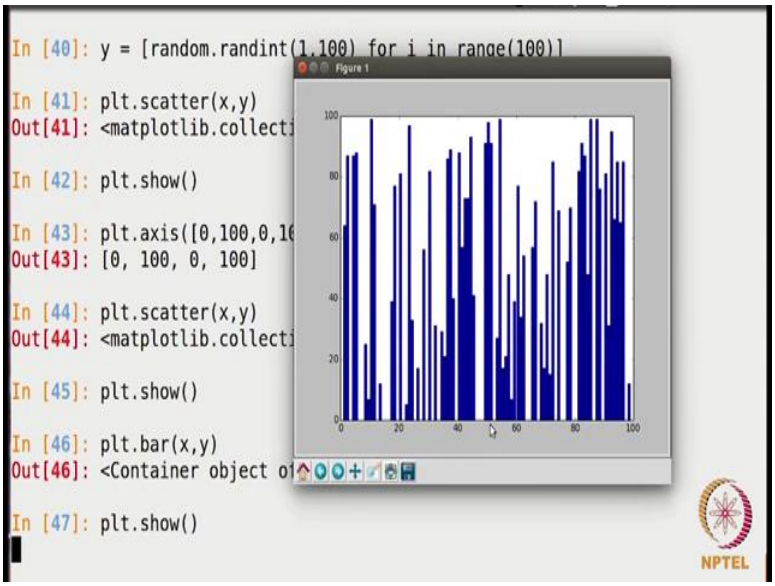
Now, you see x and y are starting from 0 now and it is looking quite better, you can clear on, we can change the colours of these dots alright, let us get back here.

(Refer Slide Time: 28:52)



Now, let me show you another kind of plot that you can have using this package. If you want to bar chart or a few data, so you can use a function bar; plt.bar, I am going use the same x and y values here again, so that is all; plt.show.

(Refer Slide Time: 29:12)



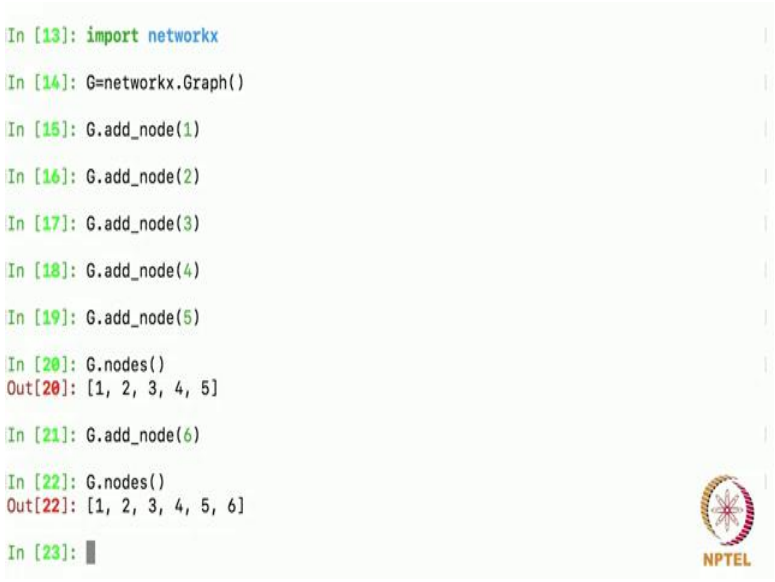
We get a bar chart here, so by default we always get a blue colour, so axis is fine here 0 to 100 because we had set it already and so you can again clear on with customizing this plot as per your requirements. So, these were just few examples of the kind of flexibility that this package provides you. You can look into the documentation and look for more functions in as per your data as per your requirement.

Thank you.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 05
Introduction to Social Networks
Introduction to Networkx-1

(Refer Slide Time: 00:05)



I will now be introducing this graph package called networkx, with the help of which we can do a whole lot of graph analysis. I have pre-installed a networkx api on python, you may have to Google and understand how to install it on your computer. In case you find any difficulty, please let us know and we will help you out. Write to us on our email address and I will personally help you out.

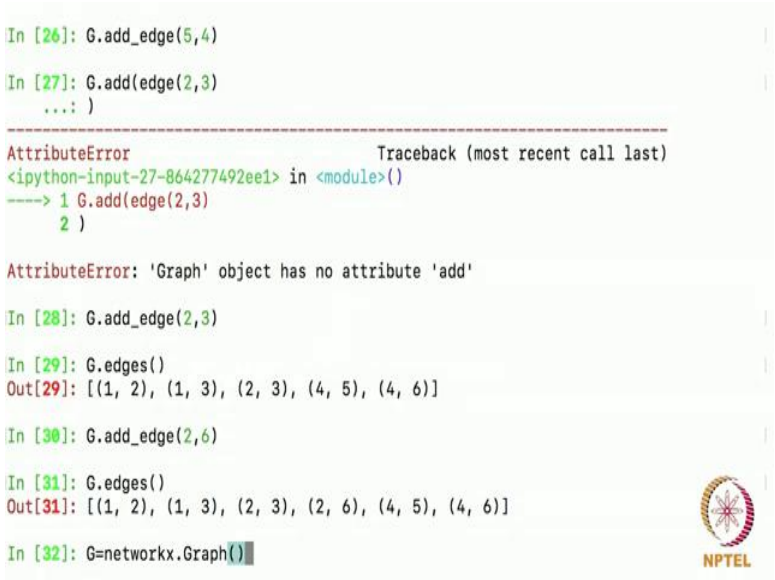
So, we first say import networkx and the library function networkx gets loaded and then I say G = networkx.graph and then there is a graph that gets created and G gets assigned to it and now I say G add node 1 which means vertex 1 gets added. Similarly, I say add node 2 and so on, I will add some 5 nodes to it, 3, add node 4, add node 5 and when I say G.nodes, it will show me all the nodes that I have just now added, I will add one more node; let us say 6 and then I say G nodes, it will print all the nodes that are just now added.

(Refer Slide Time: 01:31)



Now, I can start adding the edges; how do I add the edges? G.add_edge 1 , 2; G.add_edge 1 , 3; G.add_edge 4 , 6; G.add_edge 5 , 4; G.add_edge 2 , 3 and so on, I have added I did not close the bracket.

(Refer Slide Time: 02:00)

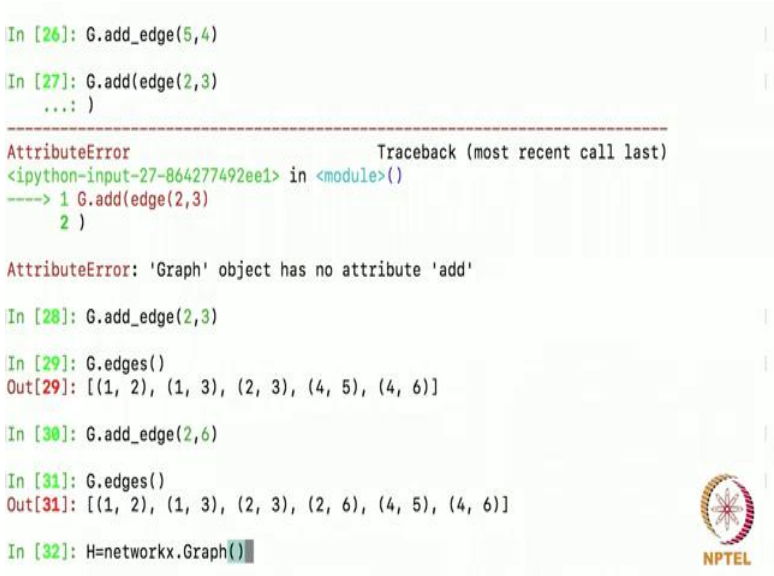


I have added all the let me do it once again it 3, 1 error; G add underscore edge, there is a mistake there, 2 , 3 and that is it. So, I have added a few edges; I can see all the edges by giving this ,nd G.edges, you see the 5 edges that I just now added is here. Let me add more edge to show you that it is actually getting included, I will add the edge 2 , 6 which

is missing here; add edge 2 , 6 and I will display edges, one more edge gets added; 2 , 6 got added here as you can see alright.

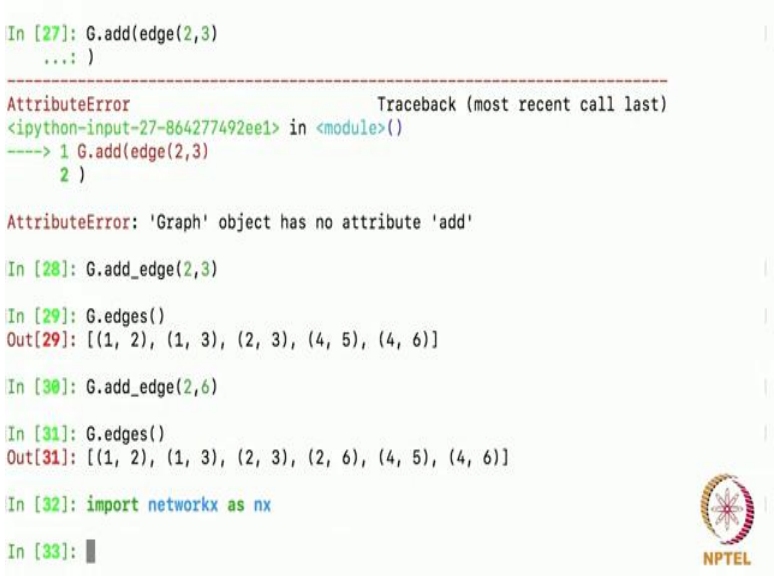
Now, how do I see this addition; now before that you saw that I said G equals networkx.graph.

(Refer Slide Time: 02:55)



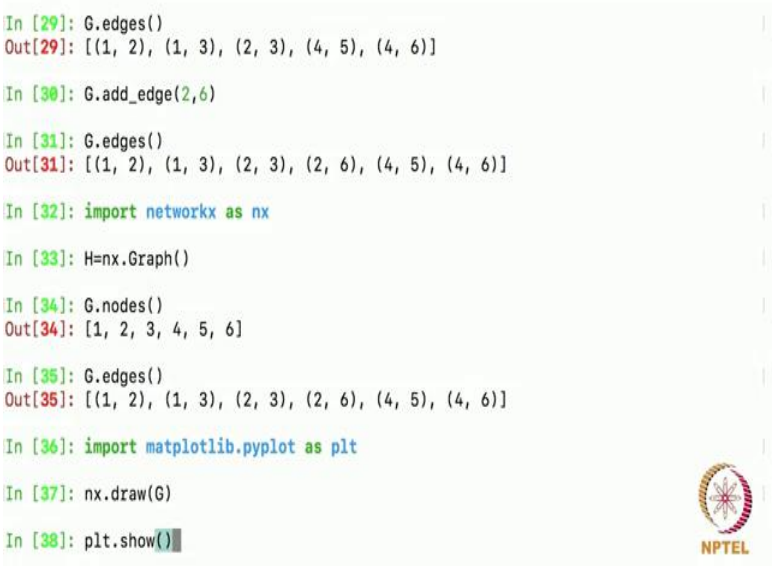
If I were to define another graph namely H; should say H = networkx.graph given that tell me using this networkx often, there is one way to create an alias like thing; a macro like thing in c, if you remember.

(Refer Slide Time: 03:10)



So, what I do is; I will say when I am importing, I will say import networkx and as nx; short form for network.

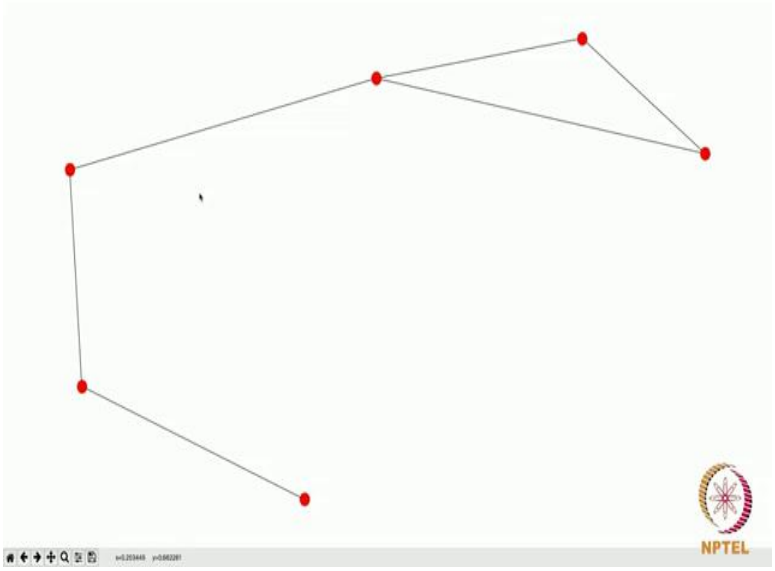
(Refer Slide Time: 03:17)



Now, I can simply say H equals nx graph; instead of networkx graph, I can say nx.graph. Now a graph gets created, anyway I just have to illustrate this usage of import networkx as nx, this is a standard way to do it. If you Google online for code using network, the first line will be import networkx as nx, I did not want you people to get confused; so I thought I will clarify it here. Going back, I created a graph G with the following nodes and the following edges. I want to see how the graph looks like, there is a package for it and I should import that package which helps me visualize the graph; it is called matplotlib, in that I only want this particular sub library of called pyplot and I am going to import that and to avoid that this big ,nd in invoking functions within this, I will say as plt.

Now, I should visualize this graph; this is the way to visualize, it is slightly complicated but you will remember it given that you will be doing it very often. You will simply say; networkx.draw; what you want to draw? You want to draw the graph G; networkx G and then you say plt.show.

(Refer Slide Time: 04:50)



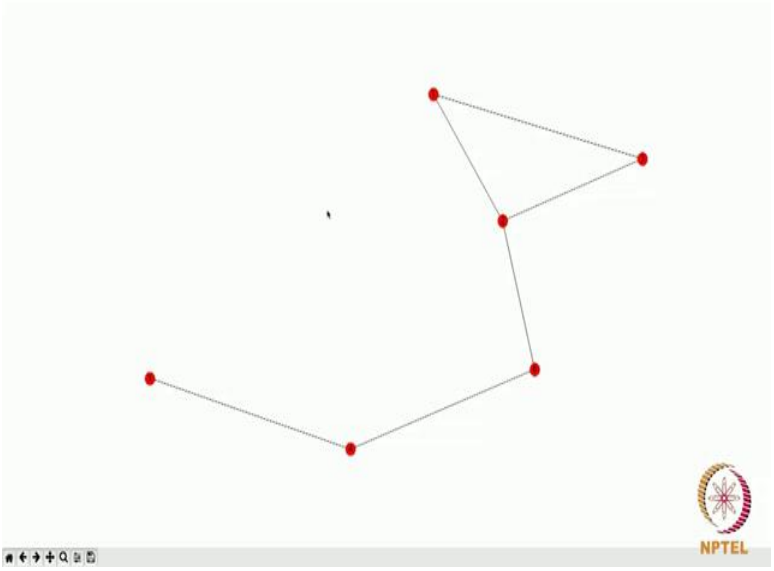
You see the graph that we just now created and we do not know which node is what.

(Refer Slide Time: 04:57)



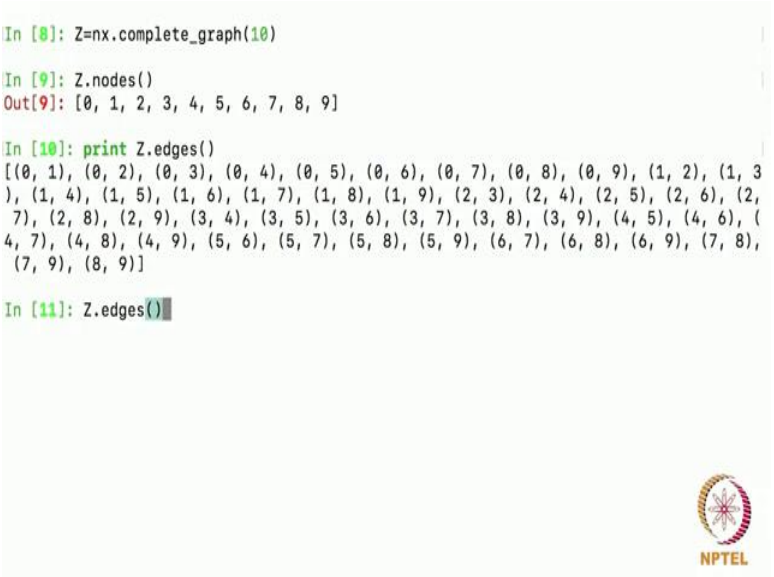
So, what will do is to display labels I just issue the same ,nd nx draw G and I say with labels equals true. If I include this, you will get; you will see what you will get, you will also get the vertex labels and now I issue the ,nd plt show.

(Refer Slide Time: 05:14)



And you will see that look at this, the graph that I input they are with labels right now, they were without labels in the beginning as you saw.

(Refer Slide Time: 05:24)



Networkx has a lot of built in functions, I will show you one such built in function; let me say Z is equal to nx complete graph 10, this means it generates a complete graph on 10 vertices and assigns it to the variables Z. Let us see what happens, I say enter now a graph on 10 vertices is created, complete graph means it puts all possible edges between these 10 nodes a little bit of thinking will tell you that all possible friendships between 10

people is actually 45 alright. You can do the computation; you can take it as a homework and do it is a very straightforward question.

So, let me look at the number of nodes in Z; what are all the nodes in Z of course, there are 10 nodes here and the edges; print Z edges or simply Z edges, both of them will be output the edges; here are the edges.

(Refer Slide Time: 06:28)



If I simply say that Z.edges, it will put it in a line like this; that is why I said print Z.edges; it will neatly show it like this.

(Refer Slide Time: 06:33)



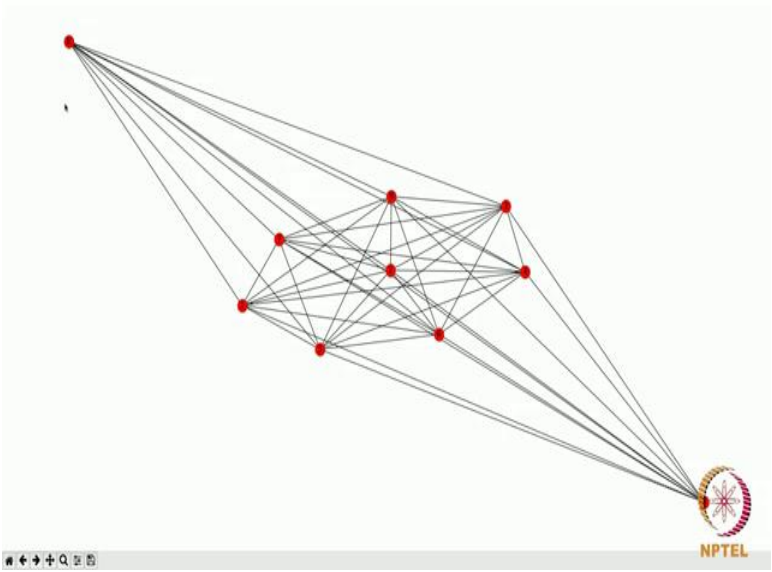
There are 45 edges here, how do I say it is 45? There is another ,nd called Z.order which stands for the number of nodes in the graph.

(Refer Slide Time: 06:42)



And Z.size which tells you the number of edges in the graph, there are 45 edges. Now, let me visualize this graph Z; how do I visualize it, nx draw Z; I want it with labels, so I say with labels equals 1 and then I say plt show.

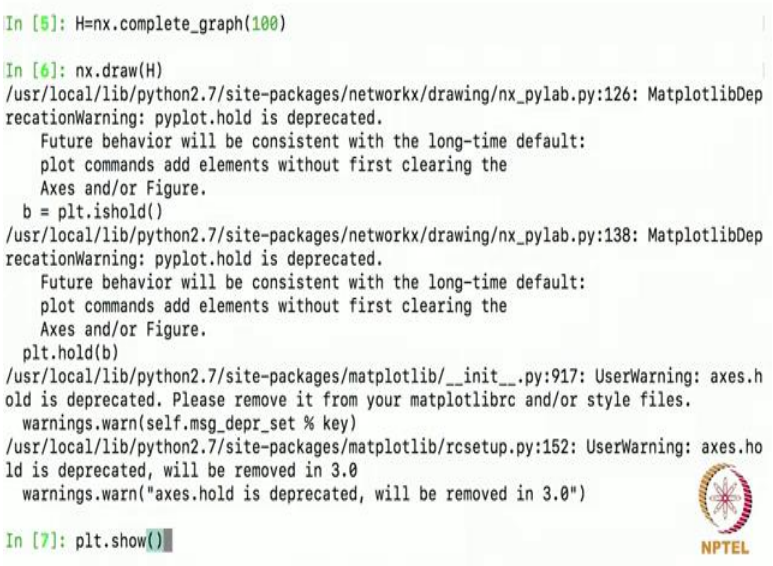
(Refer Slide Time: 07:02)



It will show me the graph and here is my complete graph on 10 vertices, as you can see every node is connected to every other node; that is what you mean by a complete graph

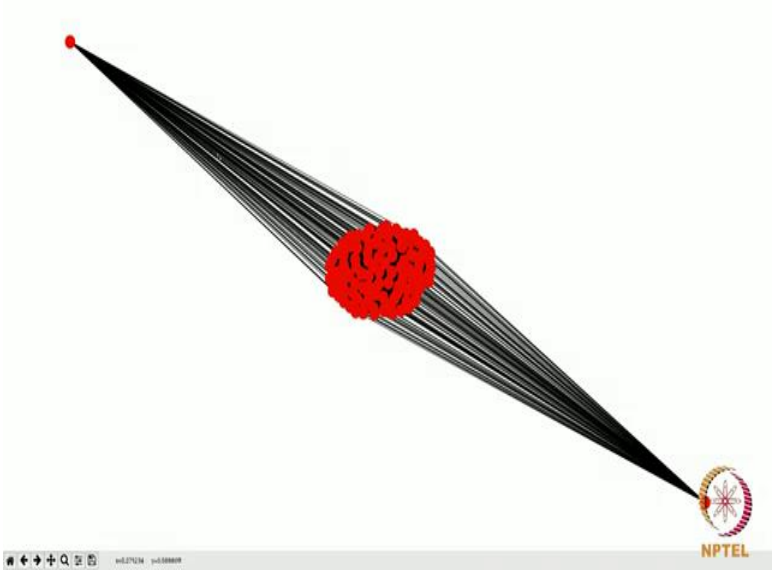
correct. Now, let us try seeing a complete graph on 100 vertices, let us see how it looks like; how does a complete graph on 100 vertices look like?

(Refer Slide Time: 07:22)



I will say H equals nx, complete graph on 100 vertices then I will draw this H; I will not label it labeling will make it look clumsy, the typical warning message I told ignore this; plt show.

(Refer Slide Time: 07:51)



So, this is a complete graph on 100 vertices, so there are 100 vertices and all possible edges. You can ask me, what is this vertex and this vertex? These are all visualization

styles that are in built into this matplotlib. You can in fact change this visualization to; there are ways to do that. But as of now, all I want to show you people is that you can see that there are 100 vertices and they are connected to each other and this is how we visualize it.

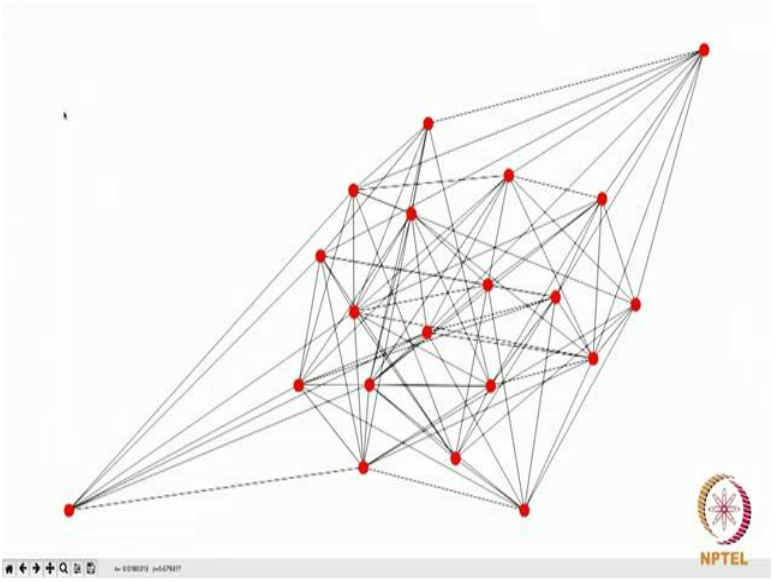
(Refer Slide Time: 08:18)



Networkx actually has a lot of built in functions, I will show you one such built in function which you may want to use more often. So, look at this I am going to generate a graph let us say G; which is equal to, the ,nd is called gnp random graph; it is an nx; gnp random graph on 20 vertices and I say 0.5 here, What does this mean? This means, I am generating a graph on 20 vertices and edges I am putting them with probability 0.5, which means for every edge; I put the edge with probability 0.5, I do not put the edge with probability 0.5; by that I mean, I toss a coin; if I get a head I put that edge, if I get a tail I do not put that edge.

How does the graph look like? A nice way to explain this is there are 20 people and there are friendships forming between them, they decide whether 2 people should become friends or not by tossing a coin; this is such an experiment that is what you mean by gnp graph. More details, I will be covering in my lecture do not worry much if you did not understand what I said here; all I am trying to do is generate a graph G with 20 nodes with some random edges; how does this graph look like? Lets us draw and see this plt show.

(Refer Slide Time: 09:50)

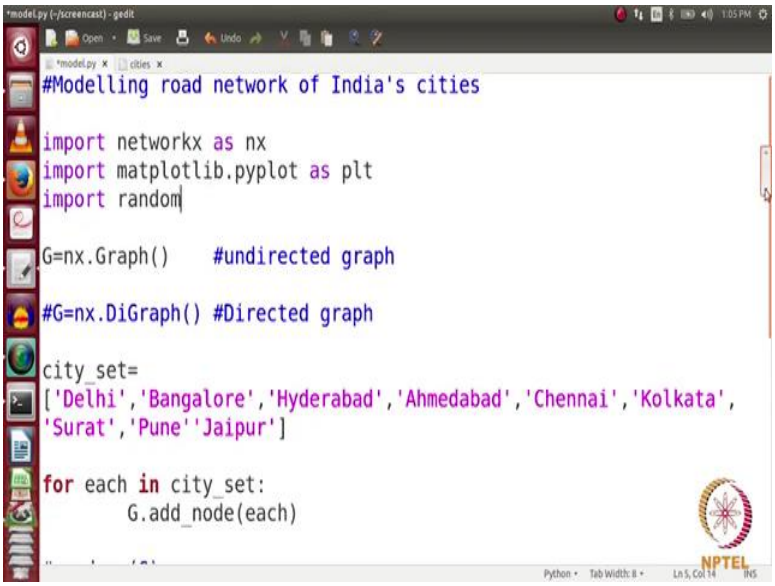


This is how the graph looks like on 20 nodes, where edges are put uniformly at random. Unlike the previous case, this is not a complete graph as you can see; for example, there is no edge between this vertex and this vertex, this is not a complete graph; this is just a graph on 20 nodes with random edges.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 06
Introduction to Social Networks
Introduction to Networkx-2

(Refer Slide Time: 00:08)



Hello everybody, in the screen cast we are going to do a very interesting programming assignment, what we want to tell you through the screen cast is how do you pick a real world problem and model it with the help of python and networkx. So, we will be using both python as well as networkx in the screen cast. So, that you can learn how to put both of them together and use the real world problem which we will be picking is the network of cities.

So, we have a lot of cities in India. So, we represent every city as a node and then we have edges between these cities we which are the roads connecting these cities. So, we will be assuming that between 2 cities there is one road though there can be many. So, we pick one and then there might be a travelling time associated with each of the road. So, you can visualize this as a network where nodes are the cities and edges are the roads connecting them some roads might be good some roads might be bad some roads can be congested some roads can be free and so on which affect their travelling time if you

might if you would have heard of the travelling salesman problem you would be very easily able to imagine it, but its otherwise also its easy to imagine. So, you have a network where there are cities and there are roads connecting the cities. So, we will be making this network with the help of python and networkx will be visualizing it and we will be looking at various properties of this network.

So, let us get started how do we do it? So, what we are going to do is we are going to model the road network of India let say India's cities, let us say we are going to model the road network of cities in India as I have already told you. So, what is the graph here? So, let me first save this file of this folder screen cast here let say I save it with the name cities.Mumbai for better event let say model.py. So, I have a model.py here. So, let us see how do we model the road network? So, let me first make a graph. So, for making a graph I need the networkx package which needs to be imported.

So, we import networkx as nx. So, now, we can use it and we are going to take the graph s undirected. So, we are going to assume that if there is a road between let say Delhi to Mumbai. So, assume there is a road between Delhi and Mumbai. So, it will take you equal number e it will take you equal time to go from Delhi to Mumbai and from Mumbai to Delhi. So, hence we can have one edge and the graph is undirected. So, here we are going to make an undirected graph G equals to nx.graph. So, when you write G equals to nx.graph it automatically creates an undirected graph for you.

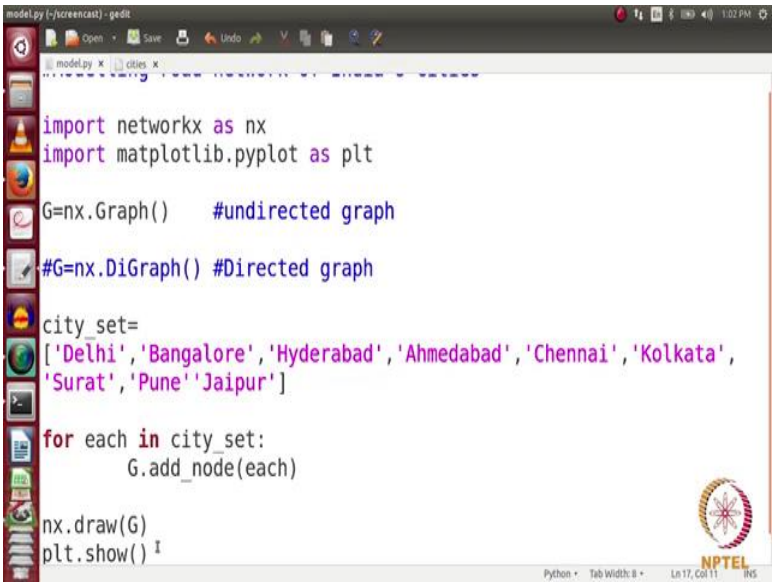
So, let me tell you there if the possibility of making directed graphs as well. So, a simple code by which you can make a directed graph is nx.digraph. So, if you use this command and next of digraph there will be a directed network created. So, it will be a directed graph here we are working with undirected graph. So, just come into this statement here. So, we have made a graph G equals to nx.graph here now I have a graph what I want to do is I want to add a nodes to this graph and every node should represent a city.

So, let us have an array of city here let say city set which is the set of cities where from where we will be choosing the cities. So, I have a file here just I looked at Wikipedia and looked at some of the cities let us pick some cities form here let us say, let us pick all these cities control c and let say I will put all these cities here. So, this city may insert strings. So, one thing well we are working with strings you will have to put these inputs.

So, I will put all of the inputs. So, I have a set of cities here. So, this is the set of cities now we want to create nodes in this graph which should be the cities.

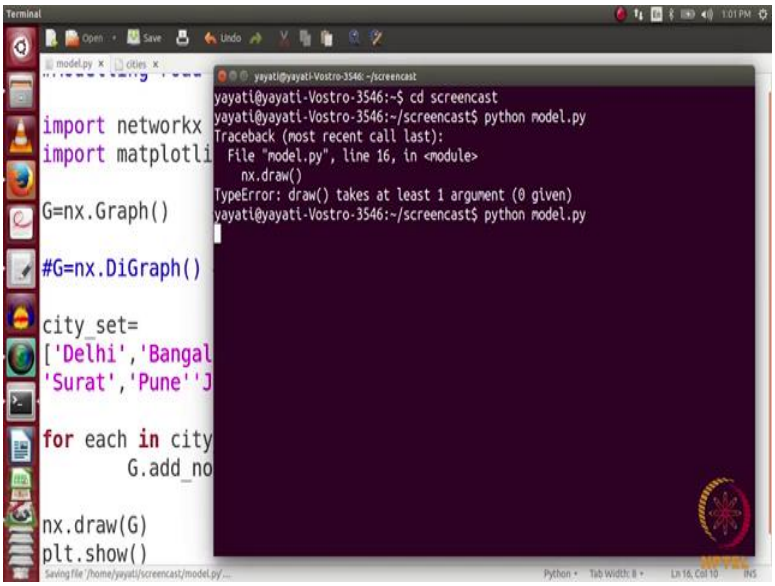
So, what we can do is. So, we have a list here we have to create cities and the values of the cities from should be from this list what we can simply do is for each in city set. So, if you would have gone through the module of using libs in python you would know that each is an iterater which will go through each of the elements of these lists cities set one by one. So, for each in city set what we want to do is we want to add a node this network. So, we do G.add node which is the command for adding node and each. So, we have these nodes and getting the network now I want to visualize this network we know that the patent which is used for visualization nx.draw and for visualizing we need to import one more module here draw import matplotlib.pyplot as plt.

(Refer Slide Time: 06:46)



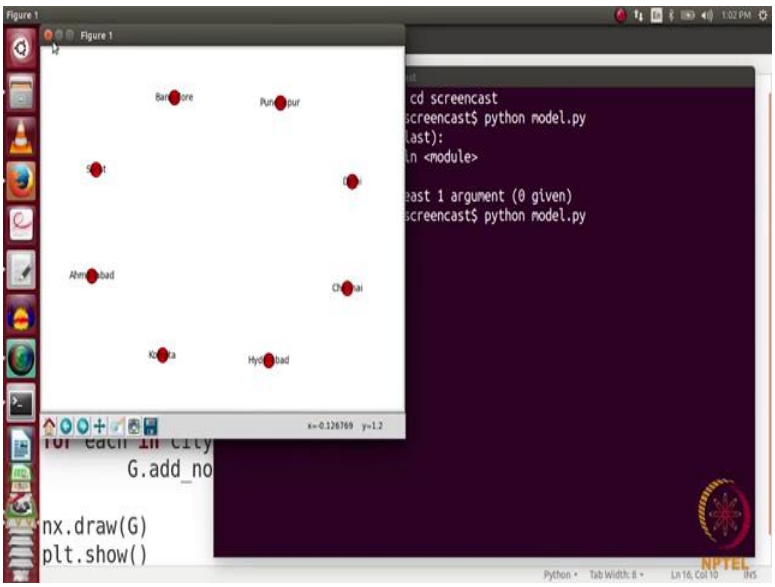
So, what we can do here is nx.draw and then plt.show. So, let us run this and let us see how does it work? So, I open up terminal window here.

(Refer Slide Time: 07:00)



So, I will mainly we will be in this terminal window here. So, we are in the home. So, we will first go to this folder, whose name was screen cast and then we run this python; python model.py some problem. So, it says that the draw function takes at least 1 argument. So, when we are nx.draw we need to pass the graph here which is G and then run it.

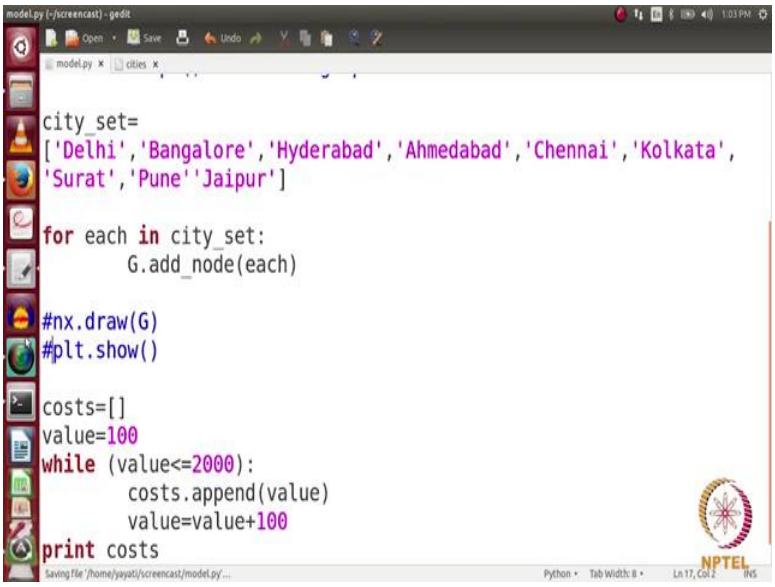
(Refer Slide Time: 07:35)



So you can here; that we have a network created where each node is one city in this network.

So, we added 1, 2, 3, 4, 5, 6, 7, 8 cities, we have added in this network perfect. So, now, we have a graph in which there are cities, there are 8 cities and next n is to add edges between these cities of roads between these cities and every road should have a have travelling cost associated with it. So, this travelling cost we represent it by the weight of an edge. So, we have to add edges here. So, how do we add edges it is the question. So, we are going to add these edges randomly for now how do we do that.

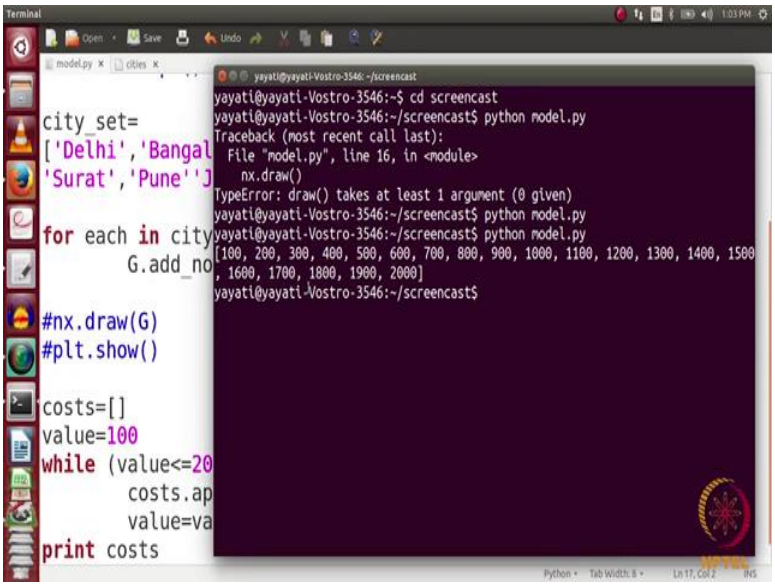
(Refer Slide Time: 08:15)



So, first of all we have a let say array costs what we are going to do is we are going to add some random edges in this network and each edge will have a random value random weight random travelling cost which will be picked from this lift cost. So, we want to first of all populate this list costs so that we can pick values form here. So, what do we do? Let us say I will add some 20 values here let say starting from 100. So, what I will be doing while value is less than or equals to 2000 costs.append and I will do here value and then value equals to value plus 1. So, it will iterate like 100 sorry it will iterate like 100, 200, 300, 400, it will keep up entering the value and it will go up to 2000.

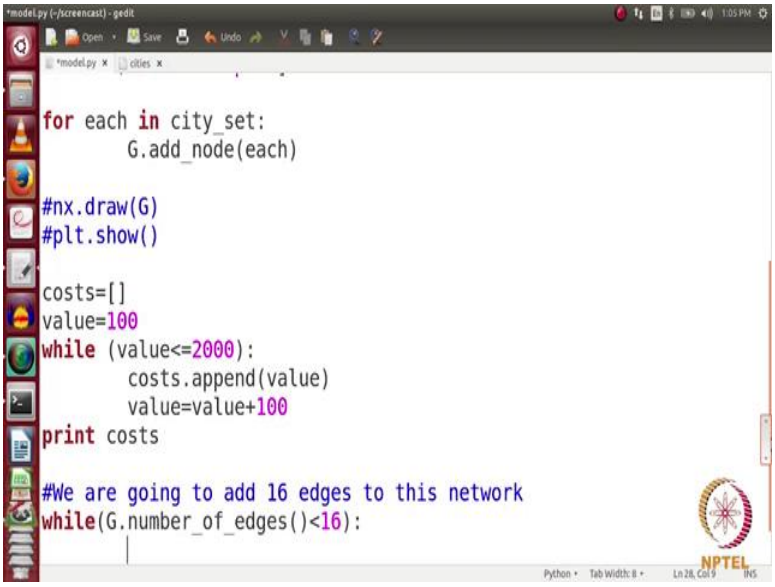
Let us see what this array looks like and we will just comment these 2 things.

(Refer Slide Time: 09:34)



So, this is how our array will look like it has values from 100 to 2000 these are array costs. So, now, we are going to add edges in this network assume that we want to add. So, there are 8 cities. So, for now let us add some 16 edges. So, we are going to add 16 edges through this network.

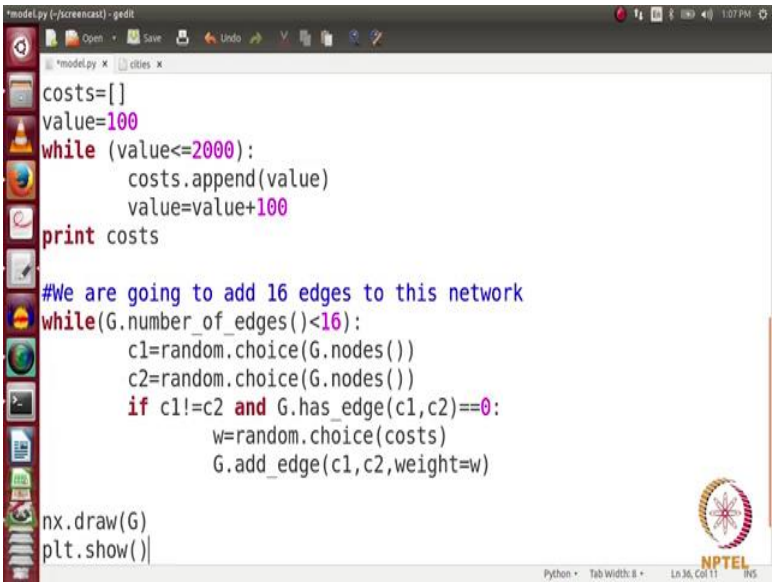
(Refer Slide Time: 09:56)



So, count is in English. So, we are going to add 16 edges to this network. So, till when your port should run while. So, for looking at the number of edges you have this code G.number of edges is less than 16.

So, this code will keep running till the number of edges in your network become 16 what you are going to do is we have choose 2 nodes randomly. So, for choosing a node randomly we note we need to import the function random. So, we import the function random here import random so that we can use it.

(Refer Slide Time: 10:58)



We come down. So, we choose city one equals to. So, if you want to choose a value from a list randomly. So, we have random.choice and we have function G.nodes. So, G.nodes give you a set which has already nodes in these networks. So, we are going to choose one of these nodes randomly that is their city 1 and then we choose city 2 which is again random.choice and again G.nodes.

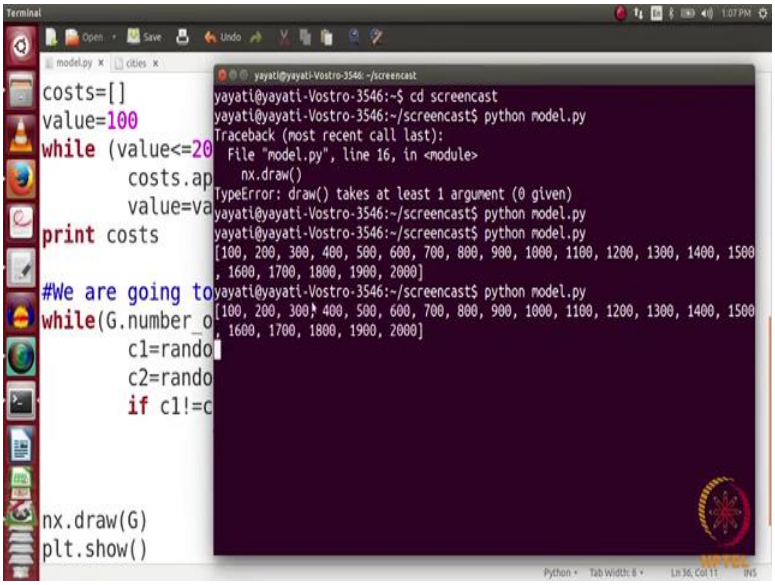
Now, one thing to note here is these 2 cities should not be same. So, only if c 1 is not equals to c 2, we are going to move next what we are going to do is we are going to choose a weight for the edge. So, let say weight which is the travelling cost now this is nothing, but random.choice again a random function and it will pick a random value from our array costs also. So, the one condition for creating an edges both of these cities should not be equal and the second condition is there should not be an edge which is already present between these 2 nodes.

So, for that we have function and G.has underscore edge it tells you whether there is an edge between 2 nodes c 1 comma c 2. So, has edge c 1 comma c 2 should be equal to 0 there should be no edge between them. So, in that case we will choose a weight here w

equals to random.choice costs and then we will add an edge in this network G.add edge and we add edge from c 1 comma c 2 and the weight of this edge is w which was a random number. So, this code keeps running this loop keeps running till we have 16 edges in the network.

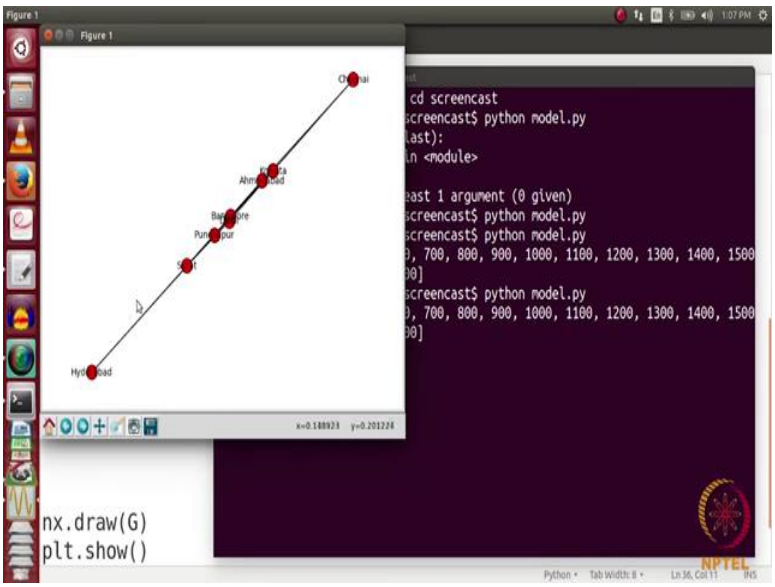
Again I want to see these networks. So, I use nx.draw G and then plt.show.

(Refer Slide Time: 13:06)



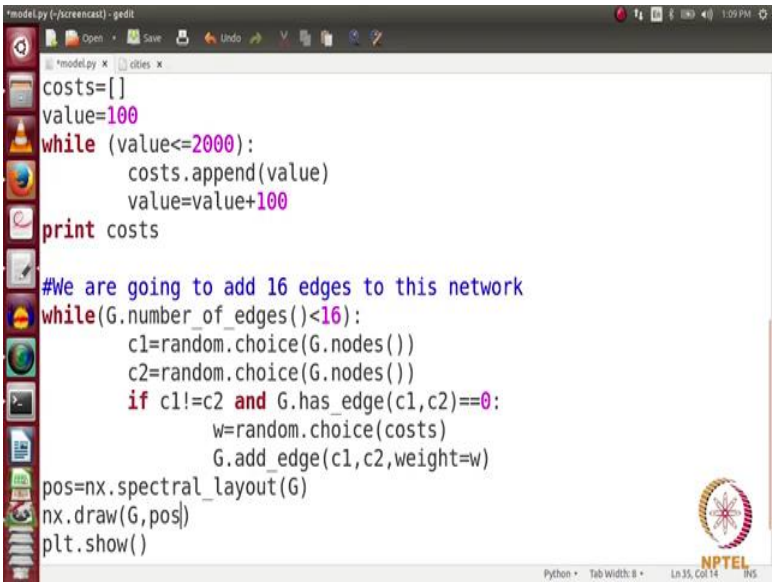
Let us execute this in c.

(Refer Slide Time: 13:09)



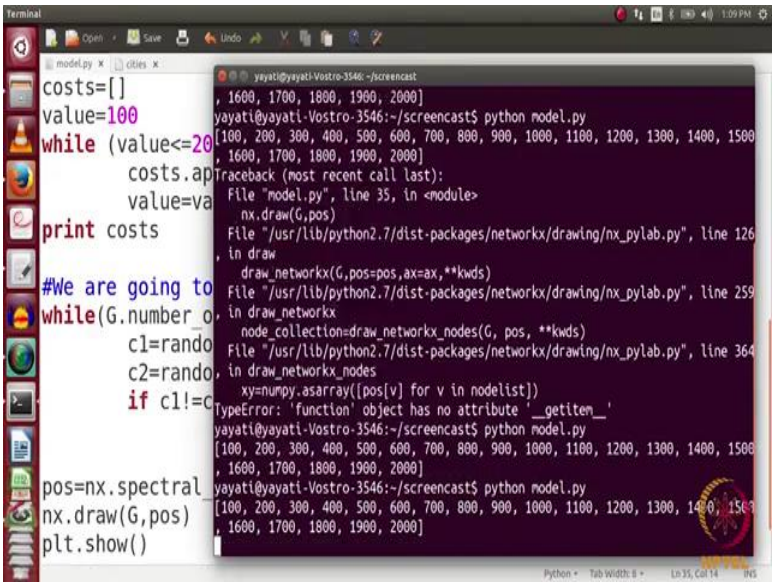
So, you see here. So, the edges are not very clear some of the edges are overlapping here. So, what we can do for this is we can change the layout of this graph. So, one of the layout that we can use is the spectral layout. So, for using this spectral layout we have a command.

(Refer Slide Time: 13:40)



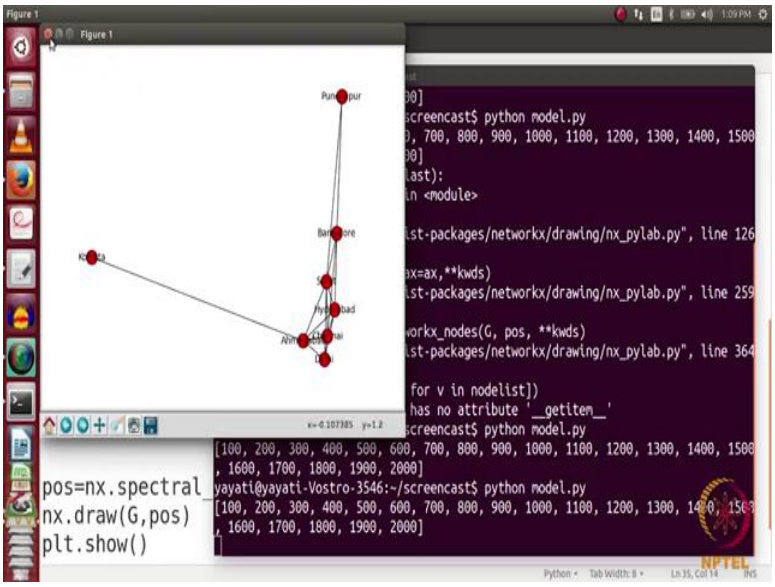
So, let us say we want to choose a layout pos equals to nx.draw underscore spectral (Refer Time: 13:52) this p o s here (Refer Time: 13:54). So, the command here is nx.and we want a spectral layout. So, we using spectral underscore layout we will pass a graph G here and while drawing it we use nx.draw pos.

(Refer Slide Time: 14:20)



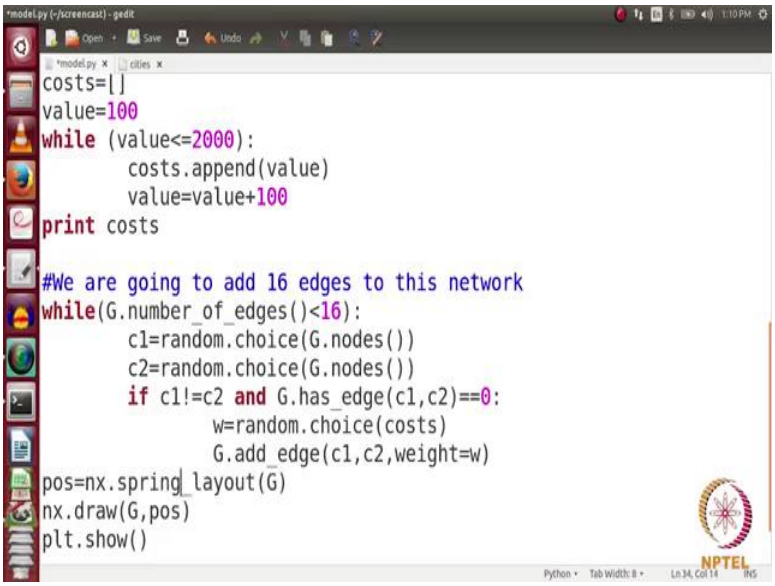
So, let us now visualize these graph python model.py.

(Refer Slide Time: 14:22)



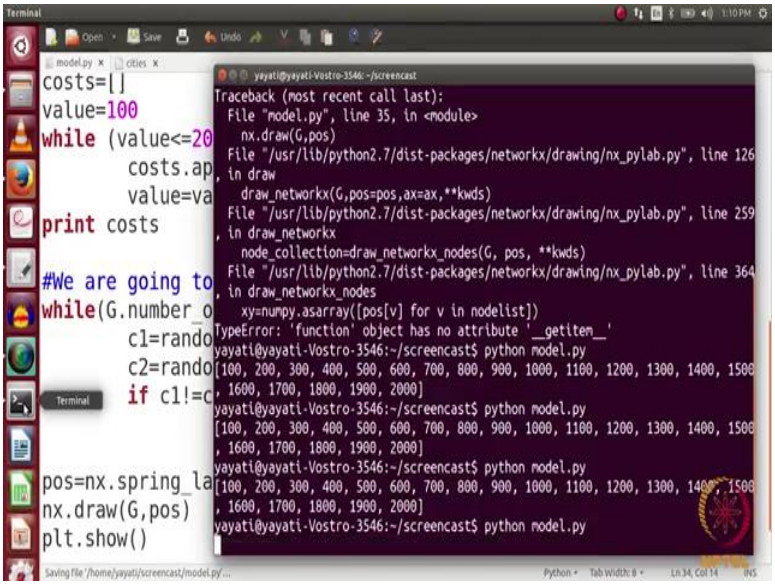
So, when we see it looks better than before ok we can also use some other layouts.

(Refer Slide Time: 14:35)

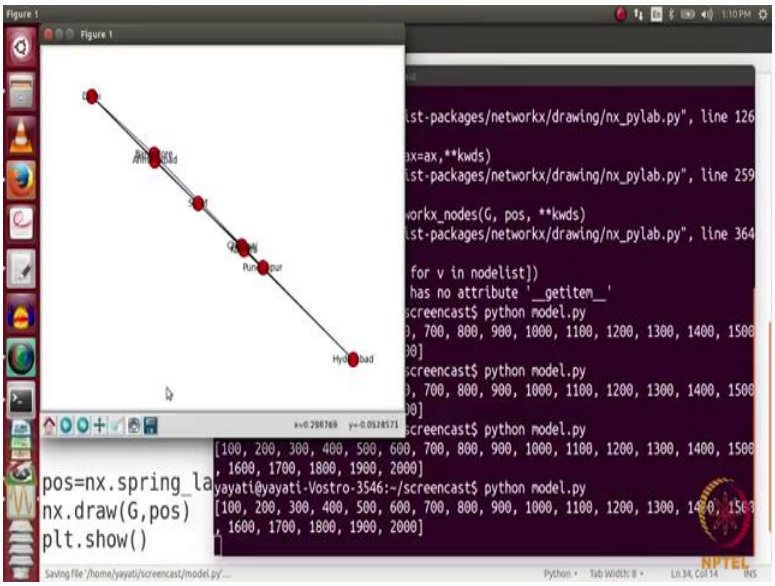


Let us say; let us try to use a spring layout look very nice and circular layout might look better here.

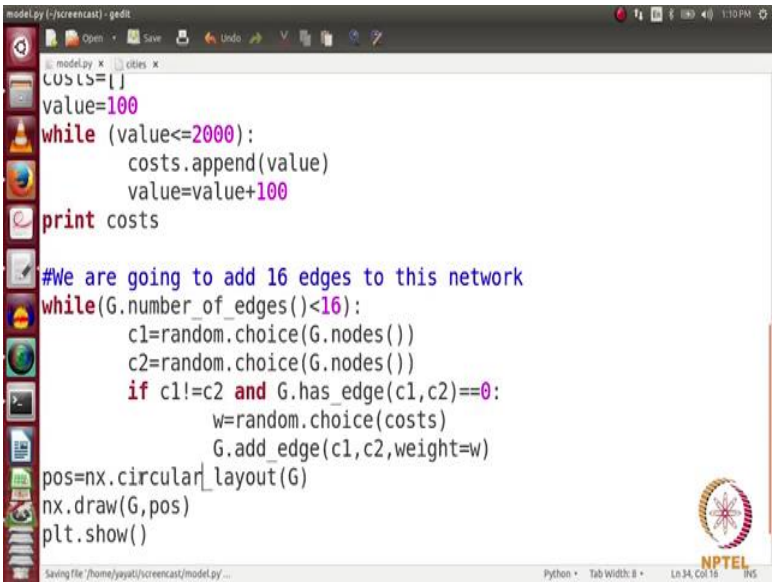
(Refer Slide Time: 14:37)



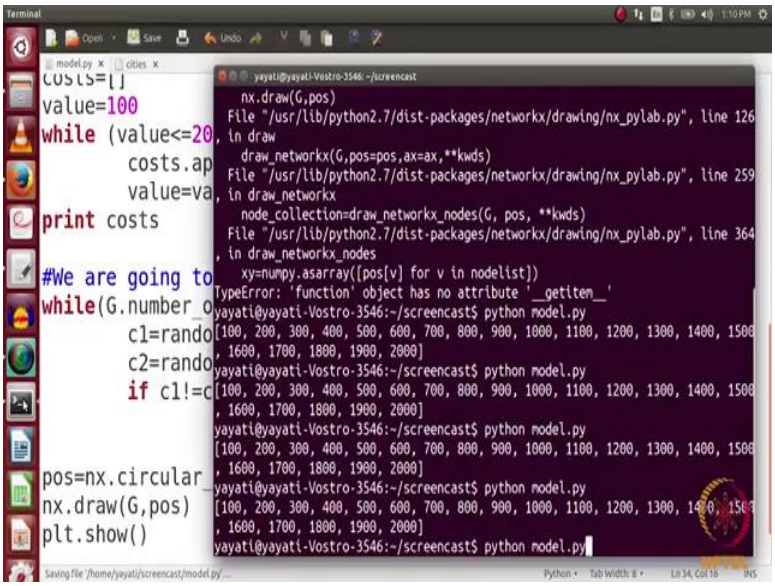
(Refer Slide Time: 14:37)



(Refer Slide Time: 14:43)

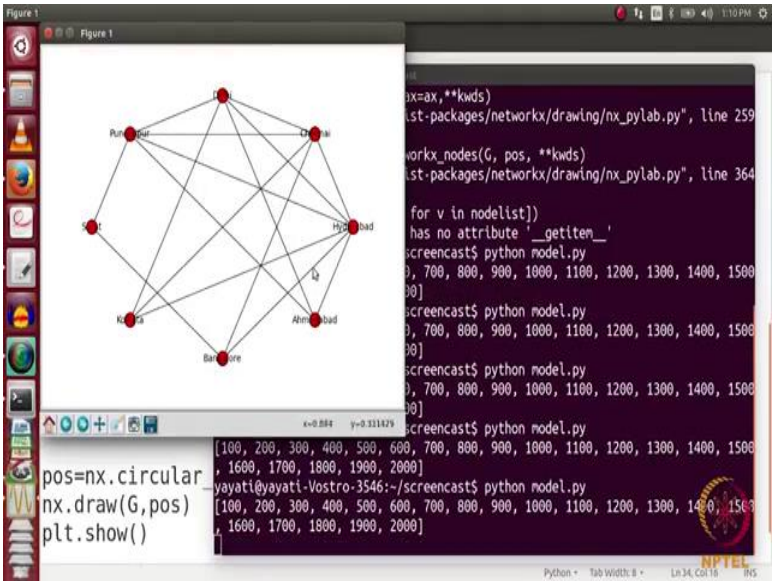


(Refer Slide Time: 14:45)



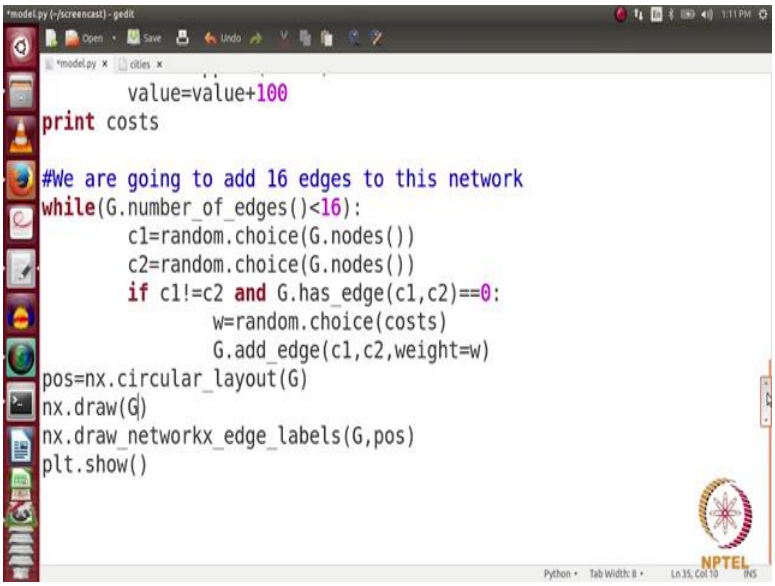
Let us try the circular layout. So, this one looks perfect. So, we have drawn this graph in the circular layout.

(Refer Slide Time: 14:46)



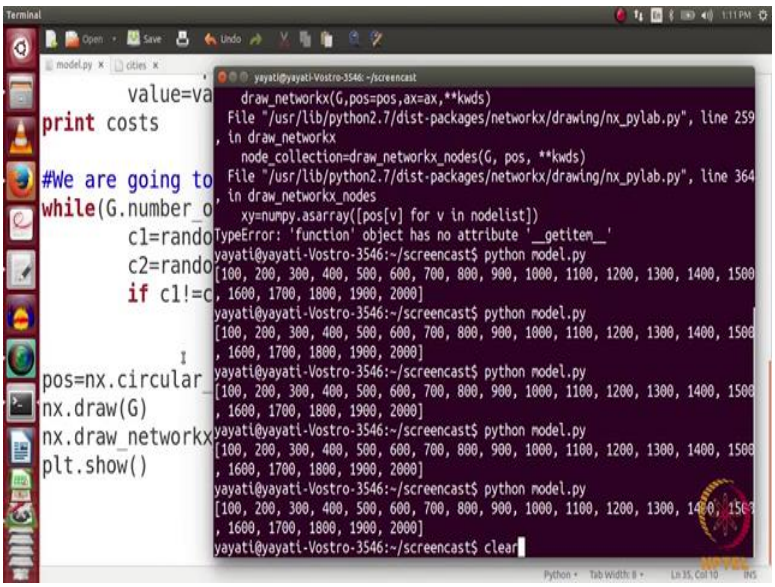
And it looks nice here. So, there are 8 cities and they have 16 edges and each of this edge has some edge weight associated with it. So, you might want to look at this edge weights as well. So, for labeling these edges with their edge weights, so here we have drawn the networkx and let us draw their edge weights are also here.

(Refer Slide Time: 15:11)



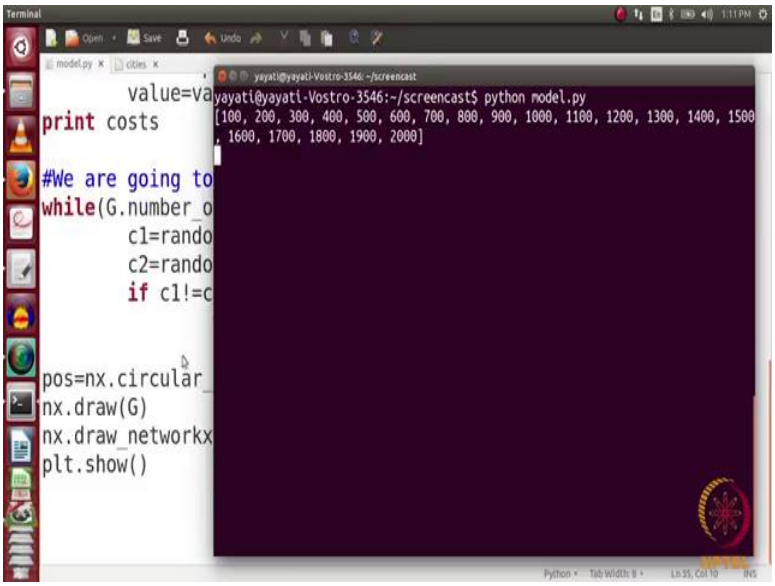
So, we have this command nx.draw underscore networkx underscore edge underscore labels and then we pass here G comma p o s.

(Refer Slide Time: 15:32)



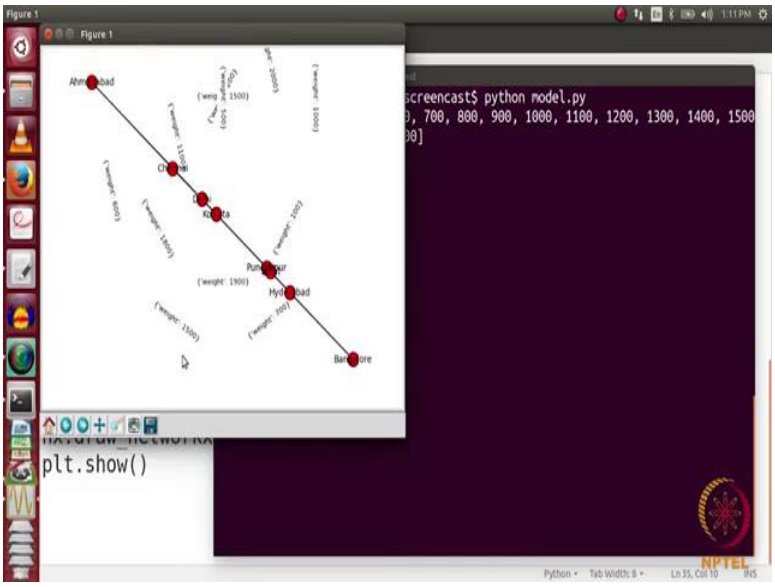
Let us see how does it work and then let me just clear it.

(Refer Slide Time: 15:36)

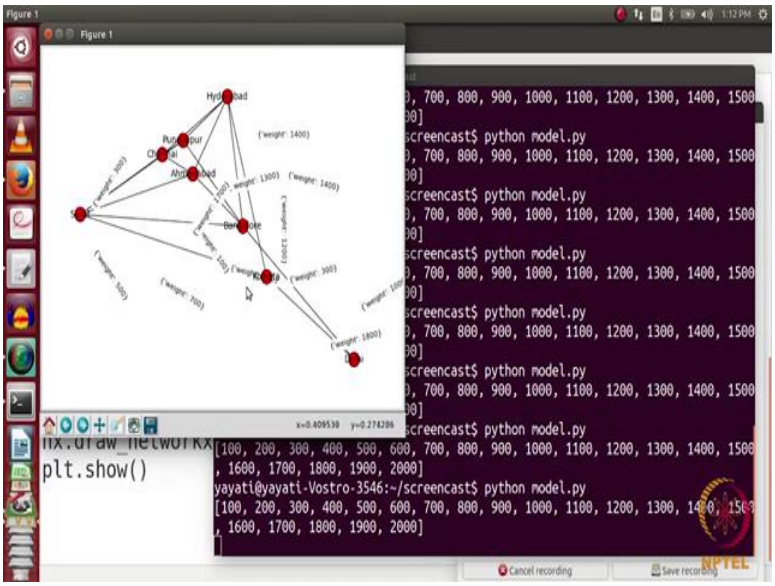


And again we run python model.py.

(Refer Slide Time: 15:37)



(Refer Slide Time: 15:51)

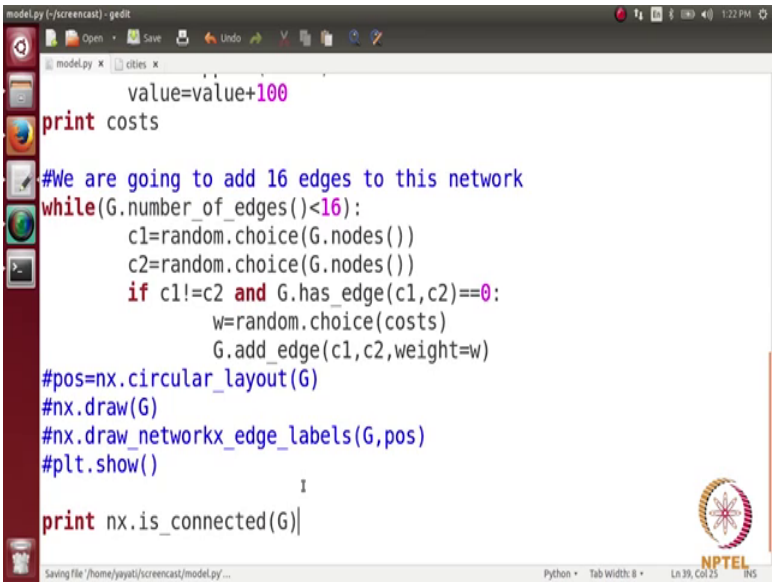


So, you see here, it has to be tried some number of times and when you draw it in a circular layout it turns out to be a pretty nice configuration here no its not very clear, but yeah still you can see the edge weights here. So, now, we have a network which has some 8 nodes and 16 edges and we have visualize this network as well we have drawn these edges randomly. So, we have drawn 16 edges randomly between random pair of nodes.

So, we have now created the network with the cities and the roads what we are next interested in his looking at the paths between these cities and the cost of travelling across that paths as well. So, first let us look at the path between these cities we want to look at the path length between every 2 possible pair of cities and by path length what I mean here is now the some of the weights of the path just to see whether I can reach from city a to city b or not. So, first of all let us that whether even this graph is connected or not because of the graph is connected it means that I can reach from every city to every other city and so on.

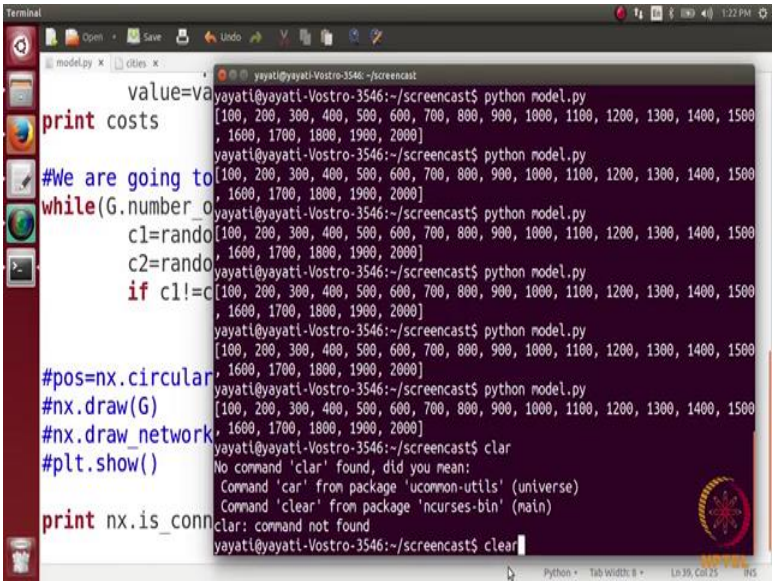
So, let us look at whether this graph is connected or not.

(Refer Slide Time: 17:14)



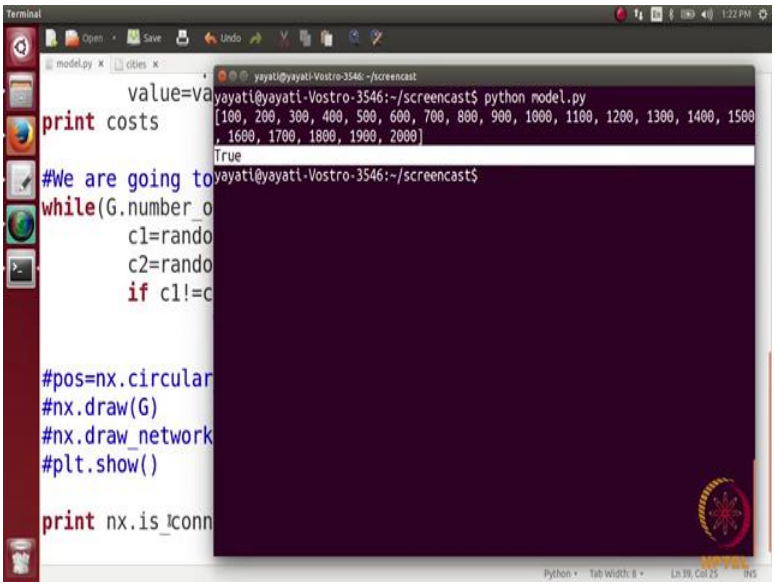
So, we will just print, so we have a command nx.is connected G which tells us whether this graph is connected or not.

(Refer Slide Time: 17:38)



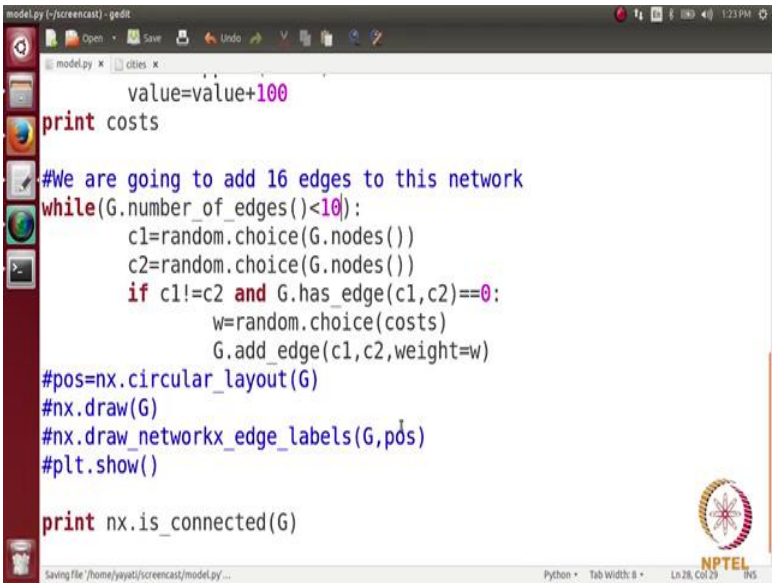
So, let us see, just clear it.

(Refer Slide Time: 17:40)



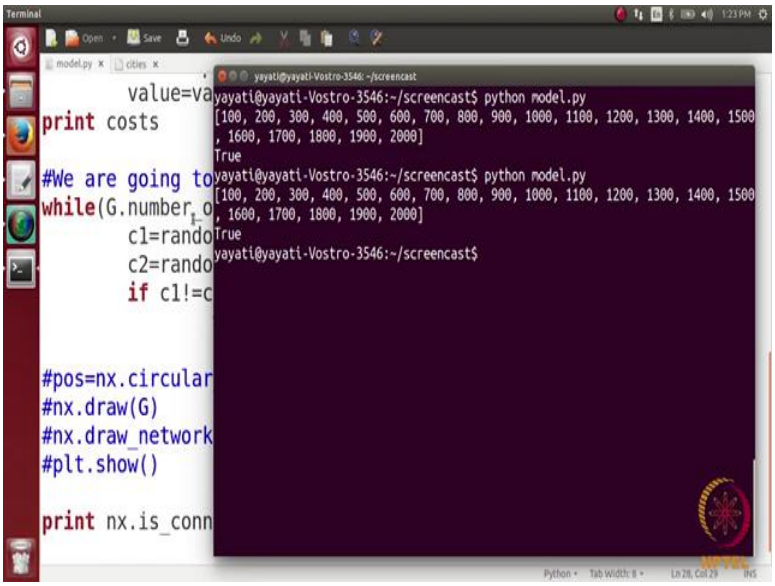
See here at the graph is connected it means that there is path between every 2 pair of nodes. So, this, so, what we want is we want to delete some edges let us say. So, that we do not have some paths between some pair of cities.

(Refer Slide Time: 18:07)



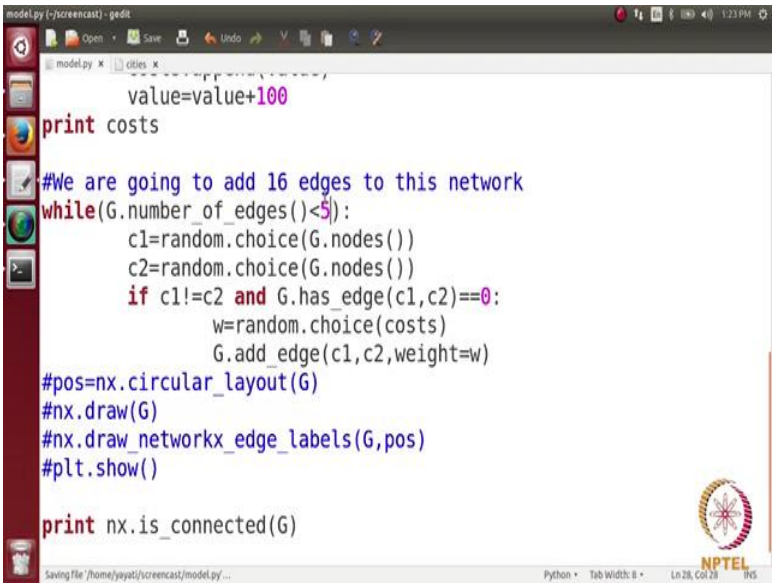
So, what we can do for that is if we can do for doing that is let us reduce the number of edges here from 16 to let say that we will just have 10 edges in this network and let see what happens in that case.

(Refer Slide Time: 18:20)



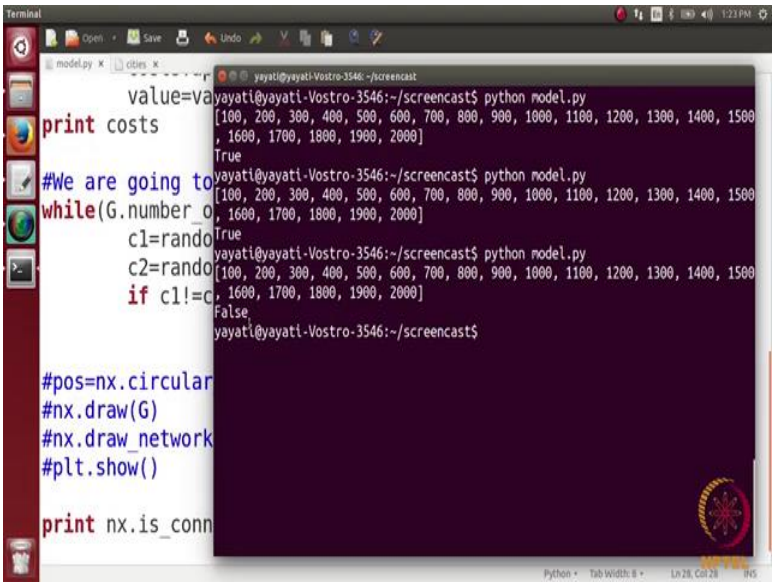
So, we have done edges again the graph is connected. So, it is actually interesting you see it is a network on 8 nodes and we have put just 10 edges even then this turning out be connected.

(Refer Slide Time: 18:34)



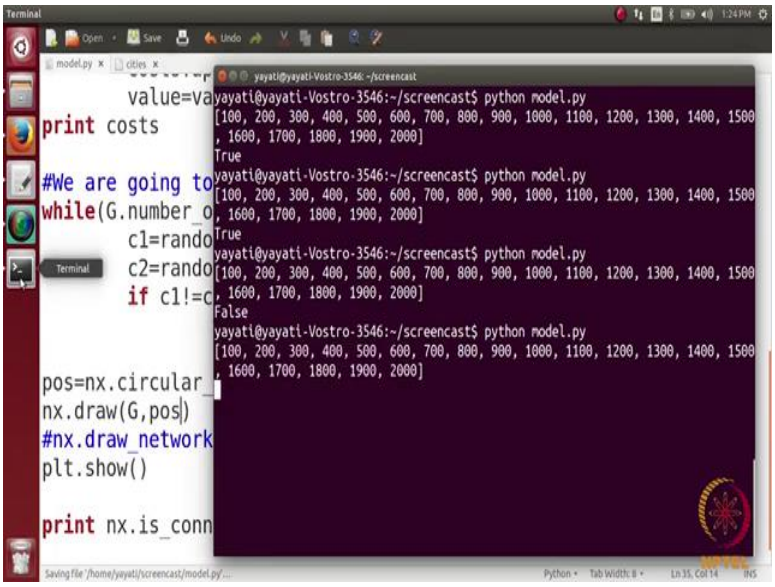
Let us reduce it to 5, let say and then when we put the number of edges be 5, it turns out that the network is not connected. So, it is false network is not connected.

(Refer Slide Time: 18:41)

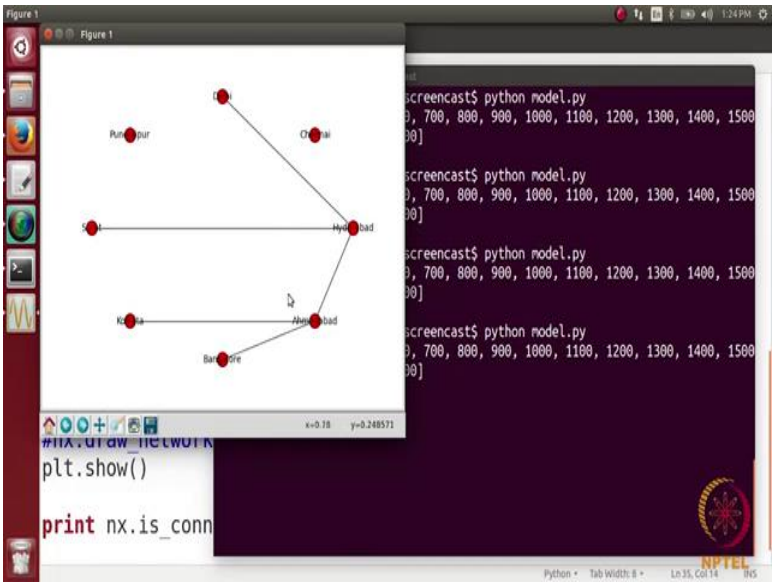


So, let us look at this network as well.

(Refer Slide Time: 18:54)

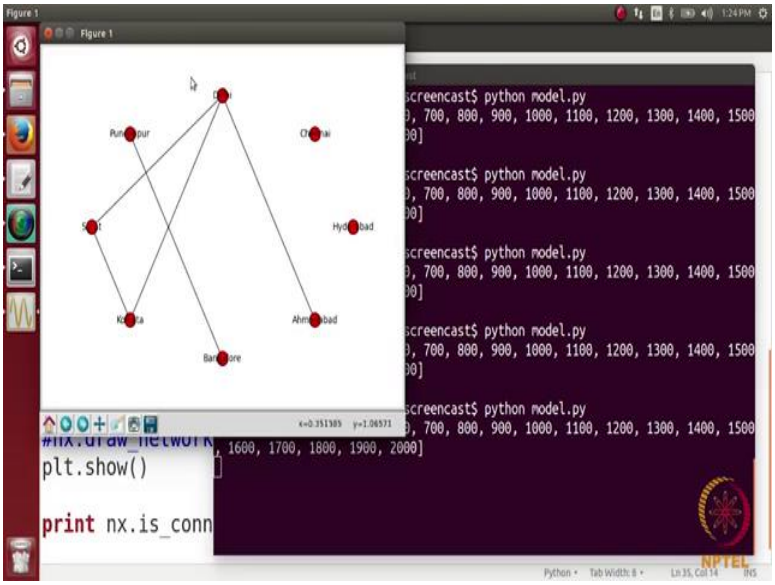


(Refer Slide Time: 18:58)



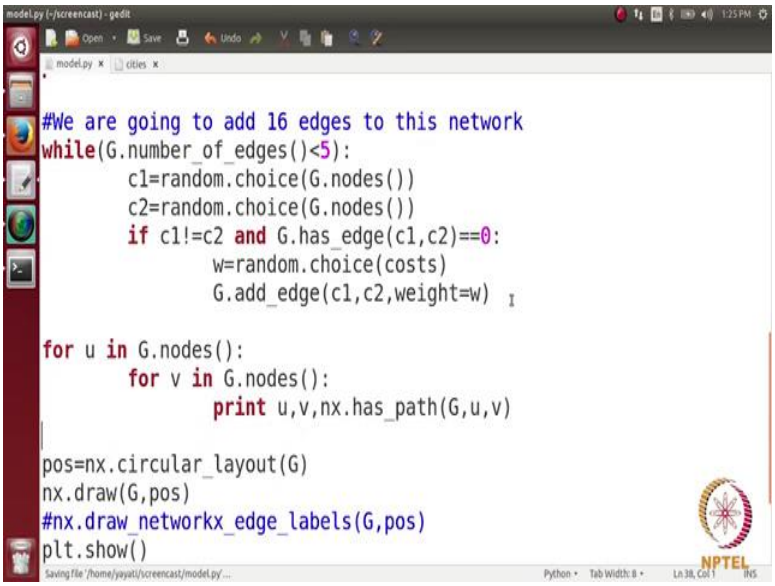
Now, you see the network here right. So, this there is this one city which is not these are which are not connected at all to any of these cities.

(Refer Slide Time: 19:06)



So, let us take one more example. So, it can be like this and so whom, so what is now we want to see assume that we want to see path between which cities path exists and between which cities path does not exist.

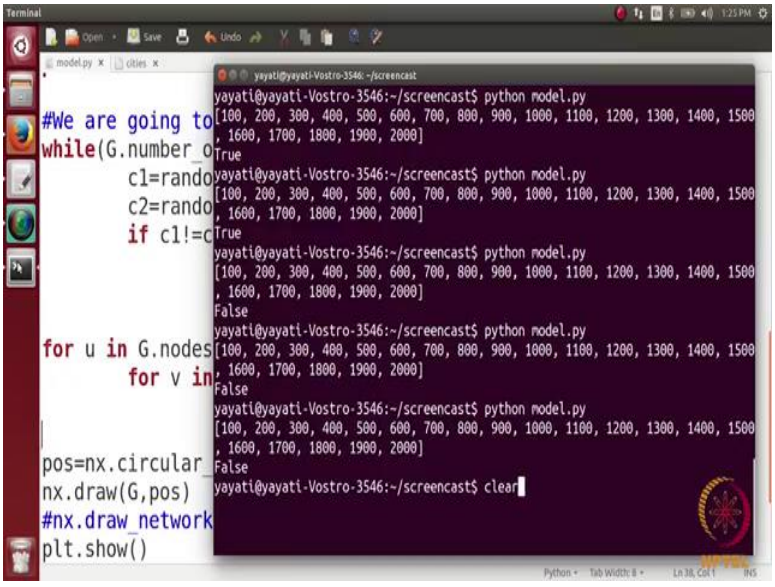
(Refer Slide Time: 19:23)



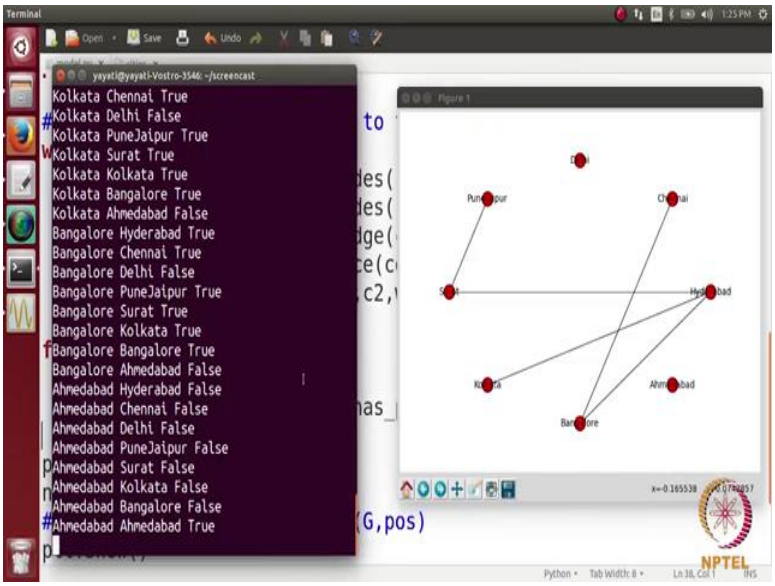
So, what we do for that is for each possible pair of cities we see that whether there is a path between them or not. So, we have a loop here for let us say u in G.nodes and then for v in G.nodes what do you do is print u v comma and then we have a function here nx.has underscore path which tells us whether there is a path between these cities or not.

So, we pass here it G u comma v right. So, what I will do is I will put this code before drawing the networks so that we can see both the things simultaneously and let see now just in.

(Refer Slide Time: 20:13)



(Refer Slide Time: 20:18)



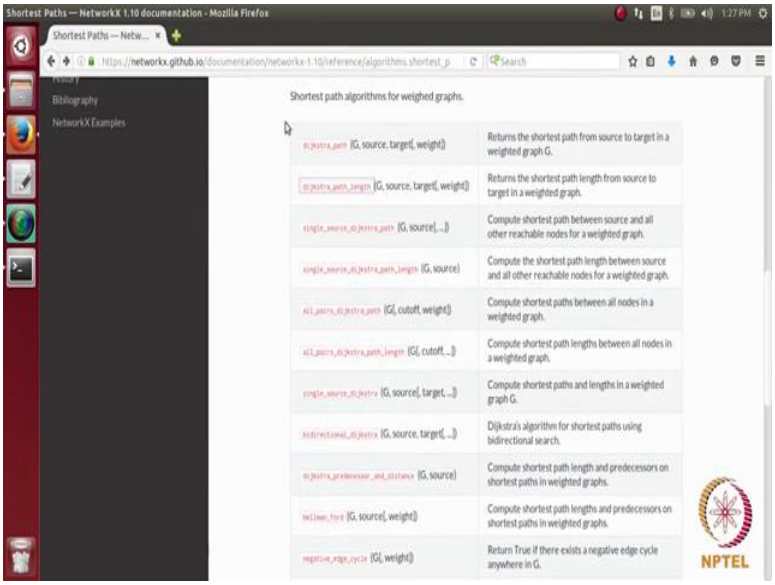
So, you see here. So, this is our network and what is. So, perfect. So, this is our network here and these are the paths. So, it tells you for every possible pair of cities. So, Hyderabad to Hyderabad is; obviously, path is there Hyderabad to Chennai there is a path Hyderabad to Delhi there is no path. So, you see here from let say Hyderabad to Pune and we look from let say Hyderabad to Pune I am very sorry we have mixed within Pune and Jaipur. So, let us say its Pune.

So, from Hyderabad to Pune we looked at there is a path. So, you can go from Hyderabad to Surat and from Surat you can go to Pune. So, this is a fictitious network if the connections might not be really this way it just a fictitious. So, do not look at it in real world then how come Hyderabad is connecting to Surat and then to Pune and so on. So, there is a path from this node Hyderabad to a node here. So, we have listed it for every 2 pair of nodes. So, you can looked at; then from Chennai we can go to Hyderabad to let say Bangalore. So, again you see here if you look the connection between let say Chennai and Hyderabad which is somewhere here yeah. So, it turns out be true and from Delhi you see from Delhi you cannot go anywhere according to these networks. So, every value from Delhi turns out to be false and so on, just close it, stand here.

Now, next what we want to look at is what are the shortest paths in this network. So, we have some really (Refer Time: 22:25) functions in networkx. So, which are the basic

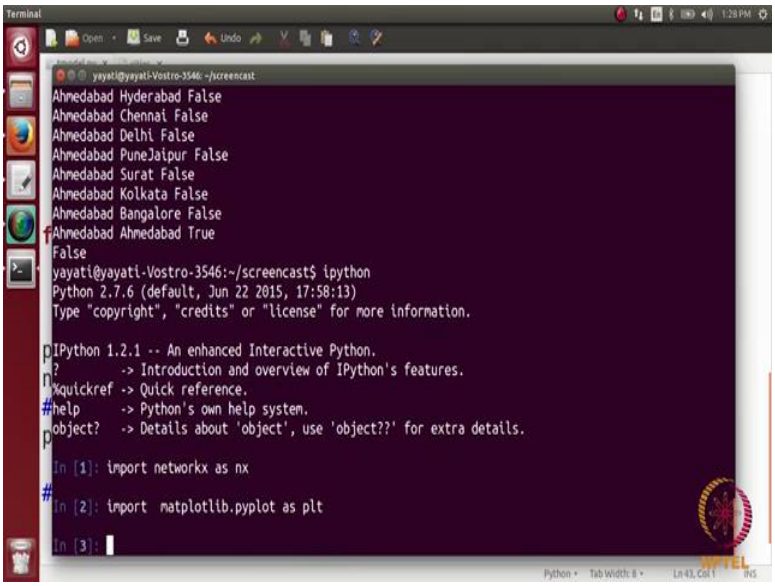
standard algorithms implemented in networkx with the help of which you can see the path length of the travelling cost between 2 cities.

(Refer Slide Time: 22:41)



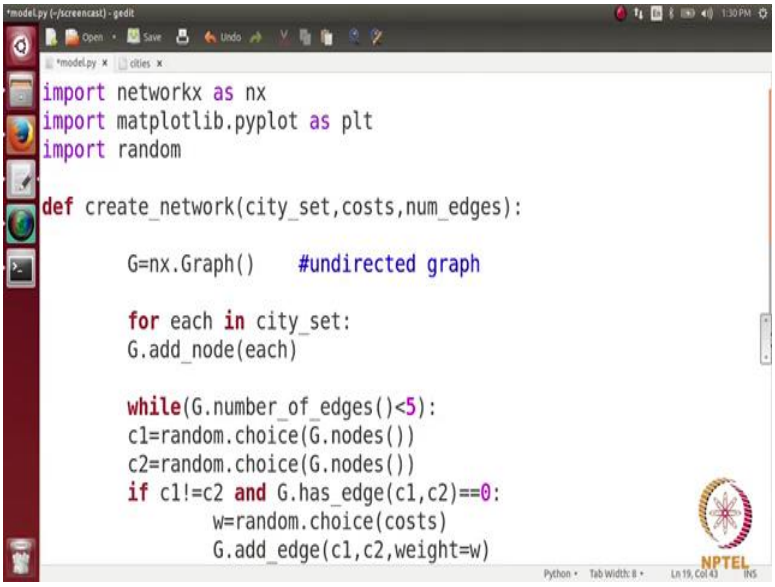
So, let me quickly show you some of these. So, these are here. So, you can see that these are the functions. So, we have one function discussed standards discussed standards cold path which tells you the shortest paths from source to target and then underscore path length which. So, given 2 nodes it tells you the length of that shortest path then single source discussed a path where you give one source node and it gives you the all the shortest paths and so on. So, we will we are going to use some of these functions and see how do they work in our network? So, for working with this network what will be going to do is let us now switch a little bit to the interactive ipython.

(Refer Slide Time: 23:30)



So, what we are going to do is we are going to turn to ipython and then we are going to import network ipython then we are going to import networkx as nx then import matplotli.py pyplot as plt perfect. So, we are going to define some functions here. So, let us con let us make this code a little bit of deny stand make some functions from this.

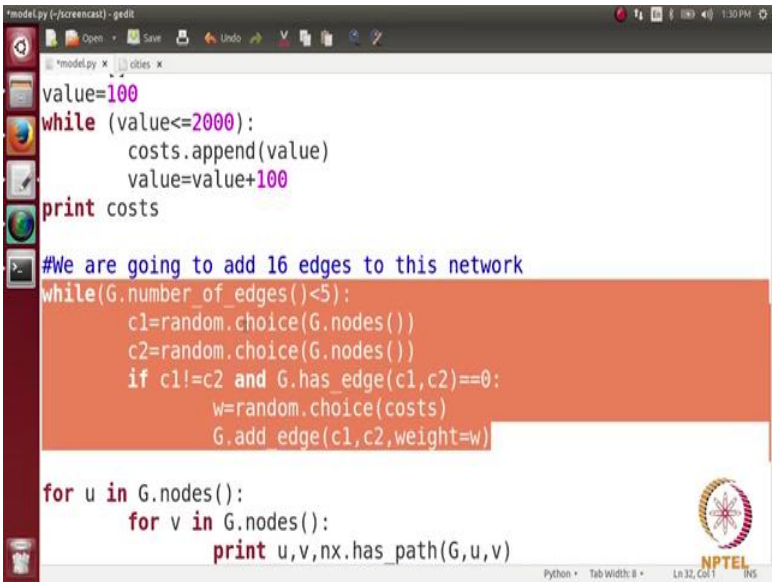
(Refer Slide Time: 23:59)



So, let me define a function fine create network and let say this function takes us input the set of cities the array costs and let say number of edges which you want to create.

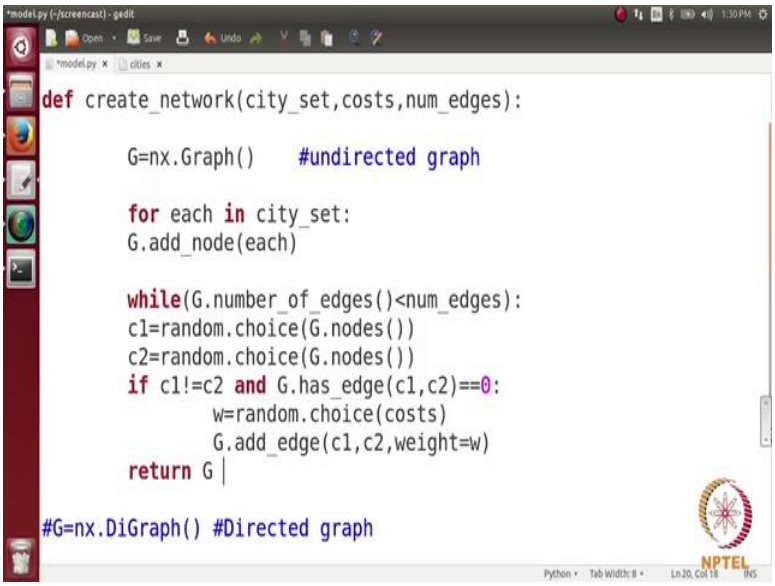
So, what all we will have this inside this function is this term will come here. So, G equals to nx.graph and then for each in city set we are going to add an edge. So, this particular code comes here for each in city set we are going to add a node and then what else.

(Refer Slide Time: 24:54)

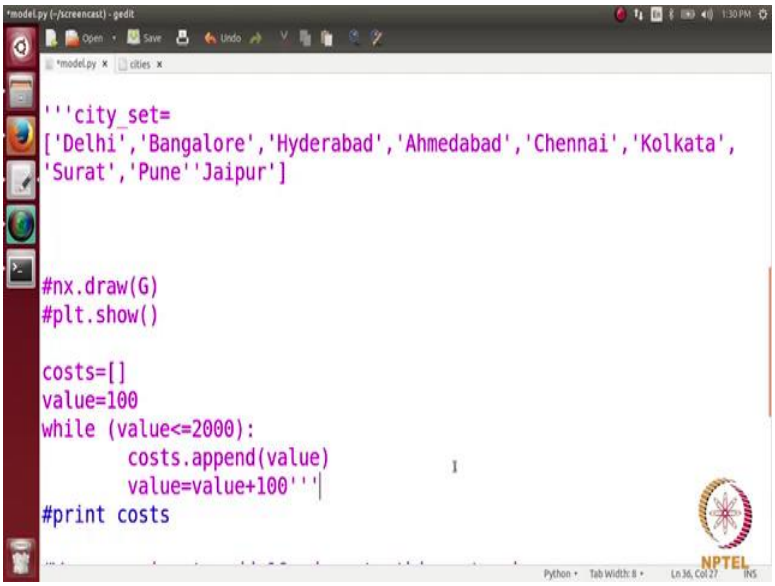


So, this entire code will be shifted there. So, this is when we are we were adding the, we were adding 5 nodes in this networks. So, will basically change this code first of all let us paste it here. So, why 0 number of edges is less than; so we are adding num edges number of edges. So, let me make it num edges here and then of right and this will return you this graph G which you can then visualize.

(Refer Slide Time: 25:31)

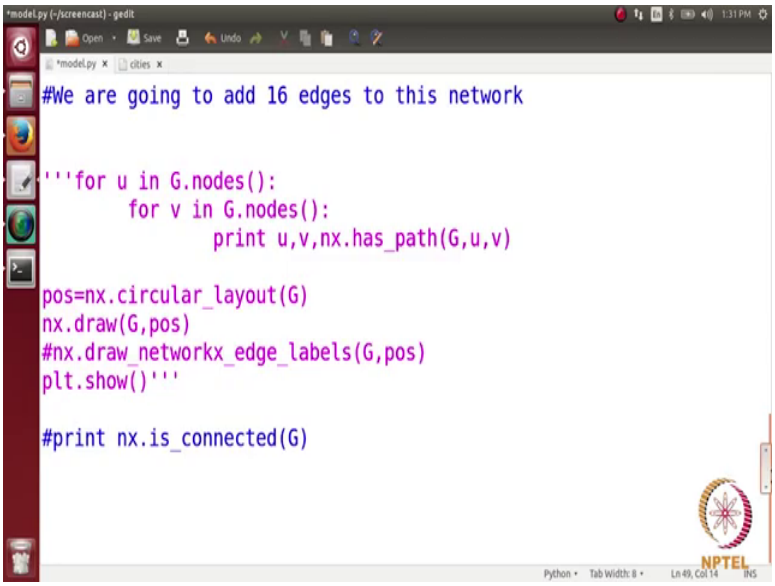


(Refer Slide Time: 25:48)



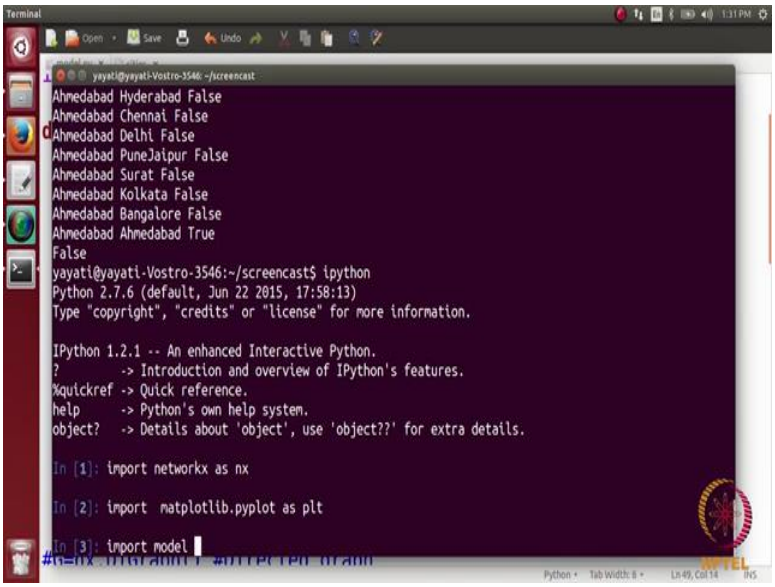
Next comment rest of are go further mean while write commented rest of the codes. So, everything is most and mostly commented here.

(Refer Slide Time: 25:57)



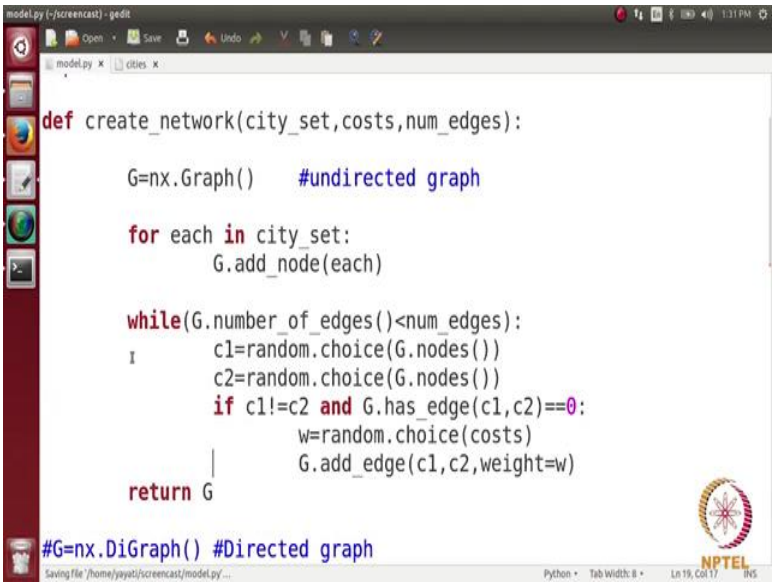
Everything is commented except for this function here. So, this function is simply what it is doing taking a city set taking array costs taking the number of edges and creating the graph.

(Refer Slide Time: 26:18)



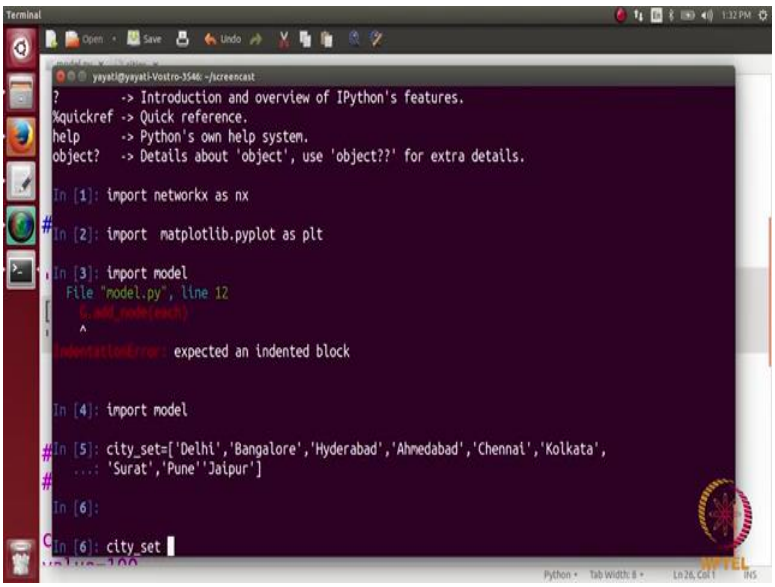
So, how do we do it here? So, we are going to import this model here. So, when added it line 12. So, this is the 4 loop.

(Refer Slide Time: 26:30)



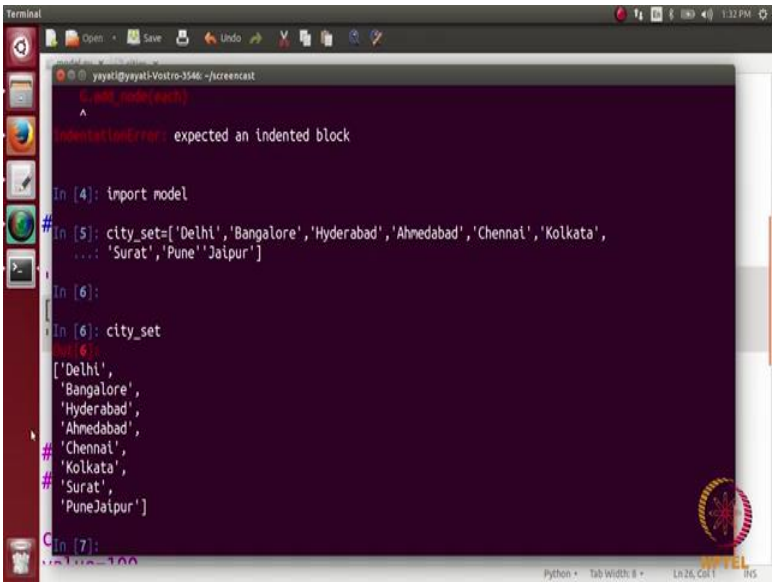
So, here you have to always take care of the indentation and let us import model and we have imported a.

(Refer Slide Time: 26:40)



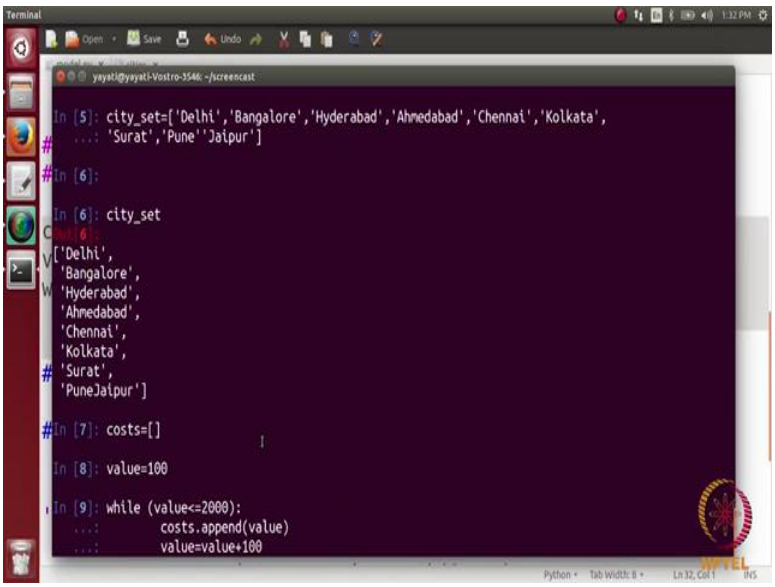
So, to call this function we should have some parameters which are the city set the costs and the number of edges. So, let us define these parameters here. So, I will just copy paste the set of cities here. I have a array here for the city set we can also see this array here.

(Refer Slide Time: 27:03)



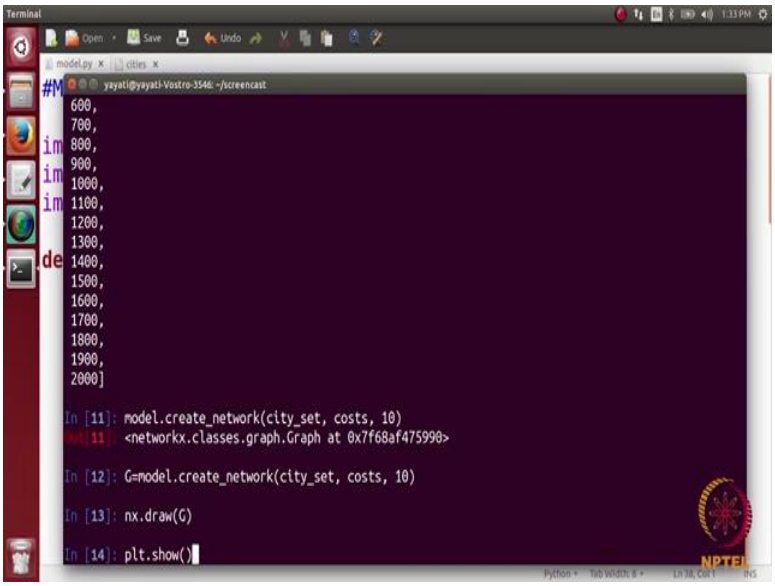
So, we have this city set and then we want these costs to be here. So, in will just take this code yes paste it here.

(Refer Slide Time: 27:16)



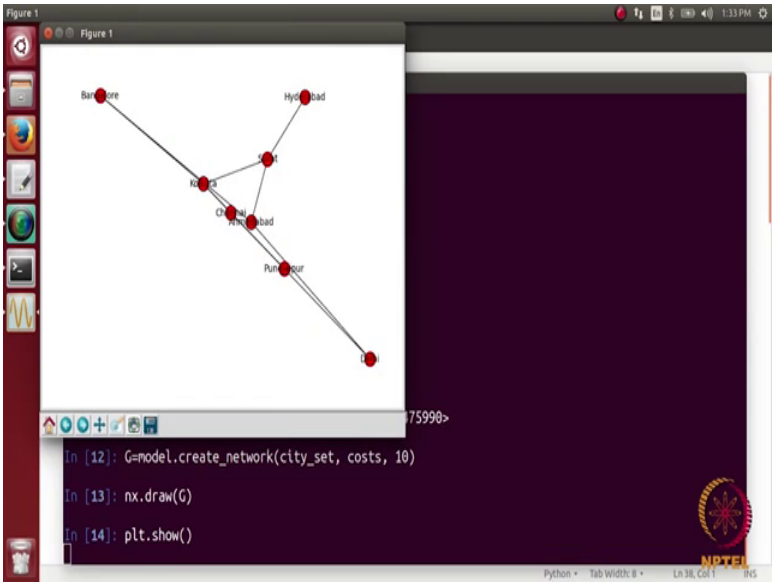
Let us look at the array cost.

(Refer Slide Time: 27:20)



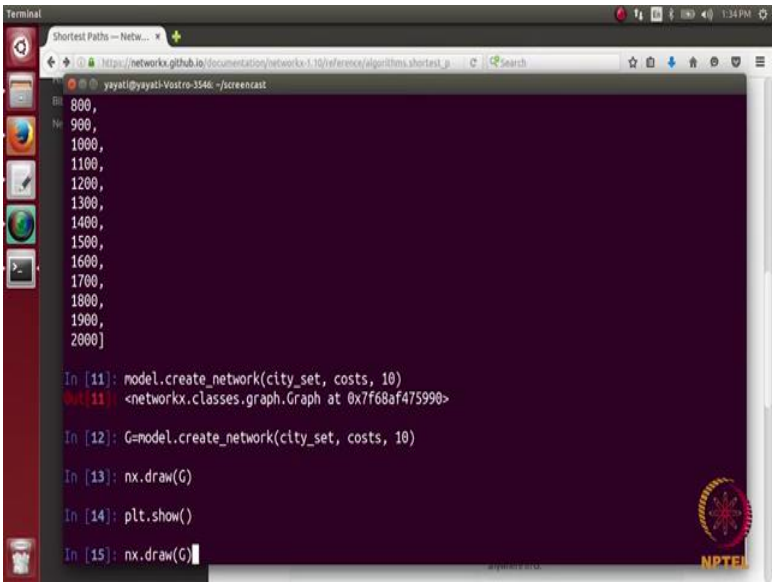
So, we have this array costs here and let say I am going to add some 10 edges. So, let us add 10 edges. So, we are going to call this function and the function name is create underscore network. So, what we are going to do is model.create underscore network and here we are going to pass the 3 parameters. So, what are a parameters one is the cost city set young. So, we are going to pass here city underscore set and then say costs and then we are going to at 10 edges in this network and. So, we have took collect this network in a variable let say this variable is g. So, G equals to this. So, let us try looking at this graph, so let say nx.draw G and then plt.show.

(Refer Slide Time: 28:24)



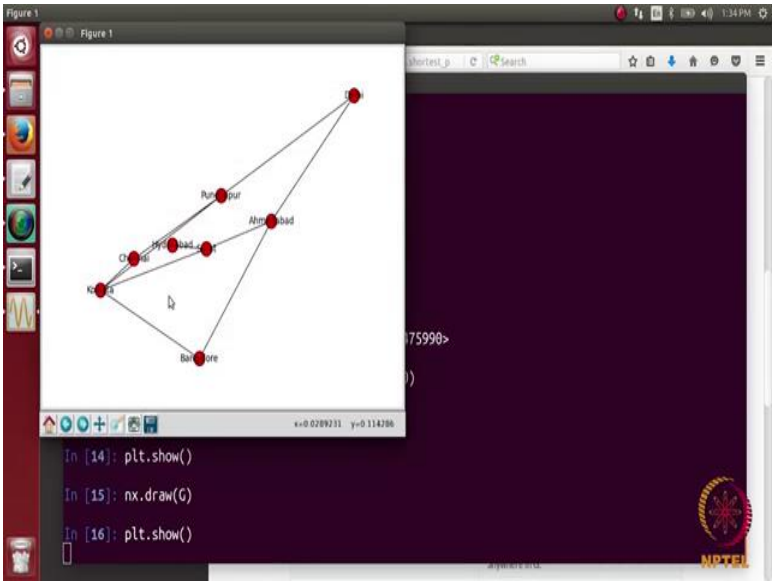
So, we can see this network here and it is connected. So, we have a graph here and every edge has a weight associated with it here. So, now, we what we want to see is we want to see the implementation of the shortest path functions. So, let us one by one take the shortest path functions and see how do they work? So, let us look at the first one here. So, if you give it the graph and the source and the target it will return you the shortest path let us try.

(Refer Slide Time: 29:00)



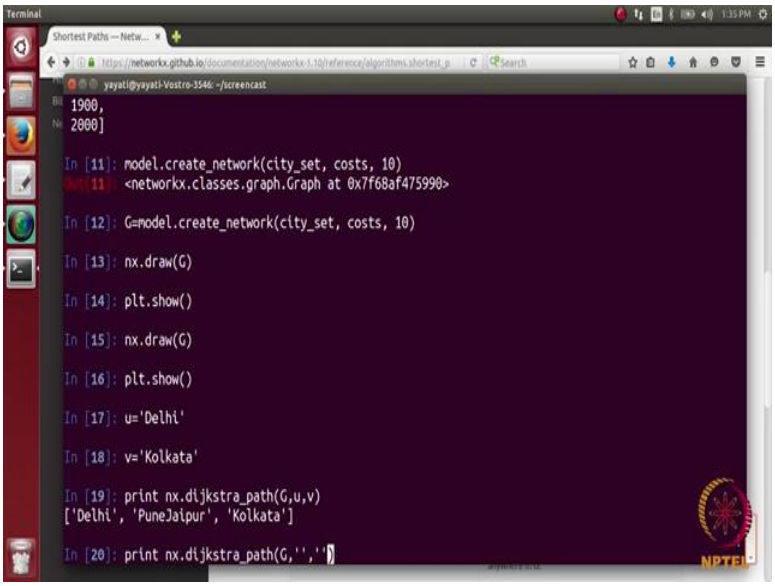
So, let us look a graph once more.

(Refer Slide Time: 29:02)



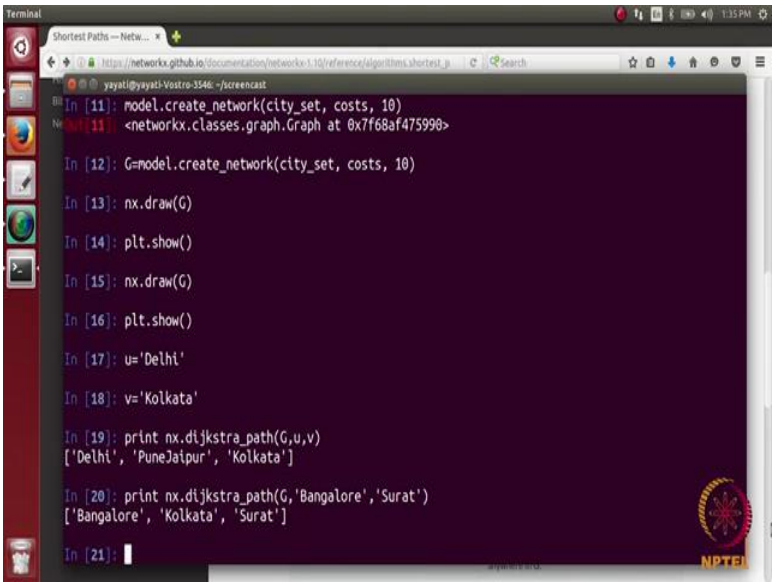
So, let us choose 2 nodes here let say we choose the aliant Kolkata.

(Refer Slide Time: 29:10)



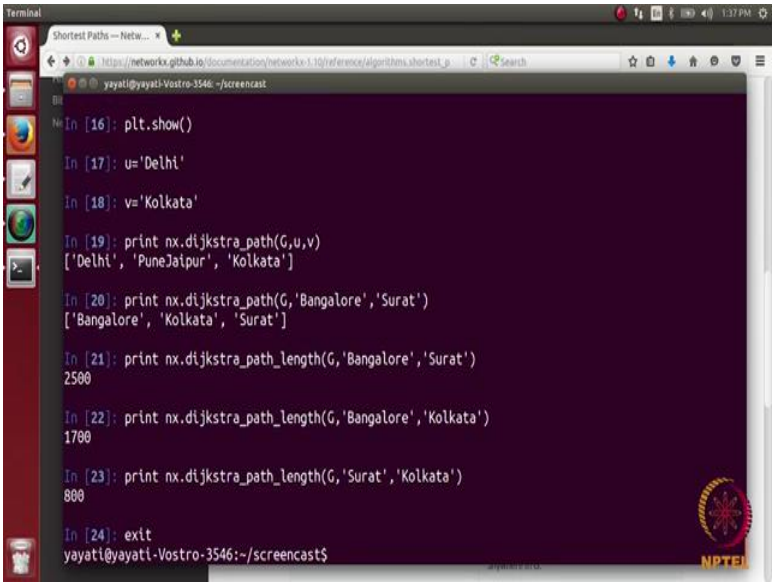
So, let say node number one u is Delhi and second node is now what we are going to do is we are going to look at the shortest path between these 2 nodes. So, what we are going to print is nx.stra and then goes there underscore path and then you paths here network here and the first node and the second node yeah. So, you get here shortest path here. So, from Delhi you go to Pune and then from Pune you can go to Kolkata, similarly we can actually see between every let us choose another some 2 different cities let say let us choose Bangalore and Surat. So, let us write it down Bangalore and let say yeah. So, you see here that Bangalore and Surat are connected with Kolkata and this is the shortest path of it.

(Refer Slide Time: 30:27)



Next function is for path length where you do not want to see the path, but you want to see the path length. So, we know here. So, we do a path length here and we. So, this path length is returning here 2500. So, the costs associated with going from Bangalore to Surat is 2500.

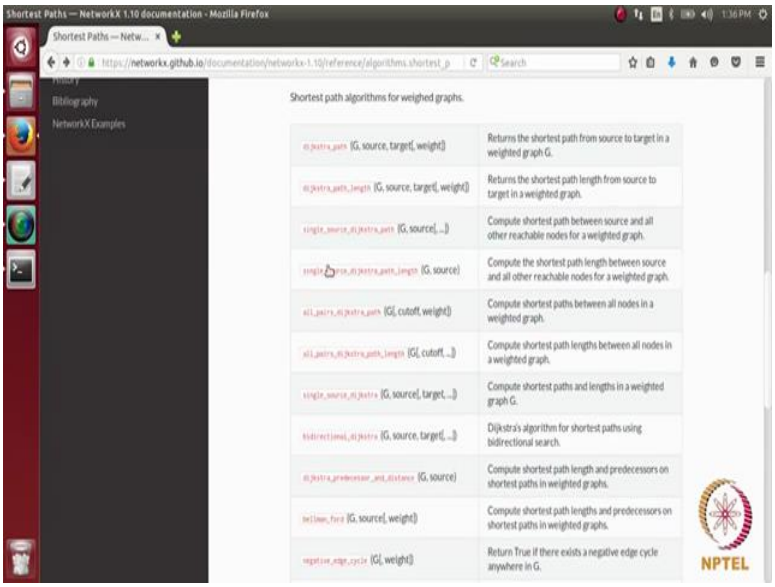
(Refer Slide Time: 30:59)



Why is it turning out to be 2500 let see? So, you let say that if you want to go from Bangalore to Kolkata you can go it in seventeen hundred and let say that you want to go from Kolkata to Surat. So, we can write it this way because it is an undirected graph its 8

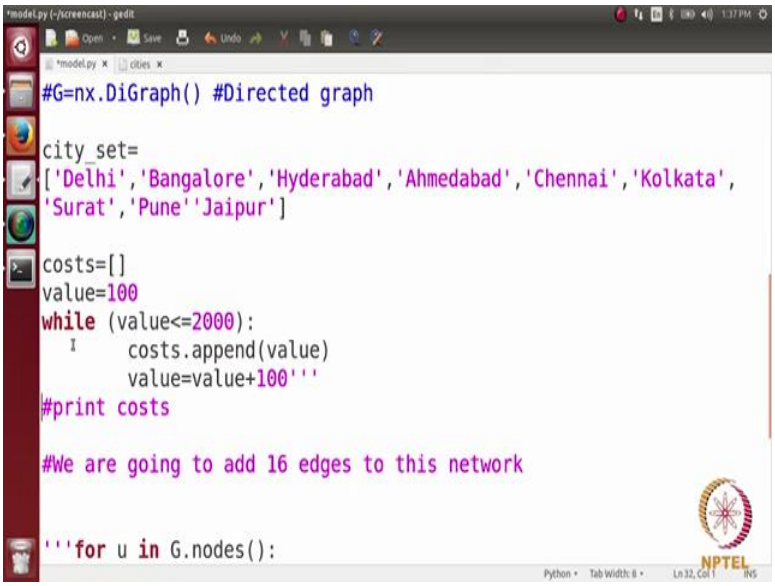
hundred when you some both of these it turns out to be 2500. So, you go from Bangalore to Kolkata which goes you 1700 and then Kolkata to Surat goes you 800 and this some turns out to be 2500 perfect.

(Refer Slide Time: 31:31)



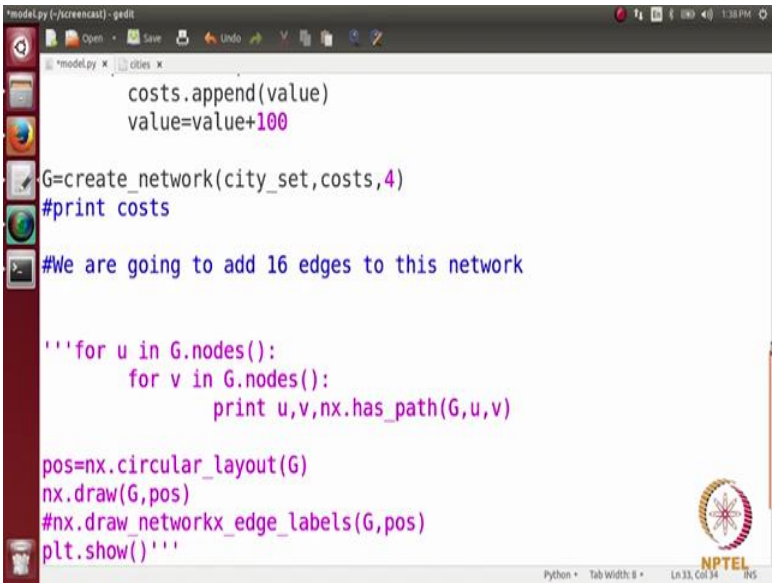
Then we have the single source dijkstra path. So, I will not going to each of this. So, what you can do here is you just give a source node and it will tell you the shortest path from the source node to every then node in the networks. Similarly you can look at its path length and then you have a all pairs extra path which will compute the shortest path between all the nodes in the network and then here also you can look at the path length as well I will just exit.

(Refer Slide Time: 32:05)



Coming back to our core now what we are now what we will do is we are going to plot the curve and let see what the curve is going to do. So, let us set first of all this costs rate to be have this city set here to have this costs here and ok.

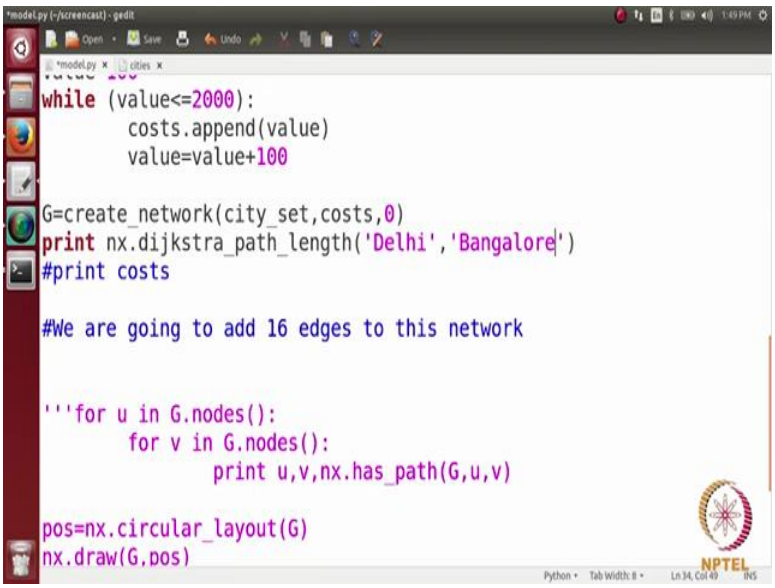
(Refer Slide Time: 32:57)



We do not need the functions and this is for let us (Refer Time: 32:52) it perfect and we are we had to call this function create underscore network and where we are passing the city set and then our costs and then the number of edges which we want to have. So, let us put very less number of edges 4.

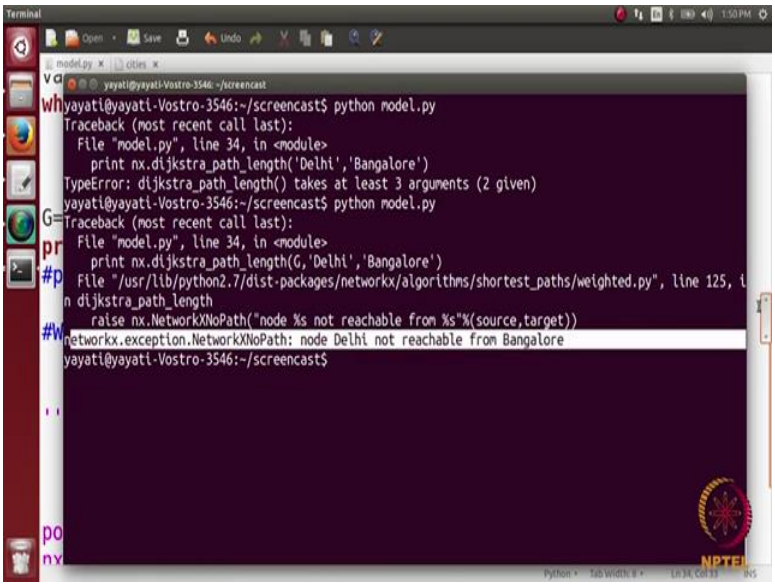
Now, one question what is going to be the path length if there is no connection between 2 nodes. So, we are interested in looking at. So, we have looked at various different extra path finding functions what does this function written if there is no path between a pair of nodes for example, assume that initially we add 0 edges in this network. So, there is no path between any 2 nodes in this networks since there is no edge. So, what happens if I print a; the extra path finding function in this case let us try to do it. So, let us choose to nodes let say Delhi and Bangalore.

(Refer Slide Time: 34:15)



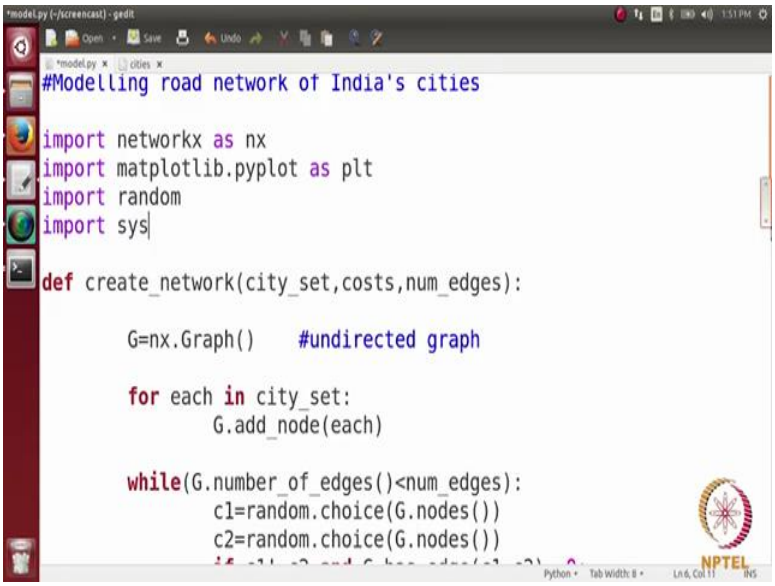
So, what I am going to do is I am going to print n x.dijkstra and then path and then length path length between see Delhi and Bangalore let us always function in a.

(Refer Slide Time: 34:43)



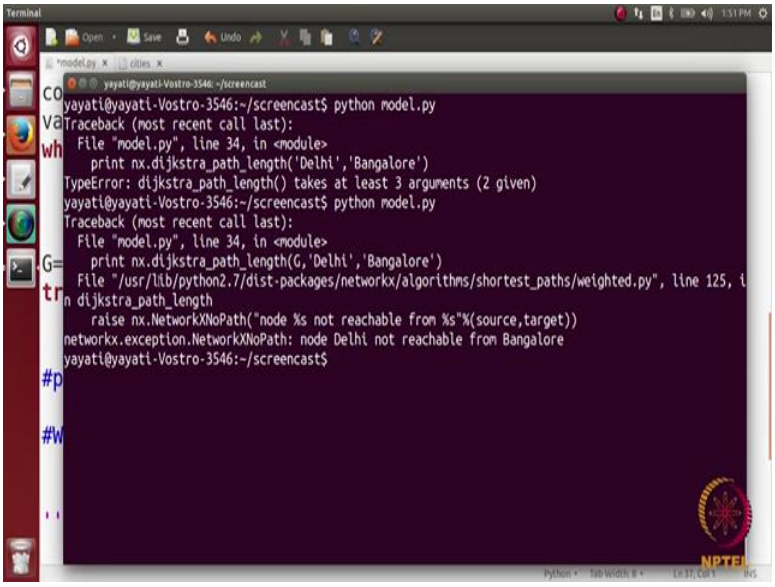
So, it has given a type error that we should have given 3 arguments. So, what is missing here is graph G right nodes run it. So, you see here that there is an exception. So, the code has writtened an exception and the exception is node Delhi not reachable from Bangalore. So, we see that how do we catch these exceptions in the network assume that I do not want to return and exception here rather I want my code to show mw as 0 here at the path length from Delhi to Bangalore not 0, it is infinity, right. So, my answer should be infinity or since a graph is very small let us represent infinity here by a big number let say 10,000. So, we are representing infinity with 10,000 though it is not a very well excepted notion, but we are just using it in this code for the sake of convenience.

(Refer Slide Time: 35:43)



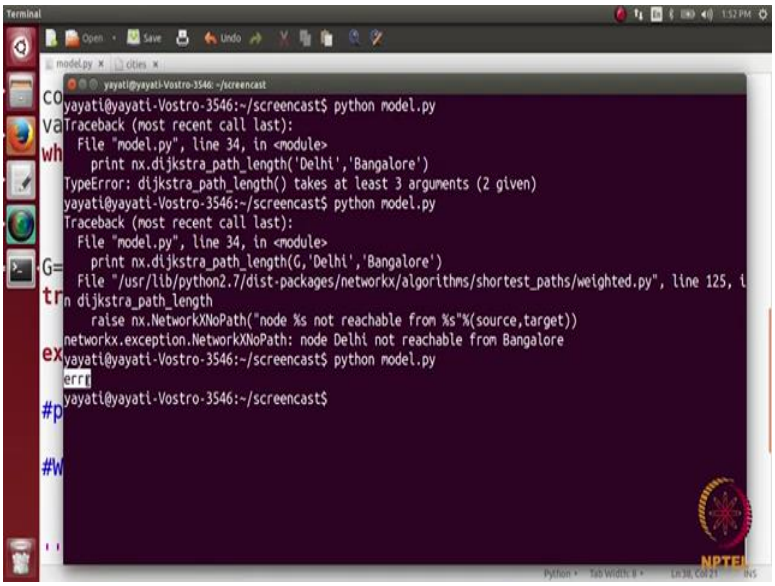
So, what we do this in this case is something called the exception handling which is very simple. So, let us look at we just need to import the module sees here after include importing this package what do I do it I just put the statement inside a try (Refer Time: 35:56).

(Refer Slide Time: 35:53)



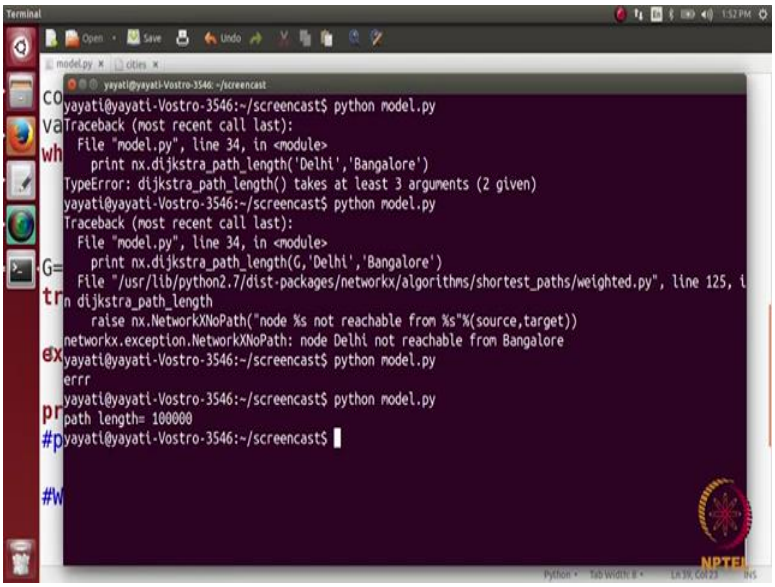
So, what my try (Refer Time: 35:58) does? It gets this path length L as nx extra path length G Delhi Bangalore and in case there is an error. So, we will looked at an error here and these exception handling and this error was a key error means here.

(Refer Slide Time: 36:25)



So, we except in the case of an exception what does we do it let say we print error let see how this piece of code works. So, you see here that they words an exception and this exception was because there was no path and we have printed here error.

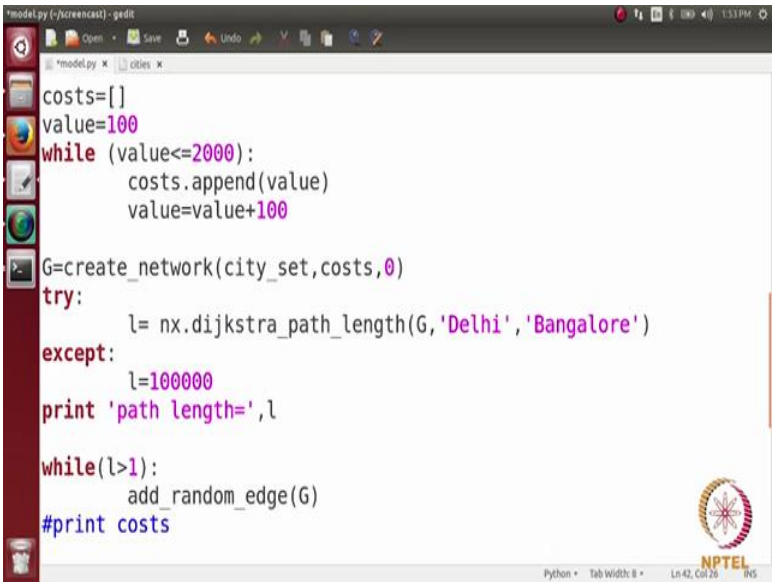
(Refer Slide Time: 36:52)



So, what we can do is instead of printing here error we can set this L to be 10,000; 10,000 we will let say even 100,000 and then we print here path length equals to L and let see here when be run this code again we get here a path length of 100,000. So, what this is doing whenever there is no path between 2 nodes it will written 100,000 which is

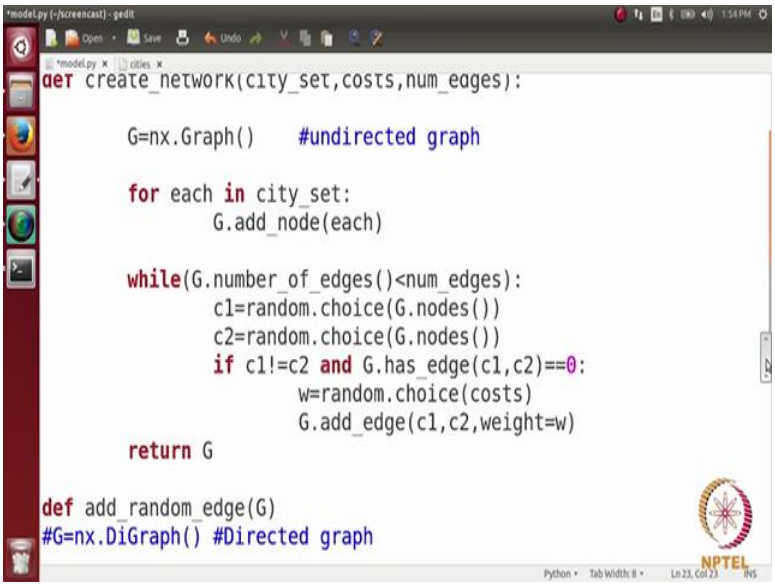
a very big number according to a code according to a network. So, it is a very small network, now what I want to do is what we want to do is we want to see a plot and what will this plot do there we be 2 cities in our network and will be adding random edges and as the at random edges we are interested to see how this path length decreases.

(Refer Slide Time: 37:59)



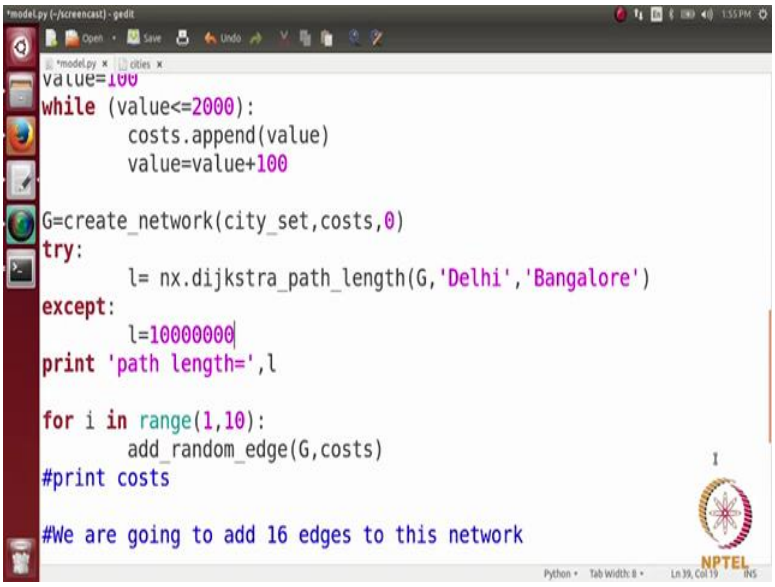
So, let us write a small piece of 4 hold this. So, what we are going to do is initially we have this graph here g. So, what we are going to do is while, we are going to do is we have calculated here I will write and L was here 10,000 what we want L to become is let say one because the minimum path length between 2 nodes is one till they become directly connected. So, till while your L it is greater than one. So, what you do here is you add an edge randomly. So, I will call a function here add random edge right add random edge in G. So, how this function is going to work is.

(Refer Slide Time: 38:50)



Lets create a function here define add random edge G and so we have the city set here.

(Refer Slide Time: 39:08)

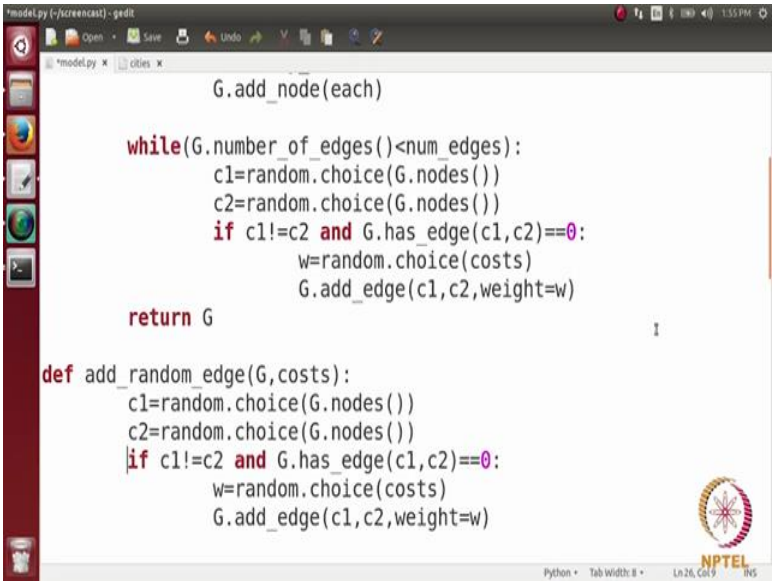


But we want to pass here the costs we goes every node should have a costs one thing here this pass length is actually equal to be cost here right. So, this is equal to a travelling cost.

So, we cannot say it is less than one. So, we will just run this code for some number of steps because path length is never going to be one here because our path length is a travelling cost here. So, what will go is for i in range 1 to 10 you will see for the 10 times

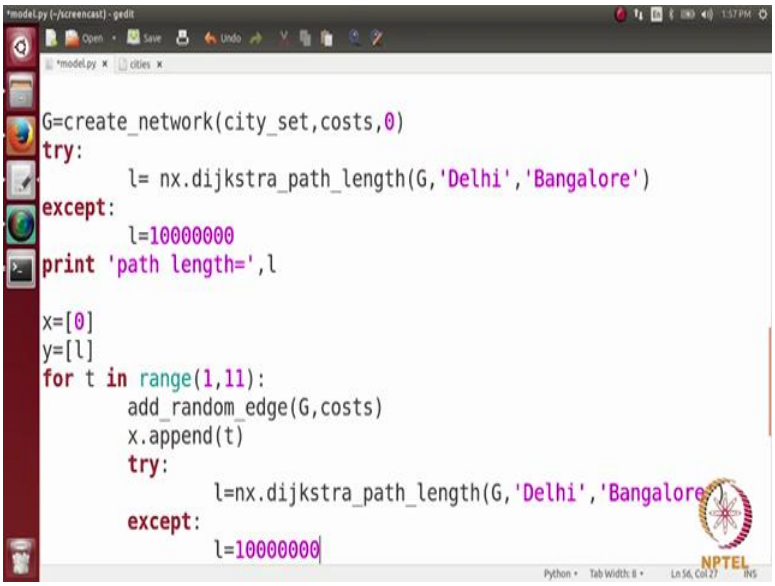
what happens here and let us make it a little bit larger number. So, add random edge G costs.

(Refer Slide Time: 40:06)



So, what it is going to do is what exactly we have done above. So, what we have done is right we have taken 2 random nodes and if there was no end between them we have chosen edge and added between them. So, exactly the same thing we are going to do here. So, essentially we are just copy pasting this code.

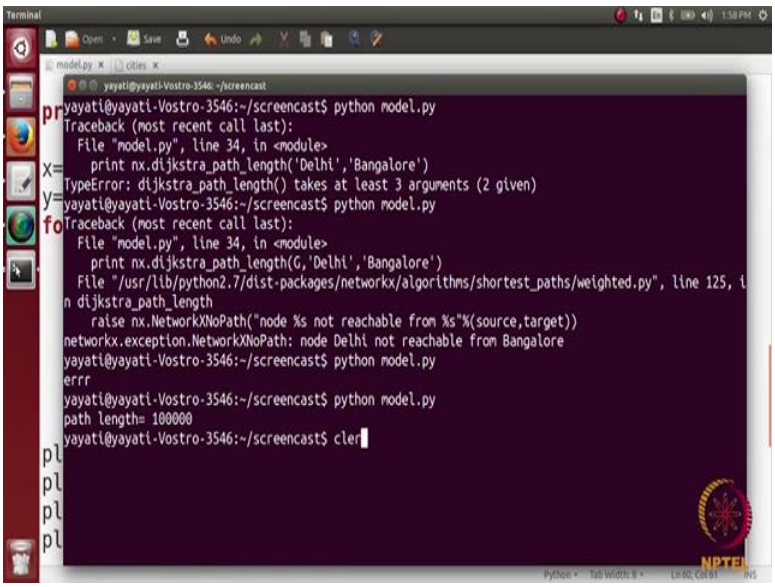
(Refer Slide Time: 40:35)



So, what we are doing is we will choose 2 nodes randomly and if there is no end between these 2 nodes will adder add an edge with their weight being equals to w. So, since done here add random edge G.costs and after that what we are going to do is. So, since we have to plot this graph over 10 timestamps we need 2 arrays 1 is for x axis, 1 is for y axis in the 11. So, x axis is a time which is essentially I let us represent it as t.

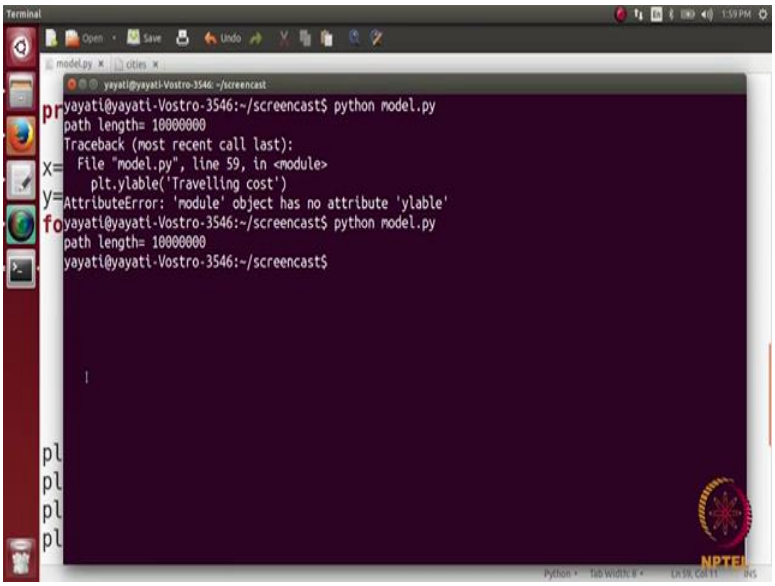
So, we will have an array x and we will have an array y x will be initially having the value 0 right because it is a 0 timestamp when y will initially having the value I which is a very big number code we are going to do here now is x.append t that is a time and in y what we have to opened again if the dijkstra path length. So, it is again going to be a try function. So, we are going to try what give me are going to try is let us say let us taking L; L equals to n x.dijkstra and then path length then G comma Delhi comma Bangalore right and then we are going to except and adder. So, this exception is in case where there is no path between the 2.

(Refer Slide Time: 42:32)



So, here we are going to set L equals to it remains the same which is the big number 1, 2, 3, 4, 5, 6. So, there are 7 0s, perfect and then what we are going to do is y.append L and then we want to plot this. So, for plotting this we have plt.plot and we have x and then y and we can actually give your axis name let us say that plt.x label that is a x axis is a time and plt.y label y axis is a let us say travelling cost and then we have a title let us say change in travelling cost as more roads are added in the network.

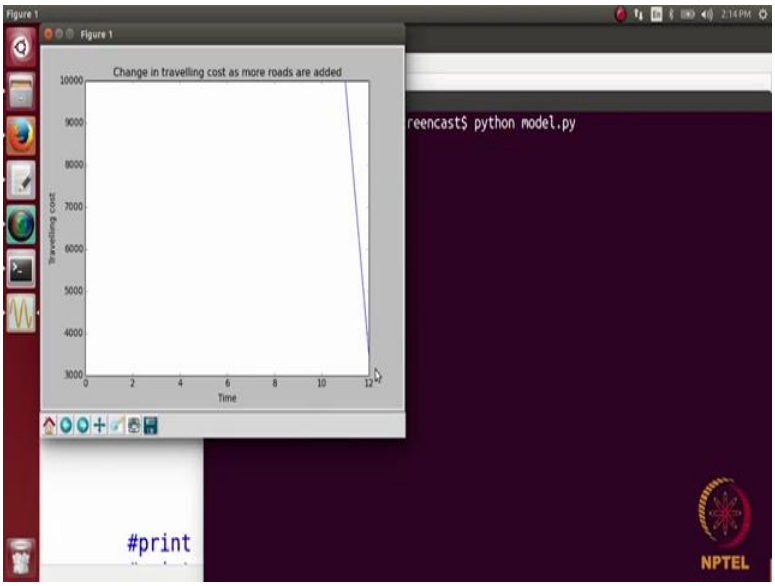
(Refer Slide Time: 43:34)



So, let us try to see. So, clear here the small spelling mistake here. So, I will take their having some problem it is printed are path length. So, here we have mention the small changes to this code. So, the value of L we have just change it to be 10,000 we can just to so that the plot looks nice and should not we where to high, but it is still higher on the path that we have very less as oppose we are just 2500 or something and then this take when y.append L has been shifted.

So, instead of adding at the end we added inside the loop because we have this break statement inside the try block which is needed. So, if this break statement is there and we could y.append at the end. So, the value is not append it. So, we append the values to are array y in between the try block and the added block and then we have a plt road show here which we were which was initially missing so which is required to show us the plot.

(Refer Slide Time: 44:56)

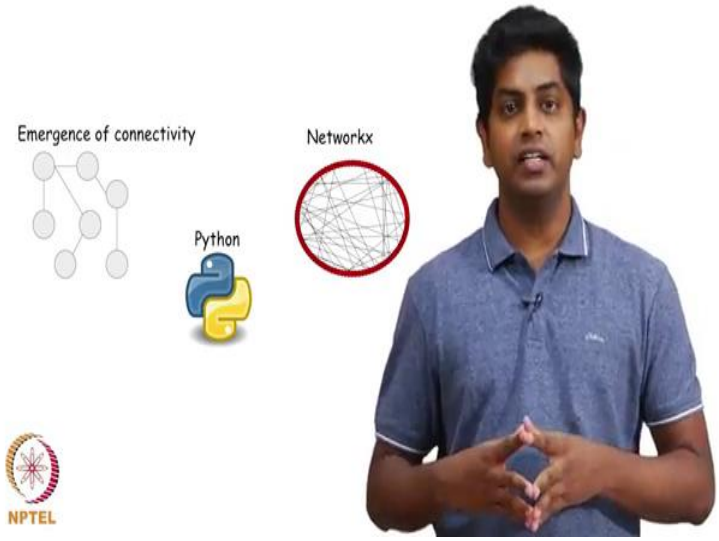


So, let us see here. So, when we execute this code. So, we see here something. So, the initial cost was 10,000. So, this was the cost which was which is actually infinity furors right. So, initially Delhi and Bangalore were not connected and after sometime steps. So, after some 11 time steps, so the random edges are keep creating in the network and after some eleven times steps this cost drops and comes to 3,000. So, it becomes connected here were just a small code. So, it might not be very clear to you. So, if you have any doubt regarding this you can leave it in the discussion forums and we can discuss it further. So, this was just a basing basic plotting function which we can do using python and networkx just an example for that.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

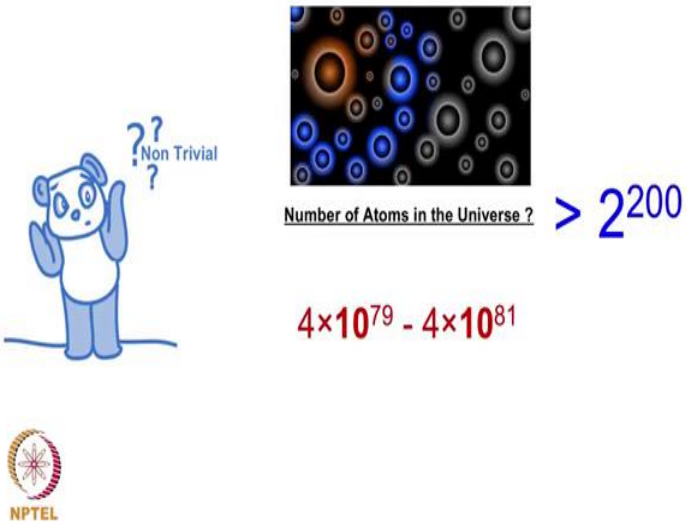
Lecture – 07
Introduction to Social Networks
Social Networks: The Challenge

(Refer Slide Time: 00:07)



We saw very nice emergent property on social networks and I thought you how to use python on network x, now let shift our gas and try looking at something that is very non trivial about social networks.

(Refer Slide Time: 00:24)



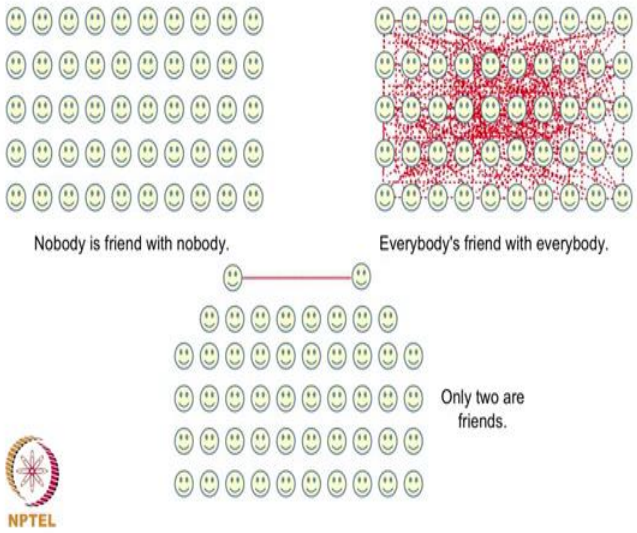
What is a non trivial about social networks? Let me ask a very seemingly unrelated question here; what you think is the atoms in the universe? So, very big number, it is roughly 2 to the 200 plus more than that 2 to the 200 plus is the number of atoms in the universe, so much to be precise. So, what? Observe: look at the example that we just now saw in one of our previous lectures.

(Refer Slide Time: 01:00)



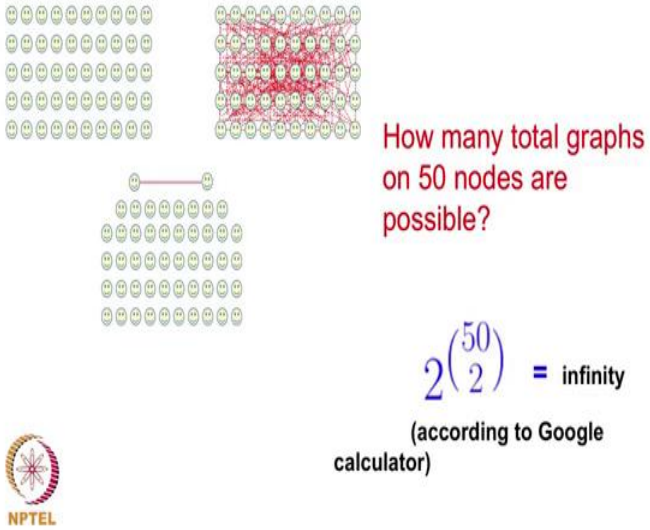
A classroom with 50 people correct and we questioned about friendships there. Now let me ask you this question, what are the total possible friendships that are possible on this graph on just 50 people?

(Refer Slide Time: 01:20)



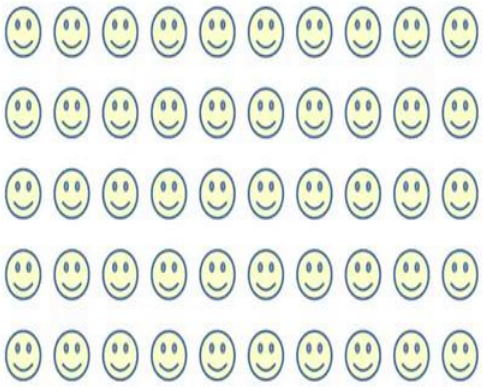
By that I mean look at this, this is one possible friendship where nobody is friends with each other, the other possibilities is all of them are friends with each other, other possibilities there are only these 2 friends rest are not friends so on and so on and so forth.

(Refer Slide Time: 01:39)



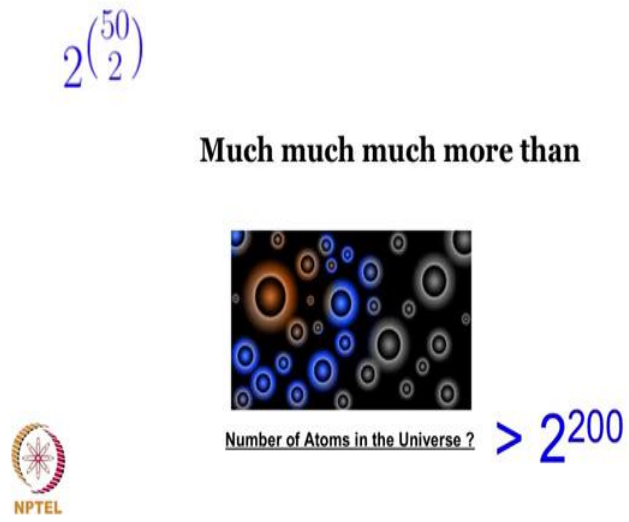
If I go on enumerating all possible graphs on 50 nodes, how many will I get? How do I even answer this question? I leave it as an exercise. I will quickly answer it here. The answer is 2 to the power of 50 joules 2 and that is a very big number and the number is so much, this is the total possible graphs on 50 nodes; just 50 nodes.

(Refer Slide Time: 02:06)



If you take 50 people in this classroom, all of you sitting here 50 people let us say.

(Refer Slide Time: 02:13)



What are all possible graphs that I can think of on the 50 people, the total number of graphs on 50 people is much much more not just more, but a lot lot lot more than the atoms in the universe.

(Refer Slide Time: 02:31)



And that is what makes analyzing social networks very very difficult, the inherent complexity in it is way too much for even a super computed to handle and this is what makes it interesting that despite the fact that there is so much of complexity, we tend to observe some very nice properties.

(Refer Slide Time: 02:41)



I am going to start enumerating this very nice questions and properties that this entire course is going to cover. Let us start with a first such a property rather what I will do right now is I will start off telling you a couple of nice; may not be couple may be 4 to 5 such nice ideas which I will be covering in the course. In fact, the course will have several nice ideas, but the main ideas I am going to tell you right now on what will be covered in the course.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 08
Introduction to Social Networks
Google Page Rank

(Refer Slide Time: 00:06)



Given a friendship network, who is the most important person? How do we judge who is very important? Do you see who has the maximum number of friendships or is there something more than just the number of links that one has?

(Refer Slide Time: 00:27)



In fact, this is the question which yielded 450 billion dollar industry; I am telling you the story of google.

(Refer Slide Time: 00:31)



So, google basically asked this question; given a bunch of web pages that are linked to each other which is the most important web page of all these things?

(Refer Slide Time: 00:44)



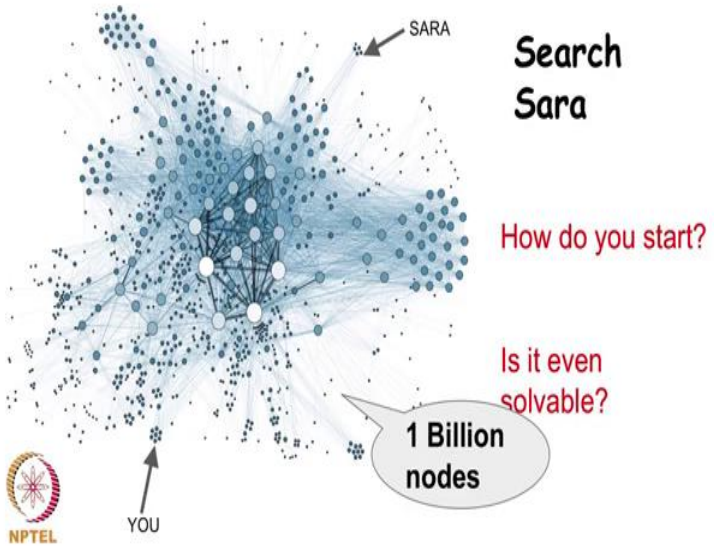
And the answer is so straight forward that when we come to this chapter, you will feel how can such an obvious observation result in 450 billion dollar industry. In fact, it might appear to be very obvious or it took team of 2 people to bring this to the lime light and use this seemingly simple concept to solve one of the most important problems in computer science which is how do you rank nodes in a network.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 09
Introduction to Social Networks
Searching in a Network

So, second in the list is the problem of searching in a network, in a network with roughly let say one billion nodes.

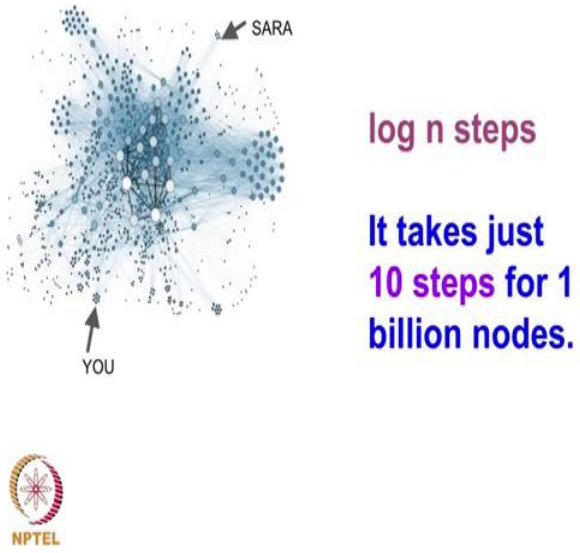
(Refer Slide Time: 00:17)



If I were to tell you go and search for a particular node starting from some random node, how will you even start? Can this even be solved? I told you there are so many possible networks that one can think of on even 15 node graph, and I am giving you a 1 billion node network; a network is 1 billion nodes, how can you search for an node here?

Well such complicated questions very interestingly they have very simple and straightforward answers. In fact, we are going to show that searching on a network especially on a real world network is very easy.

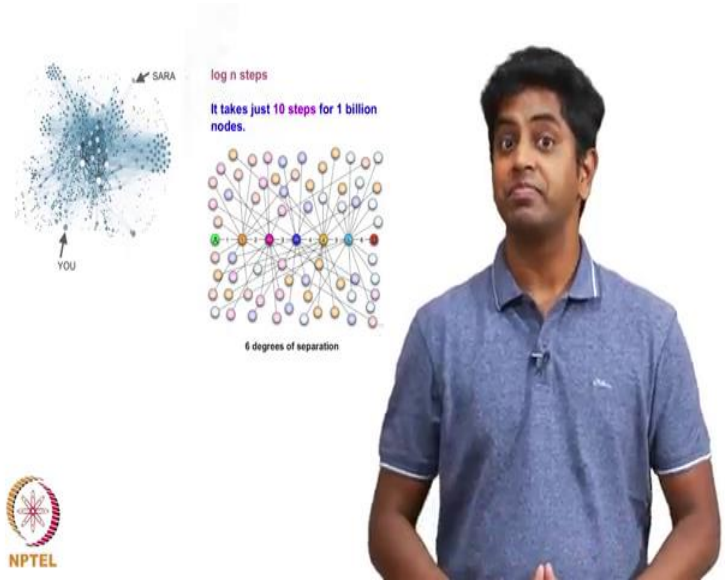
(Refer Slide Time: 01:02)



In fact, if you have n number of nodes you can search with only roughly log n number of steps to be precise actually its log square n, but the point is in log number of transactions you can achieve searching, I am sure you know what one means by logarithm, if n is the number log of n is the number of digits in n.

So, if the number of nodes in a network is n let us say 1 billion, the effort involved in searching in this big network is the number of digits in one billion single digit number or let say maximum 10, 15 or even 20, in 20 transactions you can find any node in the network, why? How?

(Refer Slide Time: 01:50)



We exploit a nice property in such networks called the small world phenomena; gorgon for the time being, but do not worry when we come there you will see it is a very straightforward concept, but very elegant very beautiful and sparkling part is that it gives you an answer like a charm.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 10
Introduction to Social Networks
Link Prediction

Next in the list is something that is actually not believable; what do you know about predictions?

(Refer Slide Time: 00:17)



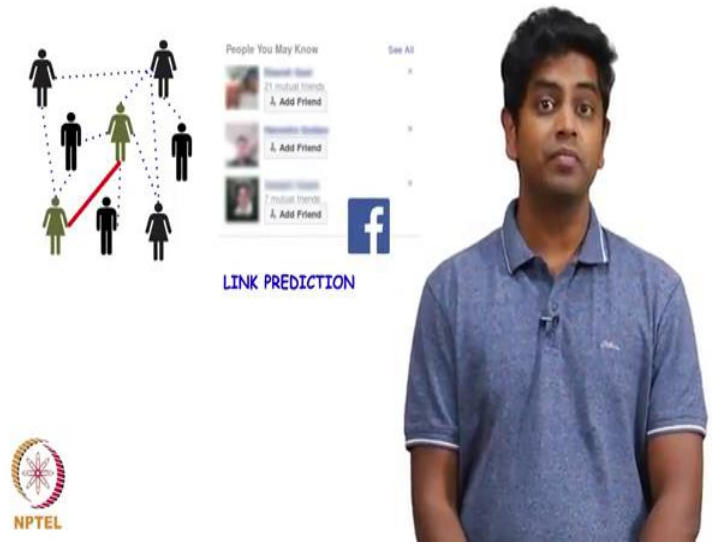
That someone come and tell you that show me your hand and I will tell you, what is your future? Your future is hidden here. I do not know the signs of it. It is may be it is true, it is not true, very debatable.

(Refer Slide Time: 00:31)



But I can assure you of the fact that predictions on networks work really really well, what do I mean by this?

(Refer Slide Time: 00:36)



I will look at your friendship network and tell you who could possibly be your next friend sounds like this is very familiar to you right, what am I talking about? Have you heard of he could be your prospective friend? Friend suggestions in Facebook; it is not a straight forward method that they use it, do not show a random person who has a lot of

common friends with you, there is sophisticated algorithm which is in play there what is that algorithm, we will discuss that in this chapter called link prediction.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 11
Introduction to Social Networks
The Contagions

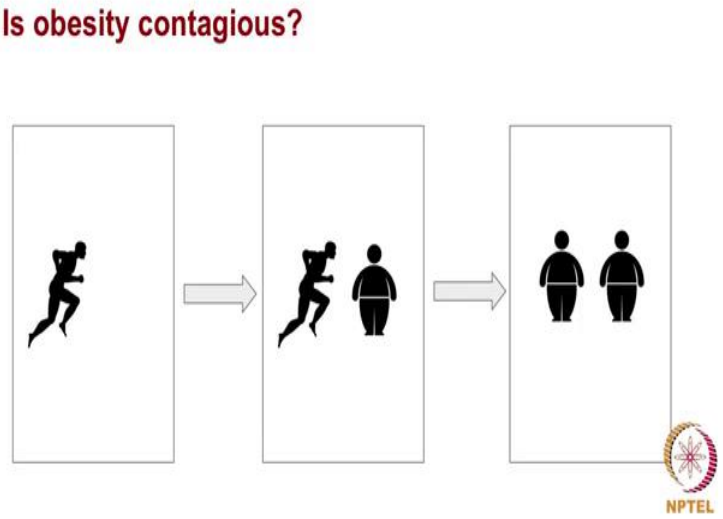
Later on we will impact look at very interesting humanly questions, what do I mean by humanly questions?

(Refer Slide Time: 00:18)



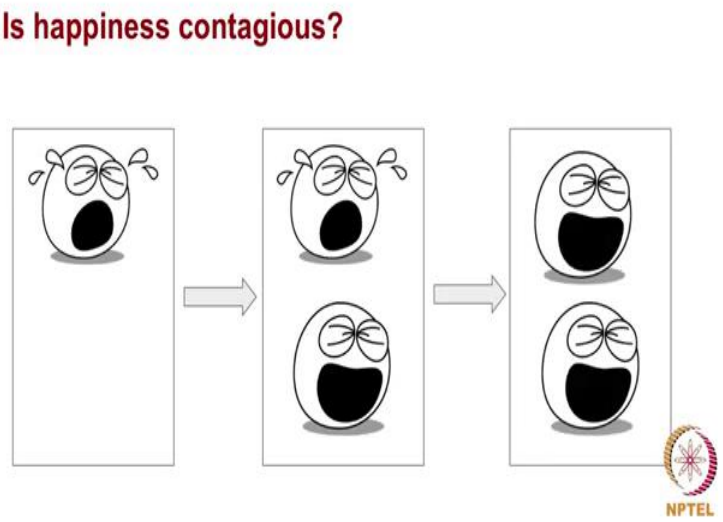
We are in fact, going to answer this so called philosophical questions with very elegant computing computer science and mathematics; what are those questions?

(Refer Slide Time: 00:25)



Do you think obesity is contagious do you think if you are friends with the person who is obese you run into the danger of becoming obese?

(Refer Slide Time: 00:36)



Do you think if you are surrounded by happy people? You will be happy too. In fact, you will be surprised to note that happiness is not just contagious to one level, it is contagious to 2-3 levels by that I mean your friend's friend; if he is happy you will be happy; sounds like cooked up story, no it is not true, this is a very well established scientific fact, we will be seeing such questions and their answers.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 12
Introduction to Social Networks
Importance of Acquaintances

(Refer Slide Time: 00:12)





Very interesting last part which would be a nice moral for all of you on the hunt for a good job is how to get a good job. What kind of friends should you make in order to get a good job?

(Refer Slide Time: 00:27)



This comes under the topic of what is called the strength of weak ties. It is believed that your acquaintances; not close friends; acquaintances friends were not so very close, but yeah they are friends, those kind of people are actually very important for you to get your dream job or only a dream job, your prospective spouse is also a recommendation given by a far away person and acquaintances not a close friend, the strength of weak ties. We will see more of it when we encounter this idea in the forth coming chapters.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

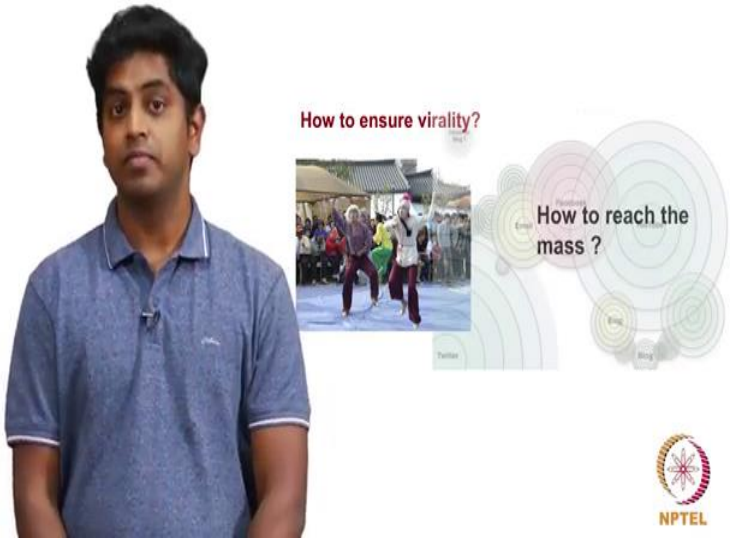
Lecture – 13
Introduction to Social Networks
Marketing on Social Networks

(Refer Slide Time: 00:19)



Lastly, how do you market a product on social networks of course, the era of putting advertisements on let us say notice board or let us say a TV commercial is long gone, today people use social networks to market their product.

(Refer Slide Time: 00:30)



What is the science of this? How do you ensure that your product becomes viral or a piece of video becomes viral? What should you note rather what should you do in order to market something so that it reaches the mass? We are going to discuss the whole lot about this and we are also going to see the sort of link between how do you market a product to how does a belief propagate, how does a piece of information propagate, how do things cascade so on and so forth.

So, these are all just some of the cool ideas that we will see in the course. In fact, the course is some 5-6 times more than what I just now introduced. These are some of the cool ideas that we will be seeing, there are lot more ideas coming up.